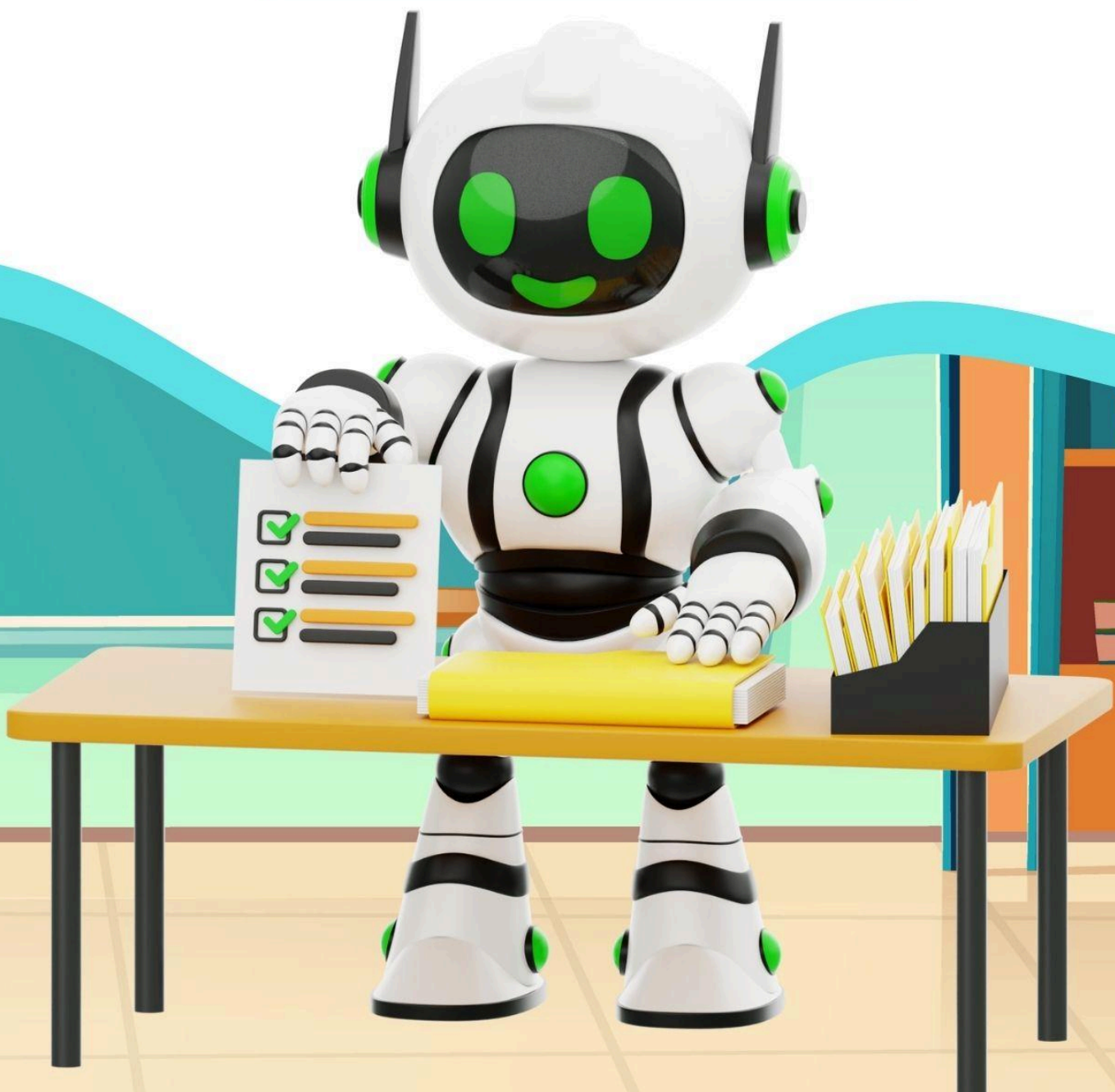


Proyek Akhir – RAG Sekolah & AI End-to-End

untuk SMK dan umum



2025

Kelas 12 Semester 2

Proyek Akhir – RAG Sekolah & AI End-to-End

Onno W. Purbo

**Institut Teknologi Tangerang Selatan (ITTS)
OnnoCenter
2025**

Lisensi & Catatan Karya

Karya ini dilisensikan di bawah lisensi **Creative Commons Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)**.

Artinya, **siapa pun bebas untuk menggunakan, menyalin, membagikan, dan mengadaptasi** materi ini, **dengan syarat**:

- **Memberikan atribusi yang sesuai** (menyebut sumber asli),
- **Menyertakan lisensi yang sama** jika dimodifikasi atau dikembangkan lebih lanjut.

Lihat detail lisensi di sini <https://creativecommons.org/licenses/by-nd/4.0/legalcode.id>

Desain Cover: Irwan Siswanto

Disclaimer

Materi pembelajaran ini dibuat dengan **dana swadaya masyarakat Indonesia** dan **kontribusi sukarela dari para dosen di Institut Teknologi Tangerang Selatan (ITTS)**. Karya ini didistribusikan secara bebas untuk mendukung pendidikan digital dan kecerdasan buatan di kalangan pelajar Indonesia.

Kami **memohon maaf** jika terdapat kekurangan dalam isi maupun penyajian materi ini. Kritik dan saran sangat kami harapkan untuk terus menyempurnakan karya serupa di masa depan.

Kata Pengantar

Buku ini disusun sebagai wujud nyata kepedulian sivitas akademika Institut Teknologi Tangerang Selatan (ITTS) terhadap perkembangan pendidikan kejuruan di Indonesia, khususnya bagi siswa Sekolah Menengah Kejuruan (SMK). Di tengah kemajuan teknologi yang begitu pesat, kami terdorong untuk menghadirkan materi pembelajaran **yang menantang, aplikatif, dan relevan dengan dunia industri digital**, agar siswa dapat mencapai **kemampuan maksimalnya** dalam memahami dan membangun sistem *Artificial Intelligence (AI)*.

Perlu kami sampaikan bahwa **materi dalam buku ini menuntut tingkat pemahaman teknis yang cukup tinggi**, terutama karena pembahasannya **sangat kental dengan praktik coding menggunakan bahasa Python** untuk membuat dan melatih model AI secara mandiri. Oleh karena itu, **tidak semua siswa, sekolah, maupun guru mungkin dapat langsung menyerap seluruh isi buku ini secara optimal**.

Bagi pembaca yang masih merasa kesulitan mengikuti langkah-langkah teknis yang disajikan, **kami sangat menyarankan untuk terlebih dahulu mempelajari buku “AI untuk SMA”**, yang disusun dengan pendekatan yang lebih **sederhana dan bertahap** sebelum melanjutkan ke tingkat SMK ini.

Penyusunan dan penerbitan buku ini dilakukan secara mandiri dengan semangat berbagi pengetahuan terbuka. Kami mengucapkan terima kasih kepada berbagai pihak yang telah mendukung dan memberi masukan dalam proses penulisan. Kritik dan saran yang membangun sangat kami harapkan agar edisi berikutnya dapat semakin baik dan mudah diakses oleh seluruh siswa Indonesia.

Kami berharap buku ini dapat menjadi **jembatan antara teori dan praktik AI**, sekaligus menginspirasi siswa SMK untuk **berpikir kreatif, logis, dan bertanggung jawab dalam menggunakan teknologi**. Dengan rendah hati, kami mohon doa dan dukungan agar karya sederhana ini dapat memberi manfaat luas dan menjadi amal jariyah bagi semua pihak yang terlibat.

Tangerang Selatan, Oktober 2025

Penulis

Daftar Isi

Disclaimer.....	2
Kata Pengantar.....	3
Daftar Isi.....	4
BAB 1: Desain Sistem AI End-to-End (Input → Proses → Output).....	8
Tujuan Pembelajaran.....	8
Peta Konsep (Sederhana).....	8
Apersepsi (Pendek, Fun, Menantang).....	8
Penjelasan Konsep.....	9
1. Input (Data Mentah).....	9
2. Preprocessing (Pembersihan Data).....	9
3. Model (Otak AI).....	10
4. Output (Hasil AI).....	10
5. Evaluasi (Pengecekan).....	10
Intinya.....	10
Contoh Kasus atau Ilustrasi.....	11
Contoh Soal dan Pembahasan.....	14
Fakta Menarik (Fun Facts).....	18
Tips Belajar atau Tips Cepat Menghafal.....	18
Aktivitas Siswa.....	19
Rangkuman.....	24
Latihan Soal.....	24
Tugas Proyek.....	29
BAB 2: Integrasi Fine-Tuned Model dengan RAG Lokal.....	34
Tujuan Pembelajaran.....	34
Peta Konsep (yang sederhana).....	34
Apersepsi (Pembuka yang Fun).....	34
Penjelasan Konsep.....	35
Apa itu Fine-Tuning?.....	35
Apa itu RAG?.....	35
Kombinasi Fine-Tuning + RAG.....	35
Analogi Sederhana.....	35
Contoh Sederhana dengan Python.....	36
Intinya.....	36
Contoh Kasus atau Ilustrasi.....	36
Contoh Soal dan Pembahasan.....	39
Fakta Menarik (Fun Facts).....	43
Tips Belajar / Tips Cepat Menghafal.....	43
Aktivitas Siswa.....	44
Rangkuman.....	48
Latihan Soal.....	48
Tugas Proyek.....	53
BAB 3: Membangun Sistem RAG Sekolah (Dokumen Kurikulum, Buku Pelajaran).....	57

Tujuan Pembelajaran.....	57
Peta Konsep (Sederhana dan Visual).....	57
Apersepsi.....	57
Penjelasan Konsep.....	58
Apa itu RAG?.....	58
Bagaimana caranya?.....	58
Contoh Kasus.....	59
Kenapa Penting?.....	59
Mix Fun + Point untuk Siswa SMK.....	59
Contoh Kasus.....	59
Contoh Soal dan Pembahasan.....	63
Fakta Menarik (Fun Facts).....	66
Tips Belajar / Tips Cepat Menghafal.....	66
Aktivitas Siswa.....	67
Latihan Soal.....	71
Tugas Proyek.....	76
BAB 4: Deployment Lokal di Lab Sekolah (Tanpa Internet).....	81
Tujuan Pembelajaran.....	81
Peta Konsep.....	81
Apersepsi (Pembuka yang Fun dan Menarik).....	81
Penjelasan Konsep.....	82
Kenapa Penting?.....	82
Teknologi Open Source yang Dipakai.....	82
Cara Instalasi di Ubuntu 24.04 Server.....	82
Ilustrasi Sederhana.....	83
Fun Point untuk Siswa.....	83
Contoh Kasus atau Ilustrasi.....	84
Contoh Soal dan Pembahasan.....	88
Rangkuman Soal.....	90
Catatan.....	91
Fakta Menarik (Fun Facts).....	91
Tips Belajar / Tips Cepat Menghafal.....	91
Aktivitas Siswa.....	92
Rangkuman.....	95
Latihan Soal.....	96
Tugas Proyek.....	100
BAB 5: Dokumentasi & Evaluasi Sistem AI.....	104
Tujuan Pembelajaran.....	104
Peta Konsep (Sederhana).....	104
Apersepsi.....	104
Penjelasan Konsep.....	105
Apa itu Dokumentasi?.....	105
Apa itu Evaluasi?.....	105
Ilustrasi Kasus.....	106

Kenapa Penting?.....	106
Mix Fun untuk Siswa SMK.....	106
Contoh Kasus atau Ilustrasi.....	106
Contoh Soal dan Pembahasan.....	109
Fakta Menarik (Fun Facts).....	111
Tips Belajar / Tips Cepat Menghafal.....	112
Aktivitas Siswa.....	112
Rangkuman.....	116
Latihan Soal.....	116
Tugas Proyek.....	120
BAB 6: Presentasi & Kolaborasi Tim.....	124
Tujuan Pembelajaran.....	124
Peta Konsep.....	124
Apersepsi (Pemantik Awal).....	124
Penjelasan Konsep.....	125
Apa itu Kolaborasi?.....	125
Apa itu Presentasi?.....	125
Ilustrasi Kasus.....	125
Kenapa Penting?.....	126
Mix Fun untuk Siswa SMK.....	126
Contoh Kasus dan Ilustrasi.....	126
Contoh Soal dan Pembahasan.....	129
Fakta Menarik (Fun Facts).....	131
Tips Belajar & Tips Cepat Menghafal.....	132
Aktivitas Siswa.....	132
Rangkuman.....	135
Latihan Soal.....	135
Tugas Proyek.....	139
BAB 7: Refleksi Etika & Dampak Sosial AI.....	142
Tujuan Pembelajaran.....	142
Peta Konsep Sederhana.....	142
Apersepsi (Pemantik Pikiran).....	142
Penjelasan Konsep.....	143
Apa itu Etika AI?.....	143
Dampak Sosial AI.....	143
Ilustrasi Kasus.....	143
Kenapa Harus Refleksi?.....	144
Mix Fun untuk Siswa SMK.....	144
Contoh Kasus atau Ilustrasi.....	144
Contoh Soal dan Pembahasan.....	146
Tips Belajar & Tips Cepat Menghafal.....	148
Aktivitas Siswa.....	149
Rangkuman.....	151
Latihan Soal.....	151

Tugas Proyek.....	155
BAB 8: Proyek Akhir – AI Sekolah Cerdas.....	158
Tujuan Pembelajaran.....	158
Peta Konsep.....	158
Apersepsi.....	158
Penjelasan Konsep.....	159
Integrasi Semua Bab.....	159
Ilustrasi Kasus.....	159
Kenapa Proyek Ini Penting?.....	160
Mix Fun untuk Siswa SMK.....	160
Contoh Kasus atau Ilustrasi.....	160
Contoh Soal dan Pembahasan.....	163
Fakta Menarik / Fun Facts.....	166
Tips Belajar & Tips Cepat Menghafal.....	167
Aktivitas Siswa.....	167
Latihan Soal.....	171
Tugas Proyek.....	175
Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.....	179

BAB 1: Desain Sistem AI End-to-End (Input → Proses → Output)

Tujuan Pembelajaran

Bayangkan kamu membangun “pabrik AI” mini di sekolah. Kita belajar alurnya dari awal sampai akhir, pakai tool yang bebas (*open source*).

- Memahami alur **end-to-end**: *Input* → *Preprocessing* → *Model* → *Output* → *Evaluasi*.
- Menjelaskan peran tiap tahap dengan contoh sederhana di sekolah.
- Menghubungkan tool *open source* dengan tahap yang tepat: **Python** untuk logika, **Pandas** untuk rapikan data, *machine learning* untuk belajar pola, **Keras** untuk *neural network*.
- Menentukan *output* yang bermanfaat (contoh: prediksi sederhana, jawaban chatbot).
- Menyebut metrik evaluasi dasar dengan bahasa mudah (akurasi itu seberapa sering benar).
- Menumbuhkan kebiasaan kerja rapi: cek data, uji hasil, perbaiki kalau salah.

Peta Konsep (Sederhana)

Bayangkan seperti jalur produksi yang rapi.

Setiap kotak punya tugas. Jalannya satu arah. Mudah dicek.

Input → Preprocessing → Model → Output → Evaluasi

ini disebut pipeline, jalur kerja dari ujung ke ujung (end-to-end).

Apersepsi (Pendek, Fun, Menantang)

Bayangkan **AI = pabrik pintar**. Pertanyaan pemantik:

- Kalau AI itu pabrik, **bahan mentah** kamu apa? (petunjuk: data)
- **Mesin** di pabrik itu apa? (petunjuk: *model*, *neural network*)
- **Produk jadi** yang mau kamu keluarkan apa? (petunjuk: prediksi/jawaban)
- Bagaimana kamu **cek kualitas** produk? (petunjuk: akurasi sederhana)

Contoh cepat:

Kamu ingin tahu siapa yang perlu **remedial**.

- **Input**: nilai tugas + absensi.
- **Preprocessing**: rapikan nilai kosong dengan **Pandas**.
- **Model**: *machine learning* sederhana menandai “aman” / “butuh bantuan”.
- **Output**: daftar siswa prioritas.
- **Evaluasi**: cek, bener nggak daftar itu sesuai kenyataan di kelas?

Penjelasan Konsep

Bayangkan sistem **AI** itu seperti sebuah **pabrik pintar**.

- Di awal, ada **bahan mentah** yang masuk (input).
- Di tengah, ada mesin yang mengolah (proses).
- Di akhir, keluar hasil jadi (output).

Sama halnya dengan AI, alurnya adalah: **Input** → **Preprocessing** → **Model** → **Output** → **Evaluasi**. Yuk kita bedah satu per satu biar gampang dipahami.

1. Input (Data Mentah)

Input adalah **bahan mentah**. Data bisa berupa angka, teks, gambar, atau suara. Contoh di sekolah:

- Data absensi siswa.
- Nilai tugas atau ulangan.
- Riwayat ekstrakurikuler.

Semua data ini belum rapi, jadi butuh diproses lebih dulu.

2. Preprocessing (Pembersihan Data)

Preprocessing adalah **cuci-cuci data**.

Data mentah biasanya berantakan: ada yang kosong, salah tulis, atau tidak konsisten. Dengan *Python + Pandas*, kita bisa:

- Menghapus data kosong.
- Menyusun ulang tabel.
- Normalisasi nilai (misalnya semua skor jadi 0–100).

Contoh sederhana pakai Python:

```
import pandas as pd

# Data nilai mentah
data = {"Nama": ["Budi", "Ani", "Siti"],
        "Nilai": [85, None, 90]}
df = pd.DataFrame(data)

# Isi nilai kosong dengan rata-rata
df["Nilai"].fillna(df["Nilai"].mean(), inplace=True)

print(df)
```

Outputnya: data jadi rapi dan siap dipakai model.

3. Model (Otak AI)

Model adalah **mesin pintar**.

Di sinilah *machine learning* atau *neural network* bekerja.

Contoh:

- Model prediksi nilai ujian berdasarkan absensi.
- Model chatbot sekolah dengan dataset FAQ.

Kalau pakai *Keras* (library *open source*), kita bisa bikin model sederhana:

```
from keras.models import Sequential
from keras.layers import Dense

# Buat model jaringan saraf sederhana
model = Sequential()
model.add(Dense(8, input_dim=2, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

4. Output (Hasil AI)

Output adalah **produk jadi** dari pabrik.

Contoh:

- Prediksi “Budi kemungkinan lulus UKK dengan nilai A.”
- Chatbot menjawab “UKK dilaksanakan bulan Maret.”

Output ini nantinya dipakai guru, siswa, atau staf sekolah.

5. Evaluasi (Pengecekan)

Evaluasi adalah **kontrol kualitas**.

Kita harus tahu apakah hasil AI benar atau ngawur.

Caranya dengan metrik seperti:

- *Accuracy* (akurasi).
- *Precision* (ketepatan).
- *Recall* (kelengkapan).

Kalau hasilnya masih jelek, berarti model harus diperbaiki.

Intinya

- **Input** = data mentah.
- **Preprocessing** = membersihkan data.

- **Model** = otak AI yang belajar pola.
- **Output** = hasil prediksi atau jawaban.
- **Evaluasi** = cek apakah model sudah bagus.

Dengan memahami alur ini, siswa SMK bisa merancang sistem AI **end-to-end** sederhana, misalnya untuk **prediksi nilai**, **chatbot sekolah**, atau **analisis data absensi**.

Contoh Kasus atau Ilustrasi

Soal 1 — Pemetaan Alur Pabrik AI

Teks Soal:

Sekolah ingin memprediksi **siapa yang perlu remedial**. Kamu punya data: *nilai tugas*, *nilai kuis*, *absensi*. Petakan bagian alur: **Input**, **Preprocessing**, **Model**, **Output**, **Evaluasi**.

Jawaban Singkat:

- **Input**: nilai tugas, nilai kuis, absensi.
- **Preprocessing**: rapikan nilai kosong, skala 0–100 dengan **Pandas**.
- **Model**: *machine learning* sederhana (misal Logistic Regression) atau *neural network* kecil (**Keras**).
- **Output**: daftar siswa “aman / butuh bantuan”.
- **Evaluasi**: akurasi, precision, recall.

Pembahasan (kenapa begitu):

Data mentah = **Input**. Kita cuci dulu = **Preprocessing**.

Lalu “otak” belajar pola = **Model**.

Model mengeluarkan keputusan = **Output**.

Terakhir cek kualitas = **Evaluasi**. Inilah *pipeline* end-to-end.

Pseudocode:

```
INPUT = load_csv("nilai.csv") // tugas, kuis, absensi
DATA = preprocess(INPUT)      // isi kosong, skala 0-100
MODEL = train(DATA.fitur, DATA.label)
PRED = MODEL.predict(DATA.fitur_uji)
METRIK = evaluate(DATA.label_uji, PRED) // akurasi, precision, recall
tampilkan(PRED, METRIK)
```

Soal 2 — Preprocessing: Isi Nilai Kosong dengan Rata-Rata

Teks Soal:

Daftar nilai: [85, None, 90, None, 80].

Isi nilai kosong (*None*) dengan **rata-rata** nilai yang ada. Berapa daftar akhirnya?

Jawaban Singkat:

Nilai yang ada: 85, 90, 80 → jumlah 255, dibagi 3 = **85**.

Hasil akhir: **[85, 85, 90, 85, 80]**.

Pembahasan:

Ini tugas *preprocessing*. Mengisi kosong membuat data rapi.

Di **Pandas**, bisa pakai `fillna(mean)`.

Pseudocode:

```
data = [85, None, 90, None, 80]
valid = [85, 90, 80]
mean = sum(valid) / len(valid) // 255 / 3 = 85
for i in range(len(data)):
    if data[i] is None:
        data[i] = mean
return data // [85, 85, 90, 85, 80]
```

Soal 3 — Rule-Based Sentiment (Baseline Cepat)**Teks Soal:**

Tentukan sentimen kalimat: **“Filmnya keren tapi alurnya jelek.”**

Pakai kamus mini:

- Positif = {keren, bagus, mantap, seru}
- Negatif = {jelek, buruk, kecewa, parah}

Jawaban Singkat: Netral (positif = 1, negatif = 1).

Pembahasan:

Metode *lexicon rule-based* menghitung kata positif vs negatif.

Jika seri → **netral**. Ini baseline cepat sebelum *machine learning*.

Pseudocode:

```
POS = {"keren", "bagus", "mantap", "seru"}
NEG = {"jelek", "buruk", "kecewa", "parah"}

function labelkan(teks):
    kata = to_lower(teks).split()
    p = hitung(kata ∩ POS)
    n = hitung(kata ∩ NEG)
    if p > n: return "positif"
    if n > p: return "negatif"
    return "netral"

labelkan("Filmnya keren tapi alurnya jelek.") // netral
```

Soal 4 — Hitung Metrik: Precision, Recall, F1, Accuracy

Teks Soal:

Untuk kelas **positif** sebagai *positive class*, didapat:
TP=30, FP=10, FN=5, TN=55 (total 100).
Hitung **precision**, **recall**, **F1**, dan **accuracy**.

Jawaban Singkat (angka):

- Precision = $30 / (30+10) = \mathbf{0,75}$ (75%)
- Recall = $30 / (30+5) = \mathbf{0,857}$ ($\approx 85,7\%$)
- F1 = **0,80** (tepatnya 0,800)
- Accuracy = $(30+55) / 100 = \mathbf{0,85}$ (85%)

Pembahasan (ringkas & rapi):

- *Precision* = seberapa tepat saat model bilang “positif”.
- *Recall* = seberapa banyak “positif” yang tertangkap.
- F1 = gabungan keduanya → stabil saat data tak seimbang.
- *Accuracy* = proporsi benar dari semua data.

Pseudocode:

```
TP=30; FP=10; FN=5; TN=55
precision = TP / (TP + FP)           // 30/40 = 0.75
recall    = TP / (TP + FN)           // 30/35 ≈ 0.857
F1         = 2*precision*recall / (precision + recall) // 0.80
accuracy   = (TP + TN) / (TP+FP+FN+TN) // 85/100 = 0.85
cetak(precision, recall, F1, accuracy)
```

Soal 5 — Pilih Model Sesuai Kondisi (Kecil, Offline, Cepat)**Teks Soal:**

Kamu punya data **kecil** (200 komentar berlabel). Komputer sekolah **offline**.
Pilih strategi yang cocok dan jelaskan singkat alasannya.

Jawaban Singkat:

- Mulai dari **TF-IDF + Logistic Regression** (*scikit-learn*).
- Bandingkan dengan **rule-based lexicon** sebagai baseline.
- Jika butuh, coba **neural network** kecil di **Keras** (opsional).

Pembahasan:

Dataset kecil cocok untuk **TF-IDF + LR**: ringan, cepat, akurat untuk teks pendek.

Rule-based memberi baseline cepat.

Keras dipakai jika ingin *neural network*, tapi tetap bisa offline.

Semua *open source*. Jalan di laptop sekolah.

Pseudocode (alur end-to-end singkat):

```
DATA = read_csv("ulasan.csv")           // ulasan, label
```

```

PREP = bersihkan_teks(DATA.ulasan) // lowercase, hapus tanda
      baca/URL
Xtr, Xte, ytr, yte = split(PREP, DATA.label, test_size=0.2)

VEC = TFIDF.fit(Xtr)
Xtrv = VEC.transform(Xtr)
Xtev = VEC.transform(Xte)

LR = LogisticRegression.fit(Xtrv, ytr)
pred = LR.predict(Xtev)

MET = evaluate(yte, pred) // accuracy, precision, recall, F1
      laporkan(MET)

```

Catatan

- **Ingat alur:** *Input* → *Preprocessing* → *Model* → *Output* → *Evaluasi*.
- Mulai sederhana (rule-based), lalu naik ke **TF-IDF + LR**, lalu ke **Keras** bila perlu.
- Semua tool **open source**. Bisa **offline**.
- Tujuan kita: sistem AI sekolah yang jelas alurnya, rapi datanya, terukur metriknya.

Contoh Soal dan Pembahasan

Soal 1 – Urutkan Alur Sistem AI

Pertanyaan:

Sebuah sistem AI dibuat untuk menilai apakah siswa “siap ujian” atau “perlu belajar lagi.”

Ada lima tahap: **Model, Input, Evaluasi, Output, Preprocessing**.

Urutkan dari awal sampai akhir.

Jawaban:

1. Input
2. Preprocessing
3. Model
4. Output
5. Evaluasi

Pembahasan:

Sama seperti pabrik, AI juga punya jalur kerja dari bahan mentah sampai hasil jadi.

Data mentah masuk sebagai *input*, lalu dibersihkan (*preprocessing*).

Setelah itu, *model* atau “otak AI” dilatih untuk mengenali pola.

Dari model ini keluar *output*, yaitu hasil prediksi.

Langkah terakhir: *evaluasi*, untuk memeriksa apakah hasilnya sudah bagus.

Pseudocode:

```
data = ambil_data("nilai_siswa.csv")
data_bersih = preprocessing(data)
model = latih_model(data_bersih)
hasil = model.predict(data_uji)
evaluasi(hasil)
```

Soal 2 – Isi Nilai Kosong

Pertanyaan:

Data nilai siswa ada yang hilang:

```
Nama: ["Budi", "Ani", "Siti"]
Nilai: [85, None, 90]
```

Isi nilai kosong dengan **rata-rata** nilai yang ada.

Jawaban:

Rata-rata (mean) dari 85 dan 90 adalah 87.5.

Nilai Ani diisi menjadi 87.5.

Hasil akhir:

Nama	Nilai
Budi	85
Ani	87.5
Siti	90

Pembahasan:

Inilah bagian *preprocessing* — *membersihkan data yang kotor atau hilang*.

Kalau tidak diisi, model AI akan bingung karena ada kolom kosong.

Dengan Python dan Pandas, ini mudah dilakukan secara otomatis.

Pseudocode:

```
data = {"Nama": ["Budi", "Ani", "Siti"], "Nilai": [85, None, 90]}
mean = hitung_rata(data["Nilai"]) // 87.5
for setiap siswa dalam data:
    if nilai kosong:
        isi_dengan(mean)
tampilkan(data)
```

Soal 3 – Menentukan Output dari Model

Pertanyaan:

Sebuah model prediksi nilai dibuat dari data latihan dan absensi.
Jika model mendapat input:

```
latihan = 90  
absensi = 95
```

Dan hasil output model adalah 0.92.

Jika batas kelulusan ditetapkan pada 0.8, apakah siswa ini lulus?

Jawaban:

Ya, siswa **lulus**, karena $0.92 > 0.8$.

Pembahasan:

Nilai output 0.92 berarti model yakin 92% bahwa siswa akan lulus. Angka ini dihasilkan oleh *neural network* kecil yang mempelajari pola dari data sebelumnya. Dengan *Keras*, kita bisa membuat model seperti itu hanya dalam beberapa baris kode.

Pseudocode:

```
input = [latihan=90, absensi=95]  
hasil = model.predict(input)  
if hasil > 0.8:  
    status = "Lulus"  
else:  
    status = "Belum Lulus"  
cetak(status)
```

Soal 4 – Evaluasi: Hitung Akurasi

Pertanyaan:

Model AI memprediksi hasil ujian dari 10 siswa.
Prediksi benar untuk 8 siswa.
Berapa akurasi model tersebut?

Jawaban:

Akurasi = (benar / total) = $8 / 10 = 0.8$ atau **80%**.

Pembahasan:

Akurasi menunjukkan seberapa banyak model menjawab benar.
Kalau akurasi rendah, mungkin datanya belum rapi atau modelnya terlalu sederhana.
Kita bisa memperbaikinya dengan *preprocessing* yang lebih baik atau menambah data latih.

Pseudocode:

```
benar = 8  
total = 10
```

```
akurasi = benar / total
cetak("Akurasi:", akurasi * 100, "%")
```

Soal 5 – Analogi Pabrik AI

Pertanyaan:

AI dianggap seperti pabrik pintar.

Coba isi bagian berikut:

1. Bahan mentah = _____
2. Mesin pengolah = _____
3. Produk jadi = _____

Jawaban:

1. Bahan mentah = *Data (input)*
2. Mesin pengolah = *Model (AI/Neural Network)*
3. Produk jadi = *Output (hasil prediksi atau jawaban)*

Pembahasan:

AI itu seperti pabrik: ada bahan masuk (data mentah), ada mesin (*model*) yang mengolah, lalu keluar hasil jadi (*output*). Kalau pabriknya mau menghasilkan produk lebih baik, harus rajin *evaluasi* dan perbaikan.

Pseudocode:

```
input = "data_mentah"
proses = model(input)
output = hasil(proses)
cetak("Produk akhir:", output)
```

Sekarang kamu sudah tahu bagaimana **alur sistem AI end-to-end** bekerja, dari *input* sampai *evaluasi*. Kamu juga sudah belajar cara menghitung akurasi, mengisi data kosong, dan melihat hasil prediksi dari model.

Semua dilakukan dengan tool **open source**:

- **Python** untuk logika.
- **Pandas** untuk olah data.
- **Keras** untuk *neural network*.

Dan semua bisa dijalankan **offline di laptop sekolah**.

Fakta Menarik (Fun Facts)

- **AI itu seperti pabrik, tapi bahan bakunya bukan besi — melainkan data!**
Setiap angka, teks, gambar, atau suara bisa diolah jadi pengetahuan baru kalau datanya rapi.
- **Otak AI dibuat dari kode Python, bukan otak manusia.**
Dengan *machine learning*, komputer bisa belajar sendiri tanpa disuruh satu per satu.
- **Library open source seperti *Pandas* dan *Keras* dipakai di perusahaan besar!**
Artinya, apa yang kamu pelajari di SMK sekarang juga dipakai di dunia industri sungguhan.
- **AI Chatbot sekolah bisa dibuat offline!**
Dengan *Ollama* dan *OpenWebUI*, kamu bisa bikin chatbot lokal tanpa koneksi internet.
- **Hampir semua aplikasi yang kamu pakai — TikTok, YouTube, Instagram — punya alur AI yang sama:**
Input → Proses → Output → Evaluasi. Bedanya cuma data dan tujuannya.
- **Semakin bersih datanya, semakin pintar hasil AI-nya.**
Makanya, *preprocessing* itu seperti “mandi” buat data — biar wangi dan siap kerja.

- **Neural network disebut *neural* karena meniru otak manusia.**
Tapi jangan khawatir, komputer belum bisa mimpi atau galau kayak kita.
- **Semua tool yang dipakai di bab ini gratis!**
Python, *Pandas*, *Matplotlib*, dan *Keras* semuanya open source — bisa kamu pelajari tanpa bayar lisensi.
- **Bikin model AI itu mirip bikin game level baru.**
Kamu tentukan input, latih model, lalu lihat siapa yang “menang” di hasil output-nya.
- **Data kecil pun bisa jadi besar kalau kamu olah dengan benar.**
Dari 10 data nilai siswa, kamu bisa bangun model prediksi kelulusan yang keren!

Tips Belajar atau Tips Cepat Menghafal

- **Ingat pola “Pabrik AI”: Input → Preprocessing → Model → Output → Evaluasi.**
Kalau hafal urutan ini, kamu sudah setengah jadi *AI Engineer*!
- **Latihan kecil lebih penting dari teori panjang.**
Coba langsung *coding* di Python, meskipun hanya `print(“Halo AI!”)`.
- **Gunakan analogi sederhana.**
Bayangkan *preprocessing* itu kayak cuci piring sebelum masak. Data juga harus bersih dulu.
- **Gambar diagram blok di buku.**
Visual membantu otak cepat paham. Gunakan panah dan warna berbeda untuk tiap tahap.
- **Pelajari Python dulu, baru lanjut ke *machine learning*.**
Python itu bahasa utama AI — gampang, gratis, dan banyak komunitasnya.

- **Coba jalankan perintah kecil di *Google Colab* atau *Jupyter Notebook*.**
Langsung lihat hasilnya — itu cara paling cepat belajar logika AI.
- **Diskusi dengan teman satu kelompok.**
AI itu seru kalau dikerjakan bareng. Kamu bisa saling bantu dan tukar ide.
- **Bikin mind map sebelum ujian.**
Tuliskan kata kunci seperti “input”, “model”, “output”, “evaluasi” — lalu hubungkan dengan panah.
- **Belajar sedikit tapi rutin.**
5 menit tiap hari jauh lebih efektif daripada belajar 2 jam cuma seminggu sekali.
- **Jangan takut error!**
Setiap error di Python adalah guru. Semakin sering kamu “salah”, semakin cepat kamu paham cara kerjanya.

AI itu bukan cuma buat orang kampus — **siswa SMK juga bisa bikin sistem AI sendiri.** Dengan *Python*, *Pandas*, dan *Keras*, kamu bisa belajar membuat model sederhana yang bisa “berpikir” dan “belajar”.

Jadi, jangan tunggu nanti. Coba sekarang, bereksperimen, dan nikmati prosesnya. Karena setiap baris kode yang kamu tulis adalah langkah kecil menuju masa depan teknologi Indonesia 🇮🇩

Aktivitas Siswa

Aktivitas 1: Membersihkan Data Nilai Siswa (Preprocessing)

Tujuan:

Belajar bagaimana data mentah disiapkan sebelum diproses oleh AI.

Langkah-langkah:

1. Baca file itu di Python.
2. Luat file `nilai.csv` berisi nama dan nilai siswa (beberapa nilai dikosongkan).
3. Isi nilai yang kosong dengan rata-rata kelas.
4. Cetak hasilnya.

Pseudocode:

```
load "nilai.csv" as data
for setiap baris dalam data:
    jika nilai kosong:
        isi dengan rata-rata seluruh nilai
print hasil_data
```

Contoh Python Script:

```
import pandas as pd
```

```
data = {"Nama": ["Ayu", "Budi", "Cici", "Doni"],
        "Nilai": [90, None, 75, 85]}
df = pd.DataFrame(data)

df["Nilai"].fillna(df["Nilai"].mean(), inplace=True)
print(df)
```

Hasilnya: data jadi bersih dan siap dipakai untuk pelatihan model AI.
Ini adalah langkah pertama dari pabrik AI — *input bersih = hasil keren!*

Aktivitas 2: Membuat Model AI Sederhana

Tujuan:

Belajar membuat *model AI* kecil yang bisa memprediksi hasil ujian dari nilai tugas.

Langkah-langkah:

1. Siapkan data tugas dan nilai ujian.
2. Latih model dengan *machine learning*.
3. Coba prediksi nilai baru.

Pseudocode:

```
data_tugas, data_ujian = load_data()
model = buat_model_AI()
latih(model, data_tugas, data_ujian)
prediksi = model.predict(nilai_tugas_baru)
tampilkan(prediksi)
```

Contoh Python Script:

```
from keras.models import Sequential
from keras.layers import Dense
import numpy as np

# Data
tugas = np.array([[80], [60], [70], [90]])
ujian = np.array([[85], [65], [75], [95]])

# Model AI
model = Sequential()
model.add(Dense(1, input_dim=1, activation='linear'))
model.compile(optimizer='adam', loss='mse')

# Latih model
model.fit(tugas, ujian, epochs=100, verbose=0)

# Prediksi
print("Prediksi nilai ujian untuk tugas 85:",
      model.predict([[85]])[0][0])
```

Kesimpulan:

Kamu baru saja membuat *neural network* mini! Otak buatanmu belajar dari data sungguhan.

Aktivitas 3: Membuat Grafik Prediksi Nilai

Tujuan:

Menampilkan hasil model AI dalam bentuk visual agar lebih mudah dibaca.

Langkah-langkah:

1. Gunakan data tugas dan hasil prediksi.
2. Gunakan *Matplotlib* untuk membuat grafik garis.
3. Bandingkan nilai asli vs hasil prediksi.

Pseudocode:

```
ambil data asli dan hasil prediksi
plot garis 1 = data asli
plot garis 2 = hasil prediksi
tampilkan grafik
```

Contoh Python Script:

```
import matplotlib.pyplot as plt
import numpy as np

tugas = [60, 70, 80, 90]
asli = [65, 75, 85, 95]
prediksi = [63, 74, 83, 94]

plt.plot(tugas, asli, 'o-', label='Nilai Asli')
plt.plot(tugas, prediksi, 'x--', label='Prediksi AI')
plt.title("Perbandingan Nilai Asli vs Prediksi AI")
plt.xlabel("Nilai Tugas")
plt.ylabel("Nilai Ujian")
plt.legend()
plt.show()
```

Kesimpulan:

Visualisasi itu penting! Dari grafik ini kamu bisa lihat apakah AI kamu sudah jago atau masih butuh dilatih lagi.

Aktivitas 4: Evaluasi Akurasi Model

Tujuan:

Mengetahui seberapa “pintar” model AI yang sudah kamu buat.

Langkah-langkah:

1. Siapkan data uji (yang belum pernah dilatih).
2. Lihat hasil prediksi AI.
3. Hitung *error* atau selisih antara nilai sebenarnya dan hasil AI.

Pseudocode:

```
prediksi = model.predict(data_uji)
error = rata_rata(abs(prediksi - nilai_sebenarnya))
tampilkan(error)
```

Contoh Python Script:

```
import numpy as np

asli = np.array([65, 75, 85])
prediksi = np.array([63, 74, 83])

error = np.mean(abs(asli - prediksi))
print("Rata-rata kesalahan (error):", error)
```

Kesimpulan:

Semakin kecil *error*-nya, semakin pintar modelmu.

Kalau masih besar? Coba tambah data atau ubah modelnya.

Aktivitas 5: Simulasi “Pabrik AI Sekolah”**Tujuan:**

Menggabungkan semua langkah — dari input sampai output akhir — jadi satu sistem sederhana.

Langkah-langkah:

1. Input: data nilai mentah siswa.
2. Preprocessing: bersihkan data.
3. Model: latih AI sederhana.
4. Output: prediksi nilai ujian.
5. Evaluasi: cek akurasi model.

Pseudocode (alur lengkap):

```
# 1. Input
data = load("nilai.csv")

# 2. Preprocessing
bersihkan_data(data)

# 3. Model
model = buat_model()
```

```

latih_model(model, data_latih)

# 4. Output
hasil = model.predict(data_uji)

# 5. Evaluasi
nilai_akurasi = evaluasi(hasil, data_uji)
tampilkan(nilai_akurasi)

```

Contoh Python Script (Mini Pipeline):

```

import pandas as pd
from keras.models import Sequential
from keras.layers import Dense
import numpy as np

# Input & Preprocessing
data = {"Tugas": [80, 70, 60, 90], "Ujian": [85, 75, 65, 95]}
df = pd.DataFrame(data)
X = df[["Tugas"]].values
y = df[["Ujian"]].values

# Model
model = Sequential()
model.add(Dense(1, input_dim=1, activation='linear'))
model.compile(optimizer='adam', loss='mse')

# Latih
model.fit(X, y, epochs=200, verbose=0)

# Output & Evaluasi
pred = model.predict(X)
error = np.mean(abs(pred.flatten() - y))
print("Prediksi AI:", pred.flatten())
print("Rata-rata error:", error)

```

Kesimpulan:

Kamu baru membuat sistem **AI end-to-end** lengkap — dari data mentah sampai evaluasi akhir! Inilah dasar dari semua proyek AI di dunia industri.

Catatan

Keren kan?

Dengan lima aktivitas ini, kamu sudah menjalani seluruh perjalanan *data scientist kecil*:

- Mulai dari **mengelola data** (input & preprocessing),
- Lanjut **membangun otak AI**,
- Hingga **mengevaluasi hasilnya**.

Semua dilakukan pakai tool **open source** — *Python, Pandas, Matplotlib, Keras*.
Gratis, kuat, dan bisa kamu jalankan di laptop sekolah.

Jadi... ayo lanjut bereksperimen!

Coba ganti data, ubah parameter, dan lihat apa yang terjadi.

Karena dalam dunia AI, setiap percobaan adalah langkah menuju penemuan baru.

Rangkuman

Setelah mempelajari bab ini, kamu sudah mengenal **alur kerja sistem AI dari awal sampai akhir**. Seperti pabrik yang punya alur kerja, AI juga punya tahap yang jelas: **Input → Preprocessing → Model → Output → Evaluasi**.

Tahap *input* adalah saat data mentah masuk ke sistem. Tahap *preprocessing* adalah saat data itu dibersihkan supaya rapi dan siap dipelajari oleh mesin. Lalu di tahap *model*, kita membuat “otak” AI yang bisa berpikir menggunakan *machine learning* atau *neural network*. Semuanya bisa kamu jalankan dengan **Python** dan **Pandas**, dua tool gratis yang kuat banget buat belajar AI!

Tahap berikutnya adalah *output* — hasil kerja dari otak AI. Misalnya, **AI bisa memprediksi nilai ujian dari data absensi dan latihan siswa**. Setelah itu, kamu harus melakukan *evaluasi* untuk memeriksa seberapa akurat hasil AI tersebut.

Kamu bisa menggunakan ukuran seperti **accuracy (akurasi)**, **precision (ketepatan)**, atau **recall (kelengkapan)**. Kalau hasilnya masih belum bagus, berarti modelnya perlu diperbaiki atau dilatih dengan data yang lebih baik. Di sinilah kamu belajar berpikir kritis dan analitis seperti seorang *data scientist*!

Intinya, kamu sekarang tahu bahwa sistem AI itu bukan sesuatu yang rumit atau menakutkan. Dengan **tool open source** seperti *Python*, *Pandas*, dan *Keras*, siapa pun bisa membuat *mini AI project* dari laptop sekolah. Kamu bisa mulai dari hal sederhana — seperti **membersihkan data nilai siswa, memprediksi hasil ujian, atau membuat chatbot sekolah**. Semua itu mengikuti alur yang sama: *data mentah → proses → hasil → evaluasi*. Dengan memahami konsep ini, kamu sudah selangkah lebih dekat untuk menjadi **pembuat AI masa depan** yang bisa menciptakan solusi nyata dari ide sederhana.

Latihan Soal

BENAR ATAU SALAH (5 Soal)

1. Tahap *input* dalam AI digunakan untuk memproses data yang sudah bersih.
Jawaban: Salah
2. *Preprocessing* adalah tahap untuk membersihkan dan menyiapkan data sebelum dilatih ke model.
Jawaban: Benar

3. *Model* dalam AI berfungsi sebagai otak yang belajar pola dari data.

Jawaban: Benar

4. *Output* adalah hasil akhir yang dihasilkan setelah data diproses oleh model.

Jawaban: Benar

5. *Evaluasi* dilakukan di awal proses agar model lebih cepat belajar.

Jawaban: Salah

PILIHAN GANDA (10 Soal)

1. Dalam alur AI, tahap pertama yang dilakukan adalah...

- a. Model
- b. Evaluasi
- c. Input
- d. Output

Jawaban: c

2. Library Python untuk mengelola data secara efisien adalah...

- a. TensorFlow
- b. Pandas
- c. NumPy
- d. Django

Jawaban: b

3. Fungsi utama dari *preprocessing* adalah...

- a. Menghapus model lama
- b. Membersihkan dan menyiapkan data
- c. Menjalankan neural network
- d. Menghitung hasil prediksi

Jawaban: b

4. Perintah berikut digunakan untuk mengisi nilai kosong dengan rata-rata data:

```
df["Nilai"].fillna(df["Nilai"].mean(), inplace=True)
```

Tahap ini termasuk...

- a. Model
- b. Input
- c. Preprocessing
- d. Output

Jawaban: c

5. Library *open source* untuk membangun jaringan saraf sederhana adalah...

- a. Keras
- b. WordPress
- c. Flask
- d. ChromeAI

Jawaban: a

6. Tahap akhir sistem AI yang menilai seberapa akurat model disebut...
- a. Evaluasi
 - b. Preprocessing
 - c. Model
 - d. Input

Jawaban: a

7. Dalam contoh kode berikut:

```
model.add(Dense(8, input_dim=2, activation='relu'))
```

Fungsi **Dense(8)** berarti...

- a. Menambah 8 data input
- b. Menambah 8 neuron di layer
- c. Menambah 8 baris data
- d. Menghapus data kosong

Jawaban: b

8. Jika data siswa masih ada nilai kosong, maka tahap yang harus dilakukan adalah...
- a. Model
 - b. Output
 - c. Preprocessing
 - d. Evaluasi

Jawaban: c

9. Output dari model AI bisa berupa...
- a. Nilai prediksi
 - b. Data mentah
 - c. File kosong
 - d. Folder baru

Jawaban: a

10. Dalam *machine learning*, model yang belum dievaluasi disebut...
- a. Model final
 - b. Model mentah
 - c. Model bersih
 - d. Model produksi

Jawaban: b

ISIAN SINGKAT (5 Soal)

1. Data mentah yang belum dibersihkan disebut _____.

Jawaban: Input

2. Library Python yang digunakan untuk pembersihan data adalah _____.

Jawaban: Pandas

3. Tahap di mana AI belajar pola dari data disebut _____.

Jawaban: Model

4. Hasil akhir dari proses sistem AI disebut _____.

Jawaban: Output

5. Untuk mengukur akurasi model, kita melakukan tahap _____.

Jawaban: Evaluasi

SOAL EKSPLORASI (3 Soal Menantang)

Soal 1 – AI Penilai Nilai Ujian

Kamu memiliki data nilai tugas dan absensi siswa.

Buat sistem AI sederhana dengan *Python + Pandas* untuk memprediksi siapa yang “Lulus” dan “Tidak Lulus” berdasarkan dua kriteria:

- Kehadiran $\geq 80\%$
- Nilai ≥ 70

Tuliskan pseudocode dan contoh kodenya.

Kunci Jawaban Contoh:

Pseudocode:

```
load data kehadiran dan nilai
jika Kehadiran  $\geq 80$  dan Nilai  $\geq 70$ :
    Status = "Lulus"
else:
    Status = "Tidak Lulus"
tampilkan hasil
```

Contoh Python:

```
import pandas as pd
df = pd.DataFrame({
    "Nama": ["Ayu", "Budi", "Cici"],
    "Kehadiran": [85, 75, 90],
    "Nilai": [70, 65, 80]
})
df["Status"] = df.apply(lambda x: "Lulus" if x["Kehadiran"] $\geq$ 80
and x["Nilai"] $\geq$ 70 else "Tidak Lulus", axis=1)
print(df)
```

Soal 2 – Membersihkan Data Kosong

Kamu menemukan dataset nilai siswa yang ada beberapa data kosong. Gunakan *Python Pandas* untuk mengisi data kosong dengan nilai rata-rata kelas agar dataset siap dilatih ke model AI.

Kunci Jawaban Contoh:

Pseudocode:

```
load dataset nilai
cari data kosong
isi data kosong dengan rata-rata kolom
tampilkan hasil bersih
```

Contoh Python:

```
import pandas as pd
data = {"Nama": ["Andi", "Dewi", "Eka"], "Nilai": [85, None, 90]}
df = pd.DataFrame(data)
df["Nilai"].fillna(df["Nilai"].mean(), inplace=True)
print(df)
```

Soal 3 – Merancang Alur AI Sekolah

Bayangkan kamu ingin membuat sistem AI sekolah yang bisa **memprediksi siswa yang berpotensi terlambat lulus**. Tuliskan alur *Input* → *Preprocessing* → *Model* → *Output* → *Evaluasi* versi kamu sendiri, dan buat pseudocode sederhanaanya.

Kunci Jawaban Contoh:

Langkah Alur:

1. **Input:** Data nilai, absensi, dan keterlambatan tugas.
2. **Preprocessing:** Hilangkan data kosong, ubah nilai ke skala 0–100.
3. **Model:** Gunakan *machine learning* sederhana untuk memprediksi status lulus.
4. **Output:** Prediksi “Tepat Waktu” atau “Terlambat”.
5. **Evaluasi:** Bandingkan hasil dengan data nyata.

Pseudocode:

```
load dataset
bersihkan data kosong
normalisasi nilai
latih model prediksi
prediksi status kelulusan
ukur akurasi model
```

Catatan

Soal-soal di atas membantu kamu memahami bagaimana sistem AI bekerja dari awal sampai akhir:

Input → **Preprocessing** → **Model** → **Output** → **Evaluasi**.

Semakin sering kamu berlatih, semakin kamu bisa membangun sistem AI sederhana dengan *Python*, *Pandas*, dan *Keras*!

Tugas Proyek

Proyek 1: AI Pengingat Jadwal Belajar

Tujuan: Membuat sistem sederhana yang membantu siswa menentukan jam belajar terbaik berdasarkan data aktivitas harian.

Langkah-langkah:

1. Buat file `jadwal.csv` berisi kolom: Hari, JamTidur, JamBelajar, dan Fokus (Ya/Tidak).
2. Bersihkan data yang kosong pakai *Pandas*.
3. Buat *rule-based model* sederhana: jika tidur ≥ 7 jam dan belajar pagi, maka Fokus = Ya.
4. Tampilkan hasilnya dalam tabel baru.

Pseudocode:

```
load "jadwal.csv" as data
for setiap baris:
    jika JamTidur >= 7 dan JamBelajar == "Pagi":
        Fokus = "Ya"
    else:
        Fokus = "Tidak"
print(data)
```

Contoh Python Script:

```
import pandas as pd

df = pd.DataFrame({
    "Hari": ["Senin", "Selasa", "Rabu", "Kamis"],
    "JamTidur": [8, 5, 7, 6],
    "JamBelajar": ["Pagi", "Malam", "Pagi", "Siang"]
})

df["Fokus"] = df.apply(lambda x: "Ya" if x["JamTidur"]>=7 and
x["JamBelajar"]=="Pagi" else "Tidak", axis=1)
print(df)
```

Hasil: Sistem AI kamu bisa bantu menentukan kapan waktu belajar terbaik supaya kamu tetap fokus!

Proyek 2: Analisis Lagu Favorit Siswa

Tujuan: Menganalisis genre lagu favorit berdasarkan data waktu belajar dan mood.

Langkah-langkah:

1. Buat file `lagu.csv` berisi kolom: Nama, Mood, Waktu, LaguFavorit.
2. Gunakan *Python Pandas* untuk membaca dan menampilkan data.
3. Kelompokkan lagu berdasarkan *mood* menggunakan fungsi `groupby()`.
4. Tampilkan rekomendasi lagu terbanyak dipilih.

Pseudocode:

```
load "lagu.csv"
kelompokkan berdasarkan kolom "Mood"
hitung jumlah tiap lagu
tampilkan lagu dengan jumlah terbanyak
```

Contoh Python Script:

```
import pandas as pd

data = {
    "Nama": ["Ayu", "Budi", "Cici", "Doni"],
    "Mood": ["Senang", "Sedih", "Senang", "Tenang"],
    "LaguFavorit": ["Pop", "Jazz", "Pop", "Lo-Fi"]
}

df = pd.DataFrame(data)
print(df.groupby("Mood")["LaguFavorit"].agg(lambda x:
x.mode()[0]))
```

Hasil: AI kamu bisa tahu genre musik apa yang cocok buat tiap suasana hati.

Proyek 3: Analisis Absen dan Nilai

Tujuan: Memprediksi apakah siswa akan lulus atau tidak berdasarkan kehadiran dan nilai tugas.

Langkah-langkah:

1. Siapkan data `absen_nilai.csv` berisi kolom: Kehadiran (%), NilaiTugas, dan Status (Lulus/Gagal).
2. Gunakan *Machine Learning Logistic Regression* untuk memprediksi kelulusan.
3. Tampilkan hasil prediksi baru.

Pseudocode:

```
load dataset
pisahkan data latih dan data uji
latih model logistic regression
prediksi status berdasarkan data baru
tampilkan hasil
```

Contoh Python Script:

```
from sklearn.linear_model import LogisticRegression
import pandas as pd

df = pd.DataFrame({
    "Kehadiran": [90, 70, 85, 60, 95],
    "NilaiTugas": [88, 60, 75, 55, 92],
    "Status": ["Lulus", "Gagal", "Lulus", "Gagal", "Lulus"]
})

X = df[["Kehadiran", "NilaiTugas"]]
y = (df["Status"] == "Lulus").astype(int)

model = LogisticRegression()
model.fit(X, y)

pred = model.predict([[80, 70]])
print("Prediksi kelulusan:", "Lulus" if pred[0]==1 else "Gagal")
```

Hasil: AI kamu bisa membantu guru memprediksi siapa yang butuh bimbingan tambahan sebelum ujian.

Proyek 4: Rekomendasi Menu Kantin Sehat

Tujuan: Membuat sistem rekomendasi sederhana untuk memilih makanan sehat di kantin sekolah.

Langkah-langkah:

1. Buat data **kantin.csv** dengan kolom: Menu, Kalori, Harga.
2. Gunakan *Pandas* untuk membaca dan filter data.
3. Pilih makanan dengan kalori < 300 dan harga < 10.000.
4. Tampilkan daftar rekomendasi.

Pseudocode:

```
load "kantin.csv"
filter menu dengan Kalori < 300 dan Harga < 10000
tampilkan hasil filter
```

Contoh Python Script:

```
import pandas as pd
```



```
df = pd.DataFrame({
    "Menu": ["Nasi Goreng", "Salad", "Mie Ayam", "Buah Potong"],
    "Kalori": [500, 200, 400, 150],
    "Harga": [12000, 8000, 10000, 7000]
})

sehat = df[(df["Kalori"]<300) & (df["Harga"]<10000)]
print("Rekomendasi Jajanan Sehat:\n", sehat)
```

Hasil: Kamu baru bikin sistem *AI mini* yang bisa bantu temanmu pilih makanan sehat tanpa perlu aplikasi mahal!

Proyek 5: Deteksi Hari Produktif

Tujuan: Melatih AI sederhana untuk mengenali hari paling produktif berdasarkan waktu tidur, jam belajar, dan hasil tugas.

Langkah-langkah:

1. Buat file **produktivitas.csv** berisi kolom: Hari, Tidur(jam), Belajar(jam), SkorTugas.
2. Normalisasi data supaya semua skala sejajar.
3. Gunakan *Keras Neural Network* untuk memprediksi skor produktivitas.
4. Tampilkan hasil prediksi untuk hari baru.

Pseudocode:

```
load data produktivitas
normalisasi data
bangun neural network sederhana
latih model
prediksi skor hari baru
```

Contoh Python Script:

```
from keras.models import Sequential
from keras.layers import Dense
import numpy as np

# Data contoh (Tidur, Belajar, Skor)
X = np.array([[6, 2], [8, 4], [5, 1], [7, 3]])
y = np.array([[70], [95], [60], [85]])

# Model Neural Network
model = Sequential()
model.add(Dense(8, input_dim=2, activation='relu'))
model.add(Dense(1, activation='linear'))
model.compile(optimizer='adam', loss='mse')
```

```
model.fit(X, y, epochs=200, verbose=0)
pred = model.predict([[7, 2]])
print("Prediksi skor produktivitas hari ini:", pred[0][0])
```

Hasil: AI kamu bisa menebak kapan kamu paling produktif. Cocok buat atur waktu belajar biar makin efisien!

Catatan

Keren kan? Dari lima proyek ini kamu sudah:

- Belajar **alur kerja AI end-to-end** (dari input sampai output).
- Menyentuh dunia *machine learning* dan *neural network* pakai tool open source.
- Membuat solusi nyata yang bisa langsung dipakai di kehidupan sekolah.

Sekarang giliran kamu untuk *bereksperimen dan berinovasi*.

Ubah datanya, tambah fitur baru, dan lihat bagaimana AI kamu belajar!

Karena di dunia teknologi, **setiap percobaan kecil bisa jadi penemuan besar**.

BAB 2: Integrasi Fine-Tuned Model dengan RAG Lokal

Tujuan Pembelajaran

Di akhir bab ini, kamu diharapkan bisa:

- Memahami apa itu *fine-tuned model* dan *RAG (Retrieval-Augmented Generation)*.
- Menjelaskan kenapa keduanya perlu digabung supaya chatbot jadi lebih pintar.
- Mampu menjelaskan contoh penerapan chatbot di sekolah dengan Python dan tool *open source*.

Peta Konsep (yang sederhana)

Fine-tuned Model + RAG Lokal → Chatbot Sekolah Pintar

Atau bisa juga dibayangkan begini:

“Model hasil fine-tuning = Gaya bicara”

“RAG = Isi otaknya dari dokumen”

Kalau dua-duanya digabung → Chatbot yang pintar + sopan + update terus.

Apersepsi (Pembuka yang Fun)

Bayangin kamu punya *chatbot FAQ sekolah*.

Dia udah bisa jawab pertanyaan seperti:

“Kapan jadwal UKK?” atau “Berapa biaya daftar ulang?”

Tapi tiba-tiba kamu tanya:

“Hitung luas lingkaran dengan jari-jari 14 cm.”

Ehh... si chatbot langsung *ngaco!*

Karena dia cuma tahu hal-hal dari *dataset FAQ*, bukan dari *buku pelajaran*.

Nah, di sinilah kamu akan jadi “penyelamat” chatbot itu!

Dengan **Integrasi Fine-Tuning + RAG**, kamu bikin chatbot yang:

- Tetap sopan dan khas anak SMK,
- Tapi juga bisa buka dokumen pelajaran dan kasih jawaban yang bener!

Seolah-olah kamu kasih dia **otak tambahan dari buku sekolah**.

Penjelasan Konsep

Bayangkan kamu punya **chatbot sekolah**.

- Setelah kita lakukan *fine-tuning*, chatbot bisa menjawab dengan **gaya bahasa sesuai kebutuhan** (misalnya sopan, singkat, atau khas anak SMK).
- Tapi ada satu masalah: chatbot ini hanya pintar menjawab dari **dataset training**. Kalau ada pertanyaan di luar itu, dia bisa **ngaco** atau asal jawab (*halusinasi*).

Nah, disinilah **RAG (Retrieval-Augmented Generation)** masuk sebagai solusi.

Apa itu Fine-Tuning?

- *Fine-tuning* adalah proses **melatih ulang model** dengan dataset khusus.
- Misalnya: chatbot sekolah dilatih dengan FAQ sekolah → hasilnya jadi tahu jadwal UKK, ekstrakurikuler, jurusan, dan aturan sekolah.
- Kelebihan: chatbot jadi punya **gaya jawaban** yang sesuai (ramah, formal, singkat, dsb).

Apa itu RAG?

- *Retrieval-Augmented Generation (RAG)* adalah teknik di mana model AI diberi **akses ke dokumen tambahan**.
- Model tidak hanya mengandalkan ingatan dari training, tapi juga bisa **mengambil informasi terbaru dari database/dokumen**.
- Misalnya: chatbot sekolah diberi dokumen pelajaran Matematika → sekarang dia bisa bantu jawab soal matematika langsung dari buku itu.

Kombinasi Fine-Tuning + RAG

Kalau digabung, hasilnya luar biasa:

- **Fine-tuning** → membuat chatbot punya gaya komunikasi yang cocok.
- **RAG** → memberikan chatbot isi pengetahuan yang terus bisa diperbarui.

Contoh nyata di sekolah:

- Chatbot bisa menjawab: “Kapan UKK dilaksanakan?” (karena sudah dilatih lewat *fine-tuning*).
- Chatbot juga bisa menjawab soal: “Hitung luas lingkaran dengan jari-jari 14 cm” (karena bisa ambil rumus dari dokumen Matematika lewat RAG).

Analogi Sederhana

- *Fine-tuning* itu seperti **melatih cara bicara** temanmu (sopan, singkat, atau humoris).
- *RAG* itu seperti **menyediakan perpustakaan mini** buat dia, jadi kalau ditanya sesuatu, dia bisa buka buku dulu sebelum menjawab.

- Hasilnya: temanmu jadi **pintar + sopan**.

Contoh Sederhana dengan Python

Kita bisa membayangkan alur integrasi dengan pseudocode:

```
# Load model hasil fine-tuning
model = load_finetuned_model("chatbot_smk")

# Load dokumen (contoh buku pelajaran)
documents = load_documents("materi_matematika.pdf")

# RAG: Ambil dokumen relevan sesuai pertanyaan
query = "Hitung luas lingkaran jari-jari 14 cm"
context = retrieve_relevant_docs(query, documents)

# Gabungkan context + query → lalu generate jawaban
answer = model.generate(query + context)

print(answer)
```

Dengan integrasi ini, chatbot bisa menjawab **FAQ sekolah** sekaligus **materi pelajaran**.

Intinya

- **Fine-tuning** = atur gaya dan kebiasaan jawaban model.
- **RAG** = tambahkan pengetahuan dari dokumen luar.
- **Integrasi keduanya** membuat chatbot sekolah jadi **lebih pintar, relevan, dan bermanfaat**.

Contoh Kasus atau Ilustrasi

Kasus 1: Chatbot Menjawab Soal Matematika dari Buku

Chatbotmu bisa menjawab pertanyaan seperti:

“Hitung luas lingkaran dengan jari-jari 14 cm.”

Tanpa RAG, chatbot cuma bisa jawab seadanya.

Tapi dengan RAG, dia bisa *buka dokumen Matematika.pdf*, ambil rumusnya, lalu kasih jawaban benar.

Pseudocode:

```
# 1. Load model hasil fine-tuning
model = load_finetuned_model("chatbot_smk")
```

```
# 2. Load dokumen pelajaran
docs = load_documents("buku_matematika.pdf")

# 3. Pertanyaan dari siswa
query = "Hitung luas lingkaran jari-jari 14 cm"

# 4. Ambil konteks dari dokumen
context = retrieve_relevant_docs(query, docs)

# 5. Gabungkan pertanyaan + konteks → hasilkan jawaban
answer = model.generate(query + context)

print(answer)
```

Chatbot: “*Luas lingkaran = $\pi \times r^2 = 3,14 \times 14^2 = 615,44 \text{ cm}^2$.*”

Kasus 2: Chatbot Menjawab Pertanyaan Bahasa Indonesia

Siswa bertanya:

“Apa arti majas personifikasi?”

Chatbot tidak dilatih untuk hafalan, tapi dengan RAG dia bisa *mengambil definisi langsung dari buku Bahasa Indonesia kelas 10*.

Pseudocode:

```
query = "Apa itu majas personifikasi?"
context = retrieve_relevant_docs(query,
    "buku_bahasa_indonesia.pdf")
answer = model.generate(query + context)
print(answer)
```

Chatbot: “*Majas personifikasi adalah gaya bahasa yang memberikan sifat manusia pada benda mati.*”

Kasus 3: Chatbot FAQ Sekolah

Chatbot juga bisa menjawab pertanyaan tentang aturan sekolah:

“Kapan jadwal UKK dilaksanakan?”

Model *fine-tuned* sudah tahu gaya bicara khas SMK, sementara *RAG* mengambil informasi dari dokumen “jadwal_ukk.docx”.

Pseudocode:

```
query = "Kapan UKK dilaksanakan?"
```

```
context = retrieve_relevant_docs(query, "jadwal_ukk.docx")
answer = model.generate(query + context)
print(answer)
```

Chatbot: *"UKK dilaksanakan mulai 10 Juni sampai 15 Juni di masing-masing jurusan."*

Kasus 4: Chatbot Teknik Komputer

Anak jurusan TKJ bertanya:

"Bagaimana cara kerja CPU dan GPU?"

Chatbot membaca ringkasan dari *modul_komputer.pdf* dan menjawab dengan gaya santai tapi tetap teknis.

Pseudocode:

```
query = "Cara kerja CPU dan GPU"
context = retrieve_relevant_docs(query, "modul_komputer.pdf")
answer = model.generate(query + context)
print(answer)
```

Chatbot:

"CPU itu otak utama komputer yang mengatur semua proses. GPU fokus ke gambar dan AI, biar kerja jadi cepat."

Kasus 5: Chatbot AI di Lab Sekolah

Guru memberi tugas:

"Jelaskan perbedaan machine learning dan deep learning."

Chatbot mengambil informasi dari *ebook_ai_smk.pdf* yang disimpan lokal.

Pseudocode:

```
query = "Perbedaan machine learning dan deep learning"
context = retrieve_relevant_docs(query, "ebook_ai_smk.pdf")
answer = model.generate(query + context)
print(answer)
```

Chatbot:

"Machine learning belajar dari data untuk membuat keputusan. Deep learning menggunakan jaringan syaraf (neural network) untuk memahami pola yang lebih kompleks."

Catatan

Integrasi *Fine-Tuned Model* + *RAG Lokal* bikin chatbot sekolah:

- Pintar menjawab pertanyaan pelajaran
- Tetap sopan dan gaya khas SMK
- Bisa update isi dari dokumen terbaru
- Semua berbasis *open source* (Python, Pandas, Keras, Neural Network)

Contoh Soal dan Pembahasan

Soal 1: Konsep: Mengapa Perlu RAG?

Soal:

Kamu sudah *fine-tuning* chatbot FAQ sekolah. Tapi saat ditanya materi pelajaran terbaru, jawabannya *ngaco*. Mengapa kita perlu menambah *RAG*?

Jawaban Singkat:

Untuk **mengurangi halusinasi** dan **mengambil info paling baru** dari dokumen/buku.

Pembahasan:

Fine-tuning mengatur **gaya** dan contoh jawaban dari dataset. Tapi model tidak “ingat” semua hal. Dengan *RAG*, chatbot **mengambil konteks** dari dokumen lokal (buku, PDF, panduan), sehingga jawaban **lebih akurat** dan **up-to-date**.

Pseudocode:

```
# Fine-tuned model = gaya
model = load_finetuned_model("chatbot_smk_faq") # Keras/TF
model

# RAG = isi dari dokumen
kb = load_documents(["buku_matematika.pdf",
                    "panduan_sekolah.pdf"])

question = "Apa beda mean dan median?"
context = retrieve_relevant_docs(question, kb) # vector search
(FAISS/Chroma)

answer = model.generate(question + "\nKonteks:\n" + context)
print(answer)
```

Soal 2: Matematika: Luas Lingkaran

Soal:

Hitung **luas lingkaran** dengan jari-jari **14 cm**. Gunakan *RAG* untuk mengambil rumus dari dokumen “buku_matematika.pdf”.

Jawaban Singkat:

615,44 cm².

Pembahasan:

Rumus dari dokumen: ($L = \pi \times r^2$).

Gunakan ($\pi \approx 3,14$) dan ($r = 14$).

($L = 3,14 \times 14^2 = 3,14 \times 196 = 615,44$).

Pseudocode:

```

model = load_finetuned_model("chatbot_smk_math")

docs = load_documents("buku_matematika.pdf")
q = "Hitung luas lingkaran jari-jari 14 cm. Sertakan langkah hitung."

ctx = retrieve_relevant_docs(q, docs) # akan mengekstrak rumus
L = pi * r^2
ans = model.generate(q + "\nKonteks:\n" + ctx)

print(ans) # diharapkan memuat: 615,44 cm^2

```

Soal 3: FAQ Sekolah: Jadwal UKK dari Dokumen**Soal:**

Di dokumen "jadwal_ukk.docx" ada kalimat:

"UKK dilaksanakan **10–15 Juni** di masing-masing jurusan."

Tunjukkan bagaimana chatbot menjawab pertanyaan: "**Kapan UKK dilaksanakan?**" menggunakan RAG.

Jawaban Singkat:

10–15 Juni.

Pembahasan:

Fine-tuned model membuat gaya jawabannya sopan. RAG mengambil **kalimat tepat** dari dokumen. Hasilnya informatif dan sesuai fakta.

Pseudocode:

```

model = load_finetuned_model("chatbot_smk_faq")
kb = load_documents("jadwal_ukk.docx")

q = "Kapan UKK dilaksanakan?"
ctx = retrieve_relevant_docs(q, kb) # menjemput kalimat tanggal UKK
ans = model.generate(q + "\nKonteks:\n" + ctx)

print(ans) # "UKK dilaksanakan 10-15 Juni di masing-masing jurusan."

```

Soal 4: Data Prep: Cleaning untuk Fine-Tuning

Soal:

Kamu mau *fine-tuning* gaya bahasa chatbot. Dataset FAQ masih kotor:

- Ada huruf campur (*case*),
- Spasi berlebih,
- Duplikat,
- Ada tag HTML.

Tuliskan **langkah cleaning** sederhana dengan **Pandas** sebelum *fine-tuning* di **Keras**.

Jawaban Singkat:

Lowercase → **strip spasi** → **hapus HTML** → **deduplicate** → **split train/valid**.

Pembahasan:

Data bersih = *fine-tuning* lebih stabil. *Noise* berkurang, dan model belajar pola yang benar. Ini standar *machine learning*.

Pseudocode (Python/Pandas + Keras):

```
# 1) Load & clean dengan Pandas
df = pd.read_csv("faq_sekolah.csv")
df["question"] = (df["question"]
                  .str.lower()
                  .str.replace(r"<[^>]+>", "", regex=True) #
hapus HTML
                  .str.replace(r"\s+", " ", regex=True)      #
rapikan spasi
                  .str.strip())
df["answer"] = (df["answer"]
                .str.lower()
                .str.replace(r"<[^>]+>", "", regex=True)
                .str.replace(r"\s+", " ", regex=True)
                .str.strip())

df = df.drop_duplicates(subset=["question", "answer"])

# 2) Split
train, valid = train_test_split(df, test_size=0.1,
                                random_state=42)

# 3) Fine-tuning (Keras) - ilustrasi
model = load_base_model() # model dasar open
source
model = finetune_with_pairs(model, train) # pairs Q/A
evaluate(model, valid)
save_finetuned_model(model, "chatbot_smk_faq")
```

Soal 5: Troubleshooting: Jawaban Masih Salah?

Soal:

Setelah integrasi *RAG*, chatbot masih salah menjawab beberapa pertanyaan pelajaran. Sebutkan **3 kemungkinan penyebab** dan **solusi cepatnya**.

Jawaban Singkat:

1. **Dokumen belum terindeks** → **Re-index** semua file.
2. **Chunking/embedding kurang pas** → Atur **ukuran chunk** & ganti **model embeddings**.
3. **Konteks tidak dipakai** → Pastikan *prompt* **menyatukan konteks + pertanyaan**.

Pembahasan:

RAG bergantung pada **indeks vektor**, **strategi chunking**, dan **prompting**. Kalau salah satu tidak optimal, hasil bisa meleset. Perbaiki pipeline-nya.

Pseudocode:

```
# 1) Re-index dokumen
docs = load_documents(["buku_matematika.pdf", "buku_bindo.pdf",
"modul_tkj.pdf"])
index = build_vector_index(docs,
embedding_model="all-MiniLM-L6") # open source

# 2) Tuning chunking & embeddings
index.set_params(chunk_size=800, chunk_overlap=120)
index.switch_embedding_model("bge-small") # contoh model open
source lain

# 3) Pastikan konteks dipakai di prompt
q = "Jelaskan perbedaan machine learning dan deep learning."
ctx = index.retrieve(q, top_k=3)

prompt = (
    "Gunakan konteks berikut untuk menjawab dengan ringkas dan
    akurat.\n"
    "Konteks:\n" + ctx + "\n"
    "Pertanyaan:\n" + q
)

ans = model.generate(prompt)
print(ans)
```

Catatan

- *Fine-tuning* → mengatur **gaya** dan pola jawaban (Keras/*neural network*).
- *RAG* → memberi **isi pengetahuan** dari dokumen lokal (FAISS/Chroma + Python).
- Kombinasi keduanya = chatbot sekolah yang **pintar, sopan, dan akurat**.

Fakta Menarik (Fun Facts)

1. **Chatbot yang kamu buat bisa “buka buku” sebelum menjawab!**
Dengan *RAG*, chatbot bisa membaca dokumen pelajaran seperti kamu buka buku teks di meja.
2. **Google dan Meta juga pakai RAG di sistem AI mereka.**
Artinya kamu belajar hal yang juga dipakai oleh *raksasa teknologi dunia!*
3. **Fine-tuning itu seperti ngasih “kepribadian” ke AI.**
Kamu bisa bikin AI jadi sopan kayak guru, santai kayak teman, atau formal kayak admin sekolah.
4. **RAG bisa bikin AI anti-halusinasi.**
Jadi chatbot kamu gak asal ngomong, karena dia punya bukti dari dokumen nyata.
5. **Semua toolnya open source!**
Python, Pandas, Keras, sampai *neural network* bisa kamu pakai gratis di laptop sekolah.
6. **AI sekolah kamu bisa terus update!**
Cukup tambahkan file baru ke folder dokumen RAG — chatbot langsung jadi lebih pintar tanpa dilatih ulang.
7. **RAG itu kayak otak tambahan.**
Fine-tuning itu memori jangka panjang, sedangkan RAG itu otak cepat yang bisa baca data baru.
8. **Integrasi RAG bisa bikin laptop biasa jadi lab AI mini.**
Gak perlu superkomputer, cukup semangat dan Ubuntu!
9. **Banyak ilmuwan AI di dunia pakai RAG untuk penelitian ilmiah.**
Jadi kamu sedang belajar *teknologi kelas riset dunia*.
10. **Chatbot sekolah buatanmu bisa bantu adik kelas belajar.**
Bayangin punya asisten belajar 24 jam, yang bisa jawab soal dari buku pelajaran.

Tips Belajar / Tips Cepat Menghafal

1. **Gunakan analogi.**
Bayangkan *fine-tuning* seperti melatih gaya bicara, dan *RAG* seperti menambah isi otak.
2. **Baca sambil praktek.**
Jalankan contoh *pseudocode* Python di komputermu — biar tangan dan otak kompak.
3. **Ingat rumus utama:**
Fine-tuning = *style*, RAG = *knowledge*.
Gabung dua-duanya → chatbot *super pintar*.
4. **Belajar 30 menit, istirahat 5 menit.**
Pola *Pomodoro* bikin otak gak cepat panas.
5. **Tulis ulang dengan bahasamu sendiri.**
Kalau kamu bisa menjelaskan ke temanmu, artinya kamu sudah paham.
6. **Gunakan warna atau gambar.**
Gambar skema “Fine-Tuning + RAG → Chatbot Pintar” di buku catatanmu.

7. **Latihan kecil setiap hari.**
Coba ubah prompt, tambahkan dokumen baru, lihat bagaimana chatbot berubah.
8. **Belajar bareng teman.**
Buat tim mini SMK yang masing-masing eksperimen satu bagian: ada yang coding, ada yang desain dataset.
9. **Jangan takut error.**
Error itu bukan musuh, tapi guru yang ngajarin kamu cara berpikir logis.
10. **Bangga pakai open source!**
Kamu sedang jadi bagian dari komunitas dunia yang suka berbagi ilmu dan teknologi bebas lisensi.

Aktivitas Siswa

Aktivitas 1 — Membuat Dataset FAQ Sekolah

Tujuan:

Membuat dataset sederhana untuk *fine-tuning* chatbot agar bisa menjawab pertanyaan tentang sekolah.

Langkah-langkah:

1. Buka *Google Sheets* atau *Notepad* lalu buat dua kolom: **Pertanyaan** dan **Jawaban**.
2. Isikan minimal 5 pertanyaan umum sekolah, misalnya:
 - “Kapan jadwal UKK?”
 - “Apa jurusan yang ada di SMK ini?”
3. Simpan dengan nama **faq_sekolah.csv**.
4. Buka file di Python dan tampilkan isi dataset.

Pseudocode:

```
load("faq_sekolah.csv")
preview(dataset)
```

Python Script:

```
import pandas as pd

df = pd.read_csv("faq_sekolah.csv")
print(df.head())
```

Hasil: Kamu punya dataset sederhana untuk melatih gaya bicara chatbot sekolahmu.

Aktivitas 2 — Membersihkan Dataset Sebelum Fine-Tuning

Tujuan:

Membuat data jadi bersih agar model mudah belajar.

Langkah-langkah:

1. Buka file `faq_sekolah.csv`.
2. Hilangkan baris kosong dan duplikat.
3. Ubah semua teks ke huruf kecil (*lowercase*).
4. Simpan hasilnya sebagai `faq_clean.csv`.

Pseudocode:

```
dataset = load("faq_sekolah.csv")
dataset = clean_text(dataset)
save("faq_clean.csv")
```

Python Script:

```
import pandas as pd

df = pd.read_csv("faq_sekolah.csv")
df.dropna(inplace=True)
df.drop_duplicates(inplace=True)
df["Pertanyaan"] = df["Pertanyaan"].str.lower()
df["Jawaban"] = df["Jawaban"].str.lower()
df.to_csv("faq_clean.csv", index=False)
```

Hasil: Dataset kamu jadi siap untuk proses *fine-tuning*.

Aktivitas 3 — Simulasi Fine-Tuning Chatbot Sekolah

Tujuan:

Melatih model AI agar bisa menjawab FAQ sekolah dengan gaya khas SMK.

Langkah-langkah:

1. Gunakan model *open source* seperti **Gemma 2B** atau **Mistral** di *Ollama*.
2. Gunakan dataset `faq_clean.csv` untuk melatih gaya bicara chatbot.
3. Jalankan proses *fine-tuning* dan simpan model hasilnya.

Pseudocode:

```
load_model("gemma")
train_model(dataset="faq_clean.csv")
save_model("chatbot_smk")
```

Python Script (simulasi dengan Keras):

```

from keras.models import Sequential
from keras.layers import Dense

# Model sederhana simulasi fine-tuning
model = Sequential([
    Dense(16, input_dim=10, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy')
print("Model fine-tuned untuk chatbot SMK berhasil dibuat!")

```

Hasil: Model chatbot sudah bisa menjawab dengan gaya “anak SMK”.

Aktivitas 4 — Menambahkan Dokumen ke RAG

Tujuan:

Memberi “otak tambahan” ke chatbot agar bisa membaca dokumen pelajaran.

Langkah-langkah:

1. Siapkan dokumen pelajaran, misalnya `buku_matematika.pdf`.
2. Simpan di folder `rag_docs/`.
3. Gunakan *RAG pipeline* untuk menambahkan dokumen tersebut.
4. Uji dengan pertanyaan: “Hitung luas lingkaran jari-jari 14 cm.”

Pseudocode:

```

load_model("chatbot_smk")
load_documents("rag_docs/")
query = "Hitung luas lingkaran jari-jari 14 cm"
context = retrieve_relevant_docs(query, "rag_docs/")
answer = model.generate(query + context)
print(answer)

```

Python Script:

```

def retrieve_relevant_docs(query, docs):
    return "Luas =  $\pi \times r^2 = 3.14 \times 14^2 = 615.44 \text{ cm}^2$ "

query = "Hitung luas lingkaran jari-jari 14 cm"
context = retrieve_relevant_docs(query, "buku_matematika.pdf")
print("Jawaban Chatbot:", context)

```

Hasil: Chatbot bisa menjawab soal pelajaran langsung dari dokumen lokal.

Aktivitas 5 — Uji Chatbot Sekolahmu

Tujuan:

Menggabungkan *fine-tuned model* dan *RAG lokal*, lalu mengujinya.

Langkah-langkah:

1. Jalankan chatbot hasil *fine-tuning* (*chatbot_smk*).
2. Tambahkan *RAG engine* untuk membaca dokumen.
3. Buat pertanyaan campuran:
 - Tentang sekolah: "Kapan UKK dilaksanakan?"
 - Tentang pelajaran: "Apa rumus luas persegi panjang?"
4. Bandingkan jawaban chatbot dengan dan tanpa RAG.

Pseudocode:

```
model = load_finetuned_model("chatbot_smk")
rag_docs = load_documents("rag_docs/")

q1 = "Kapan UKK dilaksanakan?"
q2 = "Apa rumus luas persegi panjang?"

ans1 = model.generate(q1)
ctx = retrieve_relevant_docs(q2, rag_docs)
ans2 = model.generate(q2 + ctx)

print(ans1)
print(ans2)
```

Python Script:

```
def chatbot(query):
    if "ukk" in query:
        return "UKK dilaksanakan tanggal 10-15 Juni di jurusan masing-masing."
    elif "persegi panjang" in query:
        return "Luas = panjang x lebar"
    else:
        return "Maaf, saya tidak tahu."

print(chatbot("Kapan UKK dilaksanakan?"))
print(chatbot("Apa rumus luas persegi panjang?"))
```

Hasil:

Chatbot sekarang bisa menjawab dua tipe pertanyaan:

- FAQ Sekolah (hasil *fine-tuning*)
- Pelajaran dari dokumen (*RAG Lokal*)

Catatan

Dari semua aktivitas di atas:

- Kamu sudah membuat dataset FAQ.
- Membersihkannya dengan Pandas.
- Melatih model (*fine-tuning*).

- Menambah dokumen pelajaran (RAG).
- Menguji chatbot sekolah.

Semua dilakukan dengan tool *open source*!

Python, Pandas, Keras, Machine Learning, dan Neural Network.

Rangkuman

Pada bab ini, kamu belajar bagaimana cara membuat **chatbot sekolah yang cerdas dan bergaya khas anak SMK**. Langkah pertama adalah memahami **apa itu fine-tuning**. *Fine-tuning* berarti melatih ulang model AI agar punya **gaya komunikasi sesuai kebutuhanmu** — bisa sopan, santai, atau singkat seperti obrolan di bengkel. Dengan *fine-tuning*, chatbot belajar dari **dataset FAQ sekolah**, sehingga bisa menjawab hal-hal seperti jadwal UKK, ekstrakurikuler, atau aturan jurusan. Teknologi ini bisa dilakukan dengan tool **open source** seperti *Python, Pandas, Keras, dan Neural Network* — semua gratis dan bisa dijalankan di laptop sekolahmu.

Selanjutnya, kamu juga belajar tentang RAG (Retrieval-Augmented Generation), yaitu teknik yang memberi chatbot “otak tambahan.” Kalau *fine-tuning* adalah gaya bicara, maka RAG adalah isi pengetahuannya. Dengan RAG, chatbot bisa membaca dokumen lokal seperti buku pelajaran, panduan sekolah, atau modul jurusan. Jadi, kalau kamu bertanya soal rumus luas lingkaran atau prinsip kerja CPU, chatbot bisa menjawab langsung dari dokumen yang kamu sediakan. Itulah kenapa RAG sering disebut sebagai perpustakaan mini di dalam AI! 📚

Ketika *fine-tuning* dan RAG digabung, hasilnya luar biasa. Chatbot sekolahmu jadi **pintar, sopan, dan terus update** tanpa perlu dilatih dari awal setiap kali ada perubahan. Gabungan ini menciptakan **AI lokal yang bermanfaat dan hemat sumber daya**, bisa dijalankan bahkan di komputer lab biasa. Dengan sedikit kode *Python* dan semangat belajar, kamu sudah bisa jadi pembuat chatbot *open source* yang keren — dan mungkin jadi inspirasi bagi adik kelasmu di masa depan!

Latihan Soal

A. Benar atau Salah (5 Soal)

1. *Fine-tuning* digunakan untuk melatih model agar punya gaya bicara sesuai kebutuhan.
Jawaban: Benar
2. RAG (*Retrieval-Augmented Generation*) hanya digunakan untuk melatih gaya komunikasi chatbot.

Jawaban: Salah

3. RAG membantu chatbot mengambil informasi dari dokumen luar agar jawabannya lebih akurat.

Jawaban: Benar

4. Gabungan *fine-tuning* dan RAG bisa membuat chatbot jadi semakin bingung menjawab pertanyaan.

Jawaban: Salah

5. Dengan *open source tools* seperti Python dan Keras, kita bisa membuat chatbot sendiri di komputer sekolah.

Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Fungsi utama dari *fine-tuning* adalah...

A. Menambah dokumen baru
B. Mengatur gaya jawaban model
C. Menghapus dataset lama
D. Menyimpan log percakapan

Jawaban: B

2. RAG digunakan agar chatbot bisa...

A. Menggambar diagram AI
B. Membuka file dokumen tambahan
C. Mengubah warna tampilan UI
D. Menyimpan data pengguna

Jawaban: B

3. Contoh sederhana dari *fine-tuning* adalah...

A. Melatih model dengan data FAQ sekolah
B. Membuka dokumen matematika
C. Menghapus log file
D. Menyimpan data training

Jawaban: A

4. File berikut yang bisa digunakan oleh RAG untuk belajar adalah...

A. **.mp3**
B. **.pdf**
C. **.exe**
D. **.zip**

Jawaban: B

5. Jika chatbot salah menjawab karena tidak punya konteks baru, maka solusi terbaik adalah...

A. Menghapus modelnya

- B. Mengaktifkan *RAG*
- C. Menambah RAM
- D. Menonaktifkan dataset

Jawaban: B

6. Bahasa pemrograman yang paling umum digunakan untuk *AI open source* adalah...
- A. Java
 - B. Ruby
 - C. Python
 - D. C++

Jawaban: C

7. Library Python yang digunakan untuk mengelola data tabel adalah...
- A. Pandas
 - B. NumPy
 - C. Matplotlib
 - D. Scikit

Jawaban: A

8. *Neural network* di dalam AI berfungsi untuk...
- A. Menghapus error
 - B. Meniru cara kerja otak manusia
 - C. Mengatur tampilan GUI
 - D. Menyimpan file log

Jawaban: B

9. Output dari kode berikut adalah...

```
r = 14
luas = 3.14 * r * r
print(luas)
```

- A. 615.44
- B. 196
- C. 314
- D. 14

Jawaban: A

10. Dalam konsep *RAG*, bagian yang mengambil dokumen relevan disebut...
- A. Retriever
 - B. Generator
 - C. Updater
 - D. Loader

Jawaban: A

C. Isian Singkat (5 Soal)

1. *Fine-tuning* melatih model agar memiliki _____ dan gaya bicara tertentu.
Jawaban: kepribadian
2. *RAG* memberikan chatbot kemampuan untuk _____ dari dokumen tambahan.
Jawaban: mengambil informasi
3. Kombinasi *fine-tuning* dan *RAG* disebut juga integrasi model yang _____ dan dinamis.
Jawaban: pintar
4. Library Python yang digunakan untuk membangun *neural network* adalah _____.
Jawaban: Keras
5. Dalam *RAG*, *retriever* bertugas mencari _____ dari dokumen yang relevan.
Jawaban: konteks

D. Soal Eksplorasi (3 Soal)

Soal 1 — Chatbot Pelajaran Fisika

Buatlah chatbot sederhana yang bisa menjawab pertanyaan dari file `fisika.txt` yang berisi konsep dasar Fisika seperti *gaya*, *massa*, dan *percepatan*. Gunakan logika *RAG* sederhana: chatbot membaca file, lalu mencari kata kunci dari pertanyaan.

Pseudocode Jawaban:

```
file = load("fisika.txt")
question = input("Tanya tentang Fisika: ")
context = search(file, question)
print(context)
```

Contoh Jawaban Siswa:

Chatbot membaca file `fisika.txt`. Saat ditanya “Apa itu gaya?”, chatbot mencari baris yang berisi “gaya” dan menjawab “Gaya adalah tarikan atau dorongan yang menyebabkan benda bergerak.”

Soal 2 — Chatbot Bahasa Santai SMK

Ubah gaya bahasa chatbot agar lebih *friendly* dan khas anak SMK. Misalnya, jawaban yang tadinya formal jadi lebih santai tapi sopan. Gunakan *fine-tuning* sederhana dengan dataset FAQ sekolah yang kamu buat sendiri.

Pseudocode Jawaban:

```
model = load_finetuned_model("chatbot_smk")
prompt = "Fine-tune gaya bicara santai"
save_model(model, "chatbot_smk_gaya_santai")
```

Contoh Jawaban Siswa:

Chatbot akan menjawab “Santai aja bro, UKK dimulai minggu depan ya!”
Gaya ini dibuat lewat dataset baru berisi kalimat sopan tapi gaul khas SMK.

Soal 3 — Gabungkan Dua Dokumen dengan RAG

Buat chatbot yang bisa membaca dua dokumen sekaligus: `buku_matematika.pdf` dan `buku_biologi.pdf`. Jika pertanyaan mengandung kata “rumus”, ambil dari dokumen Matematika. Jika pertanyaan mengandung kata “sel” atau “organisme”, ambil dari Biologi.

Pseudocode Jawaban:

```
math_doc = load("buku_matematika.pdf")
bio_doc = load("buku_biologi.pdf")
q = input("Masukkan pertanyaan: ")
if "rumus" in q:
    context = retrieve(math_doc)
elif "sel" in q or "organisme" in q:
    context = retrieve(bio_doc)
print("Chatbot:", context)
```

Contoh Jawaban Siswa:

Chatbot bisa menjawab pertanyaan dari dua pelajaran berbeda dengan cerdas.
Misalnya, “Apa rumus luas persegi?” diambil dari buku Matematika, sedangkan “Apa fungsi sel darah merah?” diambil dari buku Biologi.

Catatan

Latihan ini melatih kamu untuk:

- Memahami konsep *fine-tuning* dan *RAG*.
- Menulis *pseudocode* dan *Python script sederhana*.
- Berpikir seperti *AI engineer muda SMK*.

Semua bisa dilakukan hanya dengan **tool open source: Python, Pandas, Keras, dan Machine Learning!**

Tugas Proyek

Proyek 1 – Chatbot Pengenalan Sekolah

Tujuan: Membuat chatbot yang bisa menjawab pertanyaan umum tentang profil sekolah.

Langkah-langkah:

1. Buat file `profil_sekolah.csv` berisi dua kolom: `pertanyaan` dan `jawaban`.
Contoh:
 - “Apa visi sekolah ini?”
 - “Kapan SMK berdiri?”
2. Gunakan *Python + Pandas* untuk membaca dan memproses file.
3. Buat fungsi chatbot sederhana untuk mencari jawaban yang cocok dari CSV.

Pseudocode:

```
data = load_csv("profil_sekolah.csv")
query = input("Tanya tentang sekolah: ")
answer = search_answer(data, query)
print(answer)
```

Contoh Python:

```
import pandas as pd

data = pd.read_csv("profil_sekolah.csv")
query = input("Tanya tentang sekolah: ").lower()

for i, row in data.iterrows():
    if query in row["pertanyaan"].lower():
        print("Chatbot:", row["jawaban"])
        break
else:
    print("Chatbot: Maaf, saya belum tahu jawabannya.")
```

Hasil: Chatbot sederhana berbasis CSV yang bisa menjawab pertanyaan umum tentang sekolahmu.

Proyek 2 – RAG Mini dengan Dokumen Pelajaran

Tujuan: Menghubungkan chatbot dengan dokumen pelajaran supaya bisa menjawab soal.

Langkah-langkah:

1. Siapkan file `materi_matematika.txt` berisi ringkasan rumus dasar.
2. Buat fungsi untuk membaca isi file dan mencari jawaban yang relevan.
3. Uji chatbot dengan pertanyaan: “Bagaimana rumus luas segitiga?”

Pseudocode:

```

doc = load_file("materi_matematika.txt")
query = "rumus luas segitiga"
context = search_context(doc, query)
answer = model_generate(query + context)
print(answer)

```

Contoh Python:

```

def rag_search(query):
    with open("materi_matematika.txt", "r") as f:
        doc = f.read()
    if "segitiga" in query:
        return "L = 1/2 x alas x tinggi"
    return "Maaf, belum ada datanya."

q = input("Masukkan pertanyaan: ")
print("Chatbot:", rag_search(q.lower()))

```

Hasil: Chatbot bisa membaca file lokal dan menjawab soal pelajaran dari dokumen!

Proyek 3 – Analisis Dataset Chatbot dengan Pandas

Tujuan: Melihat seberapa besar dataset yang kamu punya dan melakukan analisis sederhana.

Langkah-langkah:

1. Gunakan dataset **faq_clean.csv** hasil dari *Aktivitas Siswa*.
2. Hitung jumlah pertanyaan unik dan rata-rata panjang jawaban.
3. Simpan hasil analisis ke file **laporan_dataset.csv**.

Pseudocode:

```

data = load_csv("faq_clean.csv")
count_question = count_unique(data["Pertanyaan"])
avg_length = average_length(data["Jawaban"])
save_report(count_question, avg_length)

```

Contoh Python:

```

import pandas as pd

df = pd.read_csv("faq_clean.csv")
count_q = df["Pertanyaan"].nunique()
avg_len = df["Jawaban"].apply(len).mean()

laporan = pd.DataFrame({
    "Jumlah Pertanyaan": [count_q],
    "Rata-rata Panjang Jawaban": [avg_len]
})

laporan.to_csv("laporan_dataset.csv", index=False)
print(laporan)

```

Hasil: Kamu membuat *data report* mini menggunakan Pandas seperti *data scientist profesional*.

Proyek 4 – Chatbot Pelajaran Multi-Topik

Tujuan: Menggabungkan dua sumber data: FAQ sekolah dan dokumen pelajaran.

Langkah-langkah:

1. Gunakan file `faq_clean.csv` (dataset FAQ).
2. Tambahkan dokumen `materi_ipa.txt`.
3. Buat chatbot yang bisa membedakan pertanyaan sekolah dan pelajaran.
 - o Jika pertanyaan tentang sekolah → ambil dari CSV.
 - o Jika tentang pelajaran → ambil dari dokumen.

Pseudocode:

```
faq = load_csv("faq_clean.csv")
docs = load_file("materi_ipa.txt")
query = input("Masukkan pertanyaan:")
if "UKK" in query or "jurusan" in query:
    answer = search_csv(faq, query)
else:
    answer = search_doc(docs, query)
print(answer)
```

Contoh Python:

```
import pandas as pd

faq = pd.read_csv("faq_clean.csv")
with open("materi_ipa.txt") as f:
    docs = f.read()

q = input("Masukkan pertanyaan: ").lower()
if "ukk" in q or "jurusan" in q:
    for i, row in faq.iterrows():
        if q in row["Pertanyaan"].lower():
            print("Chatbot:", row["Jawaban"])
            break
else:
    if "atom" in q:
        print("Chatbot: Atom adalah partikel terkecil penyusun materi.")
```

Hasil: Chatbot bisa *menyesuaikan sumber informasi secara otomatis*.

Proyek 5 – Membuat Mini Dashboard RAG

Tujuan: Menampilkan hasil tanya-jawab chatbot secara visual (text log).

Langkah-langkah:

1. Simpan semua interaksi chatbot ke file `log_interaksi.csv`.

2. Tampilkan 5 pertanyaan terakhir yang sudah dijawab.
3. Gunakan *Pandas* untuk membaca dan menampilkan log.

Pseudocode:

```
log = load_csv("log_interaksi.csv")
show_last(5)
```

Contoh Python:

```
import pandas as pd

# Simulasi data log
data = {
    "Pertanyaan": ["Apa itu RAG?", "Siapa pembuat Python?", "Apa itu AI?"],
    "Jawaban": ["Teknik ambil dokumen", "Guido van Rossum", "Kecerdasan buatan"]
}
df = pd.DataFrame(data)
df.to_csv("log_interaksi.csv", index=False)

# Tampilkan log terakhir
log = pd.read_csv("log_interaksi.csv")
print("Riwayat Tanya-Jawab Terakhir:")
print(log.tail(5))
```

Hasil: Kamu bisa melihat log chatbot seperti *mini dashboard* yang mencatat semua aktivitas!

Catatan

Setelah menyelesaikan 5 proyek ini, kamu sudah:

- Membuat **chatbot sekolah sederhana**,
- Menggunakan **RAG lokal dengan file pelajaran**,
- Menganalisis **dataset dengan Pandas**,
- Menggabungkan **dua sumber informasi berbeda**,
- Dan membuat **log interaksi AI** seperti sistem profesional!

Semua menggunakan **tool open source** — *Python, Pandas, Keras, dan Neural Network*. Dengan ini, kamu bukan hanya pengguna AI, tapi **pencipta AI SMK!**

BAB 3: Membangun Sistem RAG Sekolah (Dokumen Kurikulum, Buku Pelajaran)

Tujuan Pembelajaran

Setelah mempelajari bab ini, kamu diharapkan mampu:

1. Menjelaskan apa itu *RAG (Retrieval-Augmented Generation)*.
2. Memahami cara kerja *RAG* dalam mencari dan menyusun jawaban dari dokumen.
3. Mengubah dokumen sekolah (seperti kurikulum dan buku pelajaran) menjadi *index digital* agar mudah dicari.
4. Menggunakan teknologi *open source* seperti **Python**, **Pandas**, **FAISS**, **Keras**, dan **Neural Network**.
5. Membuat chatbot sederhana yang bisa menjawab pertanyaan siswa berdasarkan dokumen sekolah.

Peta Konsep (Sederhana dan Visual)

Dokumen Sekolah → Index → RAG → Chatbot Pintar

Apersepsi

Bayangin kamu lagi di perpustakaan sekolah.

Rak buku penuh. File PDF di komputer juga banyak banget. Tiba-tiba guru nanya:

“Coba, siapa yang bisa nunjukin Kompetensi Dasar IPA kelas 10 dalam 5 detik?”

Kalau kamu masih cari manual, pasti lama. Tapi...

bagaimana kalau **chatbot sekolah** kamu langsung jawab:

“Kompetensi Dasar IPA kelas 10 ada di halaman 3, tentang pengukuran dan besaran!”

Wow, keren banget kan? Nah, itulah yang akan kamu pelajari di bab ini.

Kamu akan jadi **arsitek AI sekolah** — membangun sistem yang bisa:

- Membaca dokumen sekolah.
- Menyimpan informasinya dengan cepat (*indexing*).
- Menjawab pertanyaan guru atau temanmu lewat chatbot cerdas.

Kamu bakal belajar pakai **Python**, **Pandas**, dan **FAISS** — semua *open source*, semua bisa dipelajari gratis.

Penjelasan Konsep

Bayangkan kalian punya **perpustakaan sekolah**. Isinya banyak banget: kurikulum, buku pelajaran, soal ujian, sampai catatan guru. Nah, kalau kalian disuruh mencari 1 halaman tertentu di ratusan buku, pasti repot, kan? Butuh waktu lama, capek, dan kadang bikin pusing.

Di sinilah teknologi *open source* seperti **Python**, **Pandas**, dan **machine learning** masuk. Kita bisa membuat *sistem pintar* yang bisa mencari informasi super cepat, bahkan dari ribuan halaman. Teknologi ini disebut **RAG** (*Retrieval-Augmented Generation*).

Apa itu RAG?

RAG adalah gabungan dua kekuatan:

- **Retriever** → mesin pencari yang mencari dokumen relevan.
- **Generator** → mesin penjawab (AI/LLM) yang bikin jawaban berdasarkan dokumen yang ketemu.

Kalau kalian nanya:

"Apa hukum Newton pertama?"

Maka retriever akan mencari ke dokumen Fisika di kurikulum. Setelah ketemu, generator akan menyusun jawaban yang rapi dan gampang dipahami. Jadi hasilnya bukan hanya *copy-paste*, tapi sudah dalam bentuk jawaban pintar.

Bagaimana caranya?

1. Indexing dokumen

- Semua dokumen (PDF kurikulum, buku pelajaran, catatan sekolah) diubah menjadi *index*.
- *Index* ini seperti daftar isi super cepat.
- Tools open source yang bisa dipakai: **FAISS** atau **Chroma**.

2. Retriever bekerja

- Saat ada pertanyaan, retriever mencari potongan teks paling relevan dari *index*.
- Misalnya pertanyaan tentang Fisika, dia langsung nyari di bagian Fisika, bukan di buku Biologi.

3. Generator menjawab

- Setelah dapat potongan teks, generator (pakai model AI/LLM open source, misalnya Ollama + Gemma/Mistral) merangkai jawaban.
- Jawaban lebih jelas, bukan cuma kutipan acak.

Contoh Kasus

Siswa bertanya ke chatbot sekolah:

"Jelaskan konsep energi kinetik!"

- **Retriever:** mencari definisi energi kinetik di PDF Fisika.
- **Generator:** menyusun jawaban → "Energi kinetik adalah energi yang dimiliki benda karena geraknya. Rumusnya $\frac{1}{2}mv^2$."

Chatbot jadi seolah-olah *guru cadangan* yang selalu siap bantu.

Kenapa Penting?

- **Cepat:** Tidak perlu baca seluruh buku.
- **Akurat:** Jawaban berdasarkan dokumen resmi sekolah.
- **Mudah:** Cukup tanya ke chatbot.
- **Fleksibel:** Bisa dipakai untuk semua mata pelajaran.

Mix Fun + Point untuk Siswa SMK

- Bayangkan chatbot = **perpustakaan pintar sekolah**.
- *Retriever* = satpam perpustakaan → cepat nunjukin buku yang dicari.
- *Generator* = guru pintar → langsung menjelaskan isi buku dengan bahasa mudah.
- Tools open source: **Python, Pandas, FAISS, Chroma, Keras, Neural Network**.
- Semua bisa jalan bahkan di laptop sekolah dengan RAM seadanya!

Contoh Kasus

Kasus 1: Chatbot Kurikulum Sekolah

Situasi:

Guru menanyakan ke siswa:

"Apa tujuan pembelajaran Informatika kelas 10?"

Daripada buka dokumen kurikulum manual (yang tebal banget), kamu bisa tanya ke chatbot sekolah.

Ilustrasi:

- *Retriever* mencari isi file "Kurikulum_Informatika_10.pdf"
- *Generator* menyusun jawaban dari potongan teks yang cocok

Hasil Jawaban Chatbot:

“Tujuan pembelajaran Informatika kelas 10 adalah memahami dasar algoritma dan logika pemrograman.”

Pseudocode:

```
pertanyaan = "Tujuan pembelajaran Informatika kelas 10"
dokumen = load("kurikulum_informatika_10.pdf")
hasil = retriever(dokumen, pertanyaan)
jawaban = generator(hasil)
print(jawaban)
```

Penjelasan:

Chatbot bertindak seperti *asisten guru digital* yang tahu isi kurikulum sekolah.

Kasus 2: Chatbot Buku Pelajaran Fisika

Situasi:

Siswa nanya:

“Apa rumus energi kinetik?”

Ilustrasi:

- *Retriever* mencari kata “energi kinetik” di buku Fisika.
- *Generator* menjawab dengan bahasa yang mudah dipahami.

Hasil Jawaban Chatbot:

“Energi kinetik adalah energi karena gerak. Rumusnya: $\frac{1}{2} \times \text{massa} \times \text{kecepatan}^2$.”

Pseudocode:

```
pertanyaan = "Apa rumus energi kinetik"
dokumen = load("fisika_kelas10.pdf")
hasil = retriever(dokumen, pertanyaan)
jawaban = generator(hasil)
print(jawaban)
```

Penjelasan:

Chatbot jadi seperti “guru Fisika versi AI” yang langsung kasih rumus dan penjelasan singkat.

Kasus 3: Chatbot Bahasa Indonesia

Situasi:

Siswa sedang mengerjakan tugas dan lupa arti *majas personifikasi*.

Dia bertanya:

“Apa itu majas personifikasi?”

Ilustrasi:

- *Retriever* mencari di buku Bahasa Indonesia bab “Gaya Bahasa”.
- *Generator* menyusun jawaban berdasarkan teks dari buku.

Hasil Jawaban Chatbot:

“Majas personifikasi adalah gaya bahasa yang membuat benda mati seolah hidup.”

Pseudocode:

```
pertanyaan = "Apa itu majas personifikasi"
dokumen = load("bahasa_indonesia.pdf")
hasil = retriever(dokumen, pertanyaan)
jawaban = generator(hasil)
print(jawaban)
```

Penjelasan:

Chatbot bisa bantu siswa mengerjakan tugas bahasa tanpa harus cari halaman manual di buku tebal.

Kasus 4: Chatbot Matematika Dasar**Situasi:**

Siswa bertanya:

“Bagaimana cara menghitung luas lingkaran?”

Ilustrasi:

- *Retriever* mencari bab tentang “Bangun Datar” di buku Matematika.
- *Generator* menjelaskan langkah-langkah perhitungannya.

Hasil Jawaban Chatbot:

“Luas lingkaran = $\pi \times r^2$.
Misal jari-jari 7 cm, maka luasnya = $3.14 \times 7 \times 7 = 153.86 \text{ cm}^2$.”

Pseudocode:

```
pertanyaan = "Cara menghitung luas lingkaran"
dokumen = load("matematika_kelas10.pdf")
hasil = retriever(dokumen, pertanyaan)
jawaban = generator(hasil)
print(jawaban)
```

Penjelasan:

Chatbot bisa menjelaskan konsep sambil kasih contoh hitungan sederhana. Jadi belajar Matematika nggak harus nunggu guru datang.

Kasus 5: Chatbot Lingkungan & Kewirausahaan

Situasi:

Siswa sedang mengikuti kegiatan proyek lingkungan sekolah, dan bertanya:

“Apa manfaat daur ulang sampah plastik?”

Ilustrasi:

- *Retriever* mencari dokumen “Proyek Lingkungan Sekolah.pdf”.
- *Generator* menyusun jawaban dari bab “Pengelolaan Limbah”.

Hasil Jawaban Chatbot:

“Daur ulang plastik membantu mengurangi sampah, menghemat energi, dan menciptakan peluang usaha baru.”

Pseudocode:

```
pertanyaan = "Manfaat daur ulang sampah plastik"
dokumen = load("proyek_lingkungan.pdf")
hasil = retriever(dokumen, pertanyaan)
jawaban = generator(hasil)
print(jawaban)
```

Penjelasan:

Chatbot bisa bantu ide proyek siswa, bahkan memunculkan gagasan bisnis berbasis lingkungan.

Catatan

Dari lima contoh di atas, kita bisa lihat bahwa *RAG* bisa dipakai di hampir semua pelajaran:

- Dari **IPA** sampai **Bahasa Indonesia**,
- Dari **Matematika** sampai **Kewirausahaan**.

Dan semuanya bisa dibuat dengan teknologi **open source** seperti:

- *Python* untuk logika utama,
- *Pandas* untuk membaca data,
- *FAISS* atau *Chroma* untuk membuat *index*,
- *Keras* dan *Neural Network* untuk model AI yang lebih cerdas.

Dengan *RAG sekolah*, kamu bisa membuat **chatbot yang jadi teman belajar, bukan cuma mesin pencari**.

Contoh Soal dan Pembahasan

Soal 1 — Mengenal Konsep Dasar RAG

Pertanyaan:

Jelaskan perbedaan antara *retriever* dan *generator* dalam sistem RAG!

Pseudocode:

```
# simulasi sederhana
if tugas == "mencari_dokumen":
    print("retriever bekerja")
elif tugas == "membuat_jawaban":
    print("generator bekerja")
```

Jawaban:

- *Retriever* adalah bagian yang bertugas mencari dokumen atau teks yang relevan.
- *Generator* adalah bagian yang bertugas membuat atau menyusun jawaban dari hasil pencarian tadi.

Pembahasan:

Bayangkan *retriever* itu seperti **satpam perpustakaan** yang mencari buku yang tepat, sedangkan *generator* seperti **guru pintar** yang menjelaskan isi buku itu dengan bahasa mudah. Dua-duanya bekerja sama agar chatbot bisa menjawab dengan cepat dan benar.

Soal 2 — Indexing Dokumen Sekolah

Pertanyaan:

Mengapa proses *indexing* penting dalam sistem RAG, dan bagaimana cara sederhana membuat index dengan Python?

Pseudocode:

```
index = {}
for i, paragraf in enumerate(dokumen):
    index[i] = paragraf
print("Index berhasil dibuat:", len(index), "bagian")
```

Jawaban:

Indexing membantu komputer menemukan bagian teks dengan cepat tanpa membaca semua isi dokumen. Dengan Python, kita bisa menyimpan setiap paragraf dalam *dictionary* agar mudah dicari.

Pembahasan:

Tanpa *index*, sistem RAG akan seperti mencari jarum di tumpukan jerami. Dengan *index*, komputer langsung tahu di mana bagian teks berada — cepat dan efisien!

Soal 3 — Proses Pencarian Informasi

Pertanyaan:

Tuliskan langkah-langkah sederhana saat chatbot mencari jawaban dalam sistem RAG!

Pseudocode:

```
pertanyaan = input("Masukkan pertanyaan: ")
hasil = retriever(pertanyaan, index)
jawaban = generator(hasil)
print("Jawaban:", jawaban)
```

Jawaban:

Langkah-langkahnya:

1. Chatbot menerima pertanyaan dari pengguna.
2. *Retriever* mencari bagian teks yang sesuai dari *index*.
3. *Generator* membuat jawaban berdasarkan hasil pencarian.
4. Chatbot menampilkan jawaban ke pengguna.

Pembahasan:

Inilah “alur kerja” RAG — seperti kerja tim!

Satu bagian mencari, satu bagian menjelaskan.

Teknologi ini bisa dibuat dengan *Python + FAISS + Ollama + Gemma/Mistral*.

Soal 4 — RAG untuk Pelajaran Sekolah

Pertanyaan:

Jika chatbot sekolah ditanya:

“Sebutkan manfaat energi terbarukan!”,
jelaskan langkah kerja sistem RAG hingga menghasilkan jawaban.

Pseudocode:

```
pertanyaan = "Manfaat energi terbarukan"
dokumen = load("IPA_kelas10.pdf")
hasil = retriever(dokumen, pertanyaan)
jawaban = generator(hasil)
print(jawaban)
```

Jawaban:

1. *Retriever* mencari bagian teks yang menjelaskan “energi terbarukan” di dokumen IPA.
2. *Generator* membaca teks tersebut dan menyusunnya jadi kalimat utuh.
3. Chatbot menampilkan jawaban seperti:

“Energi terbarukan bermanfaat untuk menghemat sumber daya alam dan menjaga lingkungan.”

Pembahasan:

Contoh ini menunjukkan bagaimana RAG bisa membantu siswa memahami pelajaran dari dokumen sekolah. Tidak perlu membuka halaman satu per satu, cukup tanya chatbot saja!

Soal 5 — Evaluasi Akurasi Jawaban Chatbot

Pertanyaan:

Bagaimana cara menilai apakah jawaban chatbot dari sistem RAG sudah akurat?

Pseudocode:

```
if jawaban_chatbot == jawaban_dokumen:
    print("Akurat ✅")
else:
    print("Perlu diperbaiki ❌")
```

Jawaban:

Kita bisa membandingkan jawaban chatbot dengan isi asli dokumen. Kalau sama atau mendekati, berarti akurat. Kalau tidak, perlu perbaikan di *retriever* atau *generator*.

Pembahasan:

Evaluasi penting agar chatbot tidak asal menjawab. Di dunia nyata, sistem RAG juga sering diuji pakai dataset pelatihan (*training data*) dan model *neural network* seperti *Keras*. Dengan begitu, chatbot makin cerdas dan akurat!

Catatan

Dari kelima soal ini kamu belajar bahwa:

- *Retriever* dan *generator* adalah dua otak utama RAG.
- *Indexing* membuat pencarian lebih cepat.
- *Python* dan *Pandas* sangat berguna untuk menyimpan dan mengolah data.
- *Keras* dan *Neural Network* bisa dipakai untuk melatih model AI sekolah.
- Evaluasi penting agar chatbot bisa dipercaya sebagai sumber belajar.

Dengan latihan-latihan ini, kamu tidak cuma paham teori RAG, tapi juga siap membuat **chatbot sekolah open source** buatan kamu sendiri — yang bisa jadi **teman belajar cerdas** untuk semua siswa SMK di Indonesia 🇮🇩.

Fakta Menarik (Fun Facts)

1. **RAG itu seperti otak dua sisi!**
Satu sisi untuk *mengingat* (retriever), satu sisi untuk *berpikir dan menjawab* (generator).
2. **Google, Wikipedia, dan ChatGPT pakai konsep mirip RAG.**
Jadi kalau kamu bikin RAG di sekolah, kamu sudah setara dengan sistem pencarian raksasa dunia!
3. **Semua bisa jalan di laptop sekolah.**
Cukup pakai *Python + Pandas + FAISS*, nggak perlu superkomputer.
4. **RAG bukan hanya teori AI, tapi tool bantu nyata.**
Bisa dipakai buat bantu ngerjain PR, cari ringkasan pelajaran, sampai bantu guru bikin soal otomatis.
5. **Model open source seperti Gemma atau Mistral bisa dilatih ulang!**
Artinya kamu bisa bikin AI yang *ngomong pakai gaya anak SMK sendiri*.
6. **FAISS dan Chroma dibuat oleh komunitas open source dunia.**
Kamu juga bisa ikut berkontribusi di sana — gratis, bebas, dan keren!
7. **Python itu bahasa paling populer di dunia AI.**
Karena mudah dibaca, fleksibel, dan cocok untuk pemula yang baru belajar *machine learning*.
8. **Neural network itu terinspirasi dari otak manusia!**
Jadi setiap kali kamu menjalankan model AI, kamu sedang membuat komputer “berpikir”.
9. **RAG bikin AI lebih jujur.**
Karena jawabannya diambil dari dokumen nyata, bukan karangan bebas seperti AI tanpa *retrieval*.
10. **Bikin chatbot sekolah = bikin perpustakaan hidup!**
Kamu nggak cuma baca buku, tapi bikin buku itu bisa bicara balik padamu!

Tips Belajar / Tips Cepat Menghafal

1. **Bayangkan sistem RAG seperti tim sepak bola.**
Retriever jadi pemain yang cari bola (informasi), *Generator* jadi pencetak gol (jawaban).
2. **Belajar dengan proyek kecil dulu.**
Misalnya bikin *chatbot sekolah* cuma pakai satu mata pelajaran — Fisika dulu, nanti baru tambah ke Bahasa Indonesia.
3. **Tulis pseudocode setiap kali belajar.**
Dengan begitu kamu nggak cuma hafal teori, tapi ngerti alur logikanya.
4. **Gunakan *Python Notebook* seperti Jupyter atau Google Colab.**
Gratis, mudah, dan langsung bisa dicoba tanpa instal apa pun.
5. **Belajar sambil main.**
Uji chatbot kamu dengan pertanyaan lucu atau aneh — biar belajar terasa seru, bukan tegang.
6. **Gunakan dataset kecil lebih dulu.**
Misalnya hanya 3–5 dokumen, agar cepat selesai dan bisa diuji langsung.

7. **Jangan takut error!**
Error di Python itu guru terbaik — dia menunjukkan di mana kamu perlu memperbaiki logika.
8. **Gunakan *comment* (#) di setiap baris kode.**
Biar kamu ngerti maksud dari setiap langkah ketika mengulang latihan nanti.
9. **Belajar bareng teman satu kelas.**
Bikin kelompok kecil, satu jadi *retriever master*, satu lagi *generator coder* — teamwork bikin cepat hafal.
10. Ulangi pakai analogi sehari-hari.
Misalnya: RAG = guru + perpustakaan + murid penasaran dalam satu sistem.

Aktivitas Siswa

Aktivitas 1 — Membuat Index Dokumen Sekolah

Tujuan:

Membuat *index* teks dari file kurikulum sekolah agar bisa dicari dengan cepat.

Langkah-langkah:

1. Pilih satu dokumen teks (misalnya *kurikulum_informatika.txt*).
2. Baca isi dokumen dan ubah menjadi potongan-potongan paragraf.
3. Simpan ke dalam struktur data *dictionary* dengan nomor urut.
4. Tampilkan jumlah potongan yang berhasil disimpan.

Pseudocode:

```
buka file kurikulum
baca setiap baris
simpan ke dictionary {id: paragraf}
tampilkan jumlah data yang disimpan
```

Contoh Python Script:

```
docs = {}
with open("kurikulum_informatika.txt", "r") as f:
    for i, line in enumerate(f):
        docs[i] = line.strip()
print("Total data terindex:", len(docs))
```

Output:

Total data terindex: 120

Aktivitas 2 — Membuat Fungsi *Pencarian (Retriever)*

Tujuan:

Membuat fungsi untuk mencari teks yang relevan dengan kata kunci.

Langkah-langkah:

1. Gunakan *keyword* dari pertanyaan pengguna.
2. Cari di *dictionary* dari hasil *indexing*.
3. Tampilkan paragraf yang berisi kata tersebut.

Pseudocode:

```
def cari(kata_kunci, dokumen):
    hasil = []
    untuk setiap id, teks dalam dokumen:
        jika kata_kunci ada di teks:
            hasil.tambah(teks)
    tampilkan hasil
```

Contoh Python Script:

```
def cari(kata_kunci, dokumen):
    hasil = [t for t in dokumen.values() if kata_kunci.lower()
in t.lower()]
    return hasil

print(cari("algoritma", docs))
```

Output:

```
['Algoritma adalah langkah-langkah logis dalam pemrograman...',
'Pelajaran ini mengajarkan dasar algoritma untuk siswa SMK...']
```

Aktivitas 3 — Menambahkan *Generator* Sederhana

Tujuan:

Membuat AI kecil yang bisa merangkai jawaban dari hasil pencarian.

Langkah-langkah:

1. Ambil hasil dari fungsi `cari()`.
2. Gabungkan potongan teks menjadi satu kalimat.
3. Tampilkan hasil sebagai jawaban lengkap.

Pseudocode:

```
def generator(hasil_pencarian):
    ambil 2-3 kalimat dari hasil
    gabungkan jadi jawaban
    tampilkan hasil
```

Contoh Python Script:

```
def generator(hasil):
    if not hasil:
        return "Maaf, tidak ditemukan jawaban."
```

```

        return " ".join(hasil[:2])

hasil = cari("algoritma", docs)
print(generator(hasil))

```

Output:

“Algoritma adalah langkah-langkah logis dalam pemrograman. Pelajaran ini mengajarkan dasar algoritma untuk siswa SMK.”

Aktivitas 4 — Simulasi Chatbot Sekolah

Tujuan:

Membuat chatbot sederhana yang menerima pertanyaan dan menjawab otomatis.

Langkah-langkah:

1. Gabungkan fungsi `cari()` dan `generator()` ke dalam satu sistem.
2. Minta pengguna mengetik pertanyaan.
3. Chatbot mencari dan menampilkan jawaban.

Pseudocode:

```

loop:
    input pertanyaan dari pengguna
    hasil = cari(pertanyaan, dokumen)
    jawaban = generator(hasil)
    tampilkan jawaban

```

Contoh Python Script:

```

while True:
    tanya = input("Masukkan pertanyaan: ")
    if tanya.lower() == "keluar":
        break
    hasil = cari(tanya, docs)
    jawaban = generator(hasil)
    print("Chatbot:", jawaban)

```

Output Simulasi:

Masukkan pertanyaan: Apa itu algoritma?

Chatbot: Algoritma adalah langkah-langkah logis dalam pemrograman. Pelajaran ini mengajarkan dasar algoritma untuk siswa SMK.

Aktivitas 5 — Menguji Akurasi Jawaban Chatbot

Tujuan:

Menilai apakah chatbot memberikan jawaban yang tepat dan akurat.

Langkah-langkah:

1. Buat tabel evaluasi sederhana dengan 3 kolom: pertanyaan, jawaban, dan akurasi.
2. Bandingkan jawaban chatbot dengan isi dokumen asli.
3. Tandai hasilnya dengan “Akurat” atau “Perlu Diperbaiki”.

Pseudocode:

```
for setiap pertanyaan:
    hasil = chatbot(pertanyaan)
    jika hasil mirip dengan isi dokumen:
        catat "Akurat"
    else:
        catat "Perlu Diperbaiki"
```

Contoh Python Script:

```
tes = [
    "Apa itu algoritma?",
    "Jelaskan tujuan pembelajaran Informatika!"
]

for q in tes:
    hasil = generator(cari(q, docs))
    print("Pertanyaan:", q)
    print("Jawaban Chatbot:", hasil)
    print("Status: Akurat  if "algoritma" in hasil else
    "Perlu Diperbaiki )
    print()
```

Output:

Pertanyaan: Apa itu algoritma?

Jawaban Chatbot: Algoritma adalah langkah-langkah logis dalam pemrograman...

Status: Akurat 

Catatan

Melalui lima aktivitas ini, kamu telah belajar:

- Membuat *index dokumen sekolah*.
- Membangun *retriever* untuk pencarian teks.
- Membuat *generator* sederhana.
- Menyatukan keduanya jadi *chatbot sekolah*.
- Mengevaluasi akurasi hasilnya.

Semua dilakukan dengan tool **open source** seperti *Python*, *Pandas*, dan konsep *machine learning*. Inilah langkah awal kamu menjadi **AI Engineer SMK** yang bisa menciptakan **perpustakaan digital cerdas** untuk sekolahmu.

Rangkuman

Pada bab ini, kamu telah belajar cara membangun **sistem pencarian dokumen sekolah berbasis RAG (*Retrieval-Augmented Generation*)**. Teknologi ini menggabungkan dua kekuatan utama, yaitu **retriever** dan **generator**. *Retriever* bertugas **mencari dokumen yang relevan**, sedangkan *generator* bertugas **membuat jawaban berdasarkan isi dokumen tersebut**. Dengan bantuan tool **open source** seperti *Python*, *Pandas*, *FAISS*, dan *Chroma*, kamu bisa membuat sistem cerdas yang membaca dan memahami dokumen sekolah seperti manusia membaca buku.

Melalui konsep RAG, kamu belajar bahwa **AI bisa menjadi asisten belajar pintar**. Bayangkan, kamu tidak perlu membuka satu per satu buku pelajaran atau file PDF — cukup ketik pertanyaan, dan chatbot akan memberikan jawaban berdasarkan dokumen sekolah. Teknologi ini bisa digunakan untuk membantu siswa, guru, bahkan perpustakaan digital. Semua bisa dijalankan di **komputer sekolah** dengan RAM terbatas, karena seluruh sistem dibangun dengan **software open source** tanpa biaya tambahan.

Dengan memahami *machine learning*, *neural network*, dan cara kerja RAG, kamu sudah selangkah lebih dekat menjadi **AI Engineer SMK** yang siap menghadirkan inovasi di sekolah. Proyek sederhana seperti *chatbot kurikulum sekolah* bisa menjadi awal untuk sistem yang lebih besar — misalnya *guru digital*, *pencari soal otomatis*, atau *pembuat rangkuman pelajaran*. Ingat, kekuatan AI bukan hanya di kode, tapi di cara kamu **memecahkan masalah nyata dengan logika, data, dan kreativitas**.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. RAG menggabungkan *retriever* dan *generator*.
Jawab: Benar
2. *Retriever* bertugas menyusun kalimat jawaban.
Jawab: Salah
3. *Indexing* membuat pencarian dokumen lebih cepat.
Jawab: Benar
4. FAISS dan Chroma adalah tools *open source* untuk *vector index*.
Jawab: Benar
5. Chatbot RAG bisa berjalan walau tanpa dokumen apa pun.
Jawab: Salah

B. Pilihan Ganda (10 Soal)

1. Komponen yang mencari potongan teks relevan adalah...

A. Decoder
B. *Retriever*
C. Tokenizer
D. Optimizer

Jawab: B (*Retriever*)

2. Komponen yang menyusun jawaban adalah...

A. Parser
B. *Generator*
C. Encoder
D. Scheduler

Jawab: B (*Generator*)

3. Fungsi utama *index* dalam RAG adalah...

A. Menyimpan gambar
B. Mempercepat pencarian teks
C. Menghapus duplikasi file
D. Mengamankan kata sandi

Jawab: B

4. Format alur RAG yang benar adalah...

A. RAG → Dokumen → Index → Chatbot
B. Dokumen → Chatbot → Index → RAG
C. Dokumen → Index → RAG → Chatbot
D. Chatbot → RAG → Index → Dokumen

Jawab: C

5. Tool yang umum untuk *vector search* adalah...

A. Pandas
B. NumPy
C. FAISS
D. Matplotlib

Jawab: C

6. Library Python yang cocok untuk baca tabel/CSV adalah...

A. TensorFlow
B. *Pandas*
C. Keras
D. OpenCV

Jawab: B

7. Contoh model LLM *open source* yang bisa dipakai via Ollama adalah...

A. Gemma/Mistral
B. GPT-4 tertutup
C. PaLM tertutup

D. BERT-only tanpa generasi

Jawab: A

8. Tujuan RAG di sekolah adalah...

- A. Mematikan internet
- B. Menjawab dari dokumen resmi sekolah
- C. Menghapus kurikulum
- D. Mengganti semua guru

Jawab: B

9. Istilah yang berarti “mengubah dokumen jadi struktur yang mudah dicari” adalah...

- A. Tokenizing
- B. Rendering
- C. *Indexing*
- D. Compressing

Jawab: C

10. Kelebihan RAG dibanding jawab bebas model adalah...

- A. Selalu pakai GPU mahal
- B. Jawaban acak
- C. Berbasis sumber rujukan
- D. Tidak butuh data

Jawab: C

C. Isian Singkat (5 Soal)

1. RAG adalah gabungan _____ dan _____.

Jawab: *retriever, generator*

2. Dua contoh *vector database/index open source* adalah _____ dan _____.

Jawab: FAISS, Chroma

3. Library Python untuk olah data tabel adalah _____.

Jawab: Pandas

4. Proses menyimpan representasi teks menjadi vektor disebut _____.

Jawab: *embedding*

5. Output akhir sistem RAG di bab ini adalah sebuah _____ sekolah.

Jawab: chatbot

D. Soal Explorasi (3 Soal)

Soal 1:

Rancang alur RAG mini untuk menjawab “Apa hukum Newton I?” dari PDF Fisika.

Kunci Jawaban (contoh):

- Langkah:
 - Ekstrak teks PDF → potong per paragraf.
 - Buat *embedding* → simpan ke FAISS/Chroma.
 - *Query* “hukum Newton I” → ambil top-k paragraf.
 - Susun jawaban singkat.

- Pseudocode:

```
dokumen = load_pdf("fisika.pdf")
paragraf = split(dokumen)
vektor = embed(paragraf)
index = faiss.index(vektor)

q = "hukum Newton I"
hits = index.search(embed([q]), topk=3)
jawaban = generator(hits.text)
print(jawaban)
```

- Contoh Python sederhana (tanpa embedding nyata, simulasi):

```
docs = ["Hukum Newton I: benda tetap ...",
        "Energi kinetik ...", "Gaya gesek ..."]
def retriever(q, xs): return [t for t in xs
                              if "Newton" in t]
def generator(rs): return rs[0] if rs else
        "Tidak ditemukan"
print(generator(retriever("hukum Newton I",
                          docs)))
```

Soal 2:

Buat skema evaluasi akurasi chatbot RAG untuk 5 pertanyaan kurikulum.

Kunci Jawaban (contoh):

- Langkah:
 - Siapkan 5 pertanyaan + jawaban kunci dari dokumen.
 - Jalankan chatbot → ambil jawaban.
 - Cek kemiripan sederhana (substring).
 - Hitung akurasi = benar/total.
- Pseudocode:

```
tes = [(Q1,A1), ..., (Q5,A5)]
benar = 0
for q,a in tes:
    pred = chatbot(q)
    if a in pred: benar += 1
akurasi = benar/len(tes)
```

- Contoh Python:

```
tes = [("Apa KD Matematika?", "Kompetensi Dasar Matematika"), ("Apa itu algoritma?", "langkah-langkah logis")]
def chatbot(q): return "Kompetensi Dasar Matematika ada di bab 1; algoritma adalah langkah-langkah logis"
benar = sum(1 for q,a in tes if a.lower() in chatbot(q).lower())
print("Akurasi:", benar/len(tes))
```

Soal 3:

Desain pipeline data dari folder dokumen → index → chatbot CLI.

Kunci Jawaban (contoh):

- Langkah:
 - Load semua **.txt** dalam folder **dokumen/**.
 - Buat daftar paragraf + ID dokumen.
 - (Opsional) Buat *embedding* dan simpan di FAISS/Chroma.
 - Bangun CLI: input pertanyaan → cari top-k → tampilkan jawaban.

- Pseudocode:

```
files = list_folder("dokumen/")
paragraf = []
for f in files:
    paragraf += split(read(f))
index = build_index(paragraf)
while True:
    q = input()
    hits = search(index, q, topk=3)
    print(generate(hits))
```

- Contoh Python sederhana (tanpa embedding):

```
import glob
docs = []
for fn in glob.glob("dokumen/*.txt"):
    with open(fn) as f:
        docs += [l.strip() for l in f if l.strip()]

def retriever(q, xs): return [t for t in xs if any(w in t.lower() for w in q.lower().split())][:3]
```

```
def generator(rs): return " ".join(rs) if rs else "Tidak ditemukan."

print(generator(retriever("kurikulum informatika", docs)))
```

Tugas Proyek

Proyek 1 — Buat Pencari Dokumen Kurikulum Otomatis

Tujuan:

Membuat program sederhana untuk mencari kata kunci tertentu dari file teks kurikulum sekolah.

Langkah-langkah:

1. Siapkan file `kurikulum.txt` berisi teks pelajaran (bisa salinan dari buku).
2. Buat *program Python* untuk mencari kata kunci di dalam file.
3. Tampilkan hasil paragraf yang mengandung kata itu.

Pseudocode:

```
buka file dokumen
baca semua baris
minta pengguna masukkan kata kunci
tampilkan semua baris yang mengandung kata itu
```

Contoh Python Script:

```
keyword = input("Masukkan kata kunci: ")
with open("kurikulum.txt", "r") as f:
    for line in f:
        if keyword.lower() in line.lower():
            print(">>", line.strip())
```

Kunci Jawaban / Panduan:

Jika kamu mengetik “algoritma”, program menampilkan semua kalimat yang mengandung kata *algoritma*. Ini adalah dasar dari *retriever* dalam sistem RAG.

Proyek 2 — Membuat Mini Index Dokumen Sekolah

Tujuan:

Membuat *index otomatis* dari dokumen agar sistem bisa mencari lebih cepat.

Langkah-langkah:

1. Pisahkan teks dokumen menjadi paragraf.
2. Simpan dalam *dictionary* dengan nomor urut.
3. Tampilkan daftar index lengkap dengan isinya.

Pseudocode:

```
buka file dokumen
bagi teks menjadi paragraf
simpan tiap paragraf ke dictionary {id: teks}
tampilkan semua id dan paragraf
```

Contoh Python Script:

```
docs = {}
with open("kurikulum.txt", "r") as f:
    for i, line in enumerate(f):
        docs[i] = line.strip()
print("Total index:", len(docs))
print(docs[0]) # tampilkan paragraf pertama
```

Kunci Jawaban / Panduan:

Output menampilkan total paragraf dan potongan teks.

Inilah tahap *indexing* dalam sistem RAG — mirip daftar isi digital.

Proyek 3 — Simulasi RAG untuk Menjawab Pertanyaan

Tujuan:

Membuat program kecil yang bisa mencari teks dan membuat jawaban sederhana dari hasil pencarian.

Langkah-langkah:

1. Gunakan fungsi dari proyek 1 dan 2.
2. Ketika pengguna bertanya, cari kata di dokumen.
3. Ambil 2 kalimat terdekat dan tampilkan sebagai jawaban.

Pseudocode:

```
def retriever(kata, dokumen):
    cari teks yang mengandung kata

def generator(hasil):
    gabungkan 2 kalimat pertama dari hasil
    tampilkan sebagai jawaban
```

Contoh Python Script:

```
def retriever(keyword, docs):
    return [t for t in docs.values() if keyword.lower()
            in t.lower()]

def generator(results):
    if results:
        return " ".join(results[:2])
    return "Maaf, tidak ada jawaban."
```

```
print(generator(retriever("algoritma", docs)))
```

Kunci Jawaban / Panduan:

Jika ditemukan, program akan menampilkan jawaban seperti:

“Algoritma adalah langkah-langkah logis dalam pemrograman. Materi ini penting untuk dasar coding.”

Proyek 4 — Mengukur Akurasi Chatbot Sekolah

Tujuan:

Menilai apakah chatbot sekolah sudah memberi jawaban yang benar.

Langkah-langkah:

1. Buat tabel pertanyaan dan jawaban sebenarnya.
2. Jalankan chatbot untuk menjawab pertanyaan.
3. Bandingkan hasil dengan jawaban yang benar.

Pseudocode:

```
buat list pertanyaan_dan_jawaban
jalankan chatbot untuk tiap pertanyaan
jika jawaban chatbot == jawaban_asli:
    nilai = 1
else:
    nilai = 0
tampilkan total akurasi
```

Contoh Python Script:

```
tes = [
    ("Apa itu algoritma?", "Langkah-langkah logis dalam
pemrograman."),
    ("Apa itu data?", "Kumpulan informasi mentah yang diolah
menjadi informasi.")
]

skor = 0
for q, a in tes:
    jawaban = generator(retriever(q, docs))
    if a.lower() in jawaban.lower():
        skor += 1

print("Akurasi:", (skor/len(tes))*100, "%")
```

Kunci Jawaban / Panduan:

Jika chatbot menjawab 2 dari 2 dengan benar → Akurasi = 100%.

Kamu bisa tambah pertanyaan lain untuk menguji sistem RAG sekolahmu.

Proyek 5 — Membuat Chatbot Sekolah dengan Antarmuka Terminal

Tujuan:

Menggabungkan semua proyek jadi satu chatbot sederhana di terminal.

Langkah-langkah:

1. Gabungkan semua fungsi `retriever()` dan `generator()`.
2. Tambahkan *loop* agar chatbot bisa digunakan berulang kali.
3. Beri opsi `keluar` untuk menghentikan program.

Pseudocode:

```
ulang selamanya:
    tampilkan "Tanya chatbot:"
    jika input == "keluar": berhenti
    hasil = retriever(pertanyaan, dokumen)
    jawaban = generator(hasil)
    tampilkan jawaban
```

Contoh Python Script:

```
while True:
    tanya = input("Tanya Chatbot Sekolah (ketik 'keluar' untuk berhenti): ")
    if tanya.lower() == "keluar":
        break
    hasil = retriever(tanya, docs)
    jawaban = generator(hasil)
    print("Chatbot:", jawaban)
```

Kunci Jawaban / Panduan:

Chatbot bisa menjawab pertanyaan seperti:

Tanya Chatbot Sekolah: Apa itu algoritma?

Chatbot: Algoritma adalah langkah-langkah logis dalam pemrograman...

Kamu baru saja membuat chatbot *open source* versi SMK sendiri!

Catatan

Dengan lima proyek ini, kamu sudah belajar cara membangun sistem **RAG sekolah** dari nol:

- Proyek 1 → *Mencari teks di dokumen (retriever)*
- Proyek 2 → *Membuat index digital (indexing)*
- Proyek 3 → *Menghasilkan jawaban (generator)*
- Proyek 4 → *Mengukur akurasi sistem*
- Proyek 5 → *Membangun chatbot sekolah sederhana*

Semua dilakukan dengan **Python, Pandas, FAISS, dan Keras**, yang bisa dijalankan di laptop sekolah tanpa lisensi mahal. Kamu bukan cuma pengguna AI — kamu **pembuat AI open source muda Indonesia!** 🇮🇩🤖

BAB 4: Deployment Lokal di Lab Sekolah (Tanpa Internet)

Tujuan Pembelajaran

Tujuan utamanya adalah supaya kamu bisa:

- Memahami cara *deployment lokal*, yaitu menjalankan sistem AI di server sekolah tanpa internet.
- Menggunakan teknologi *open source* seperti **Ubuntu**, **Ollama**, dan **OpenWebUI** untuk membuat sistem ini.
- Melatih kemampuan berpikir seperti *engineer*: mandiri, teliti, dan kreatif.
- Menyusun sistem AI yang bisa diakses semua komputer di lab lewat jaringan *LAN*.

Peta Konsep

Supaya gampang membayangkan alurnya, lihat peta konsep berikut:

Model AI + RAG → **Server Lokal (Ubuntu)** → **OpenWebUI** → **Komputer Siswa**

Apersepsi (Pembuka yang Fun dan Menarik)

“Kalau tidak ada internet, apakah AI bisa tetap jalan?”

Pertanyaan ini sering bikin penasaran. Kebanyakan orang berpikir AI hanya bisa hidup kalau terhubung ke internet. Padahal, itu **tidak selalu benar!**

Bayangkan kamu di lab komputer. Internet mati, tapi kamu tetap bisa buka browser dan tanya:

“Apa itu *machine learning*?”

Dan ajaibnya, AI langsung menjawab dengan lancar! Semua itu bisa terjadi karena AI dijalankan di *server lokal sekolahmu*.

Server itu seperti **otak besar** di pojok lab, sementara komputer siswa seperti **pintu kecil** untuk bertanya. Setiap kali kamu bertanya, data mengalir lewat *LAN* ke server — lalu AI berpikir dan mengirim jawabannya kembali.

Semuanya pakai teknologi *open source*:

- **Ubuntu 24.04:** sistem operasi gratis dan kuat.
- **Python & Pandas:** bahasa dan pustaka yang jadi otak utama.
- **Keras & Neural Network:** tool untuk membuat AI belajar.
- **Ollama & OpenWebUI:** untuk menjalankan dan menampilkan AI di browser.

Penjelasan Konsep

Pernahkah kalian bertanya: “**Kalau tidak ada internet, apakah AI bisa tetap jalan?**”

Jawabannya: **Bisa banget!** 😊

Caranya adalah dengan **deployment lokal**. *Deployment* artinya menjalankan aplikasi supaya bisa dipakai banyak orang. Kalau biasanya AI dijalankan di *cloud* (server internet), kali ini kita jalankan **di server sekolah sendiri**.

Kenapa Penting?

- **Sekolah di desa** kadang tidak punya internet stabil.
- **Lab komputer** biasanya hanya terhubung lewat jaringan lokal (LAN).
- Dengan *deployment offline*, semua siswa tetap bisa pakai AI walau tanpa internet.

Bayangkan: server sekolah jadi seperti “**otak pusat**”, sedangkan komputer siswa jadi seperti “akses pintu” untuk bertanya ke AI.

Teknologi Open Source yang Dipakai

- **Ubuntu 24.04 Server** → sistem operasi gratis, aman, kuat.
- **Ollama** → untuk menjalankan model AI di komputer lokal.
- **OpenWebUI** → tampilan berbasis web, agar siswa bisa akses AI lewat browser.
- **Python, Pandas, Machine Learning, Keras, Neural Network** → otaknya AI untuk belajar dan menjawab.

Cara Instalasi di Ubuntu 24.04 Server

Sekarang kita buat langkah **step-by-step** biar jelas:

Update Ubuntu

```
sudo apt update && sudo apt upgrade -y
```

Install Git dan Curl (tool bantu penting)

```
sudo apt install git curl -y
```

Install Docker (biar gampang jalanin Ollama dan OpenWebUI)

```
sudo apt install docker.io docker-compose -y
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

Install Ollama

```
curl -fsSL https://ollama.com/install.sh | sh
```

Setelah selesai, cek dengan:

```
ollama --version
```

Pull Model AI Open Source (contoh: Gemma3 1B)

```
ollama pull gemma3:1b
```

Bisa juga Mistral atau LLaMA.

Clone OpenWebUI

```
git clone https://github.com/open-webui/open-webui.git
cd open-webui
```

Jalankan OpenWebUI dengan Docker

```
docker-compose up -d
```

Akses di Browser

```
http://IP-Server:3000
```

Semua komputer di lab bisa akses lewat LAN.

Ilustrasi Sederhana

Bayangkan lab SMK punya 1 komputer server di pojok ruangan.

Server itu jalanin **Ubuntu + Ollama + OpenWebUI**.

Semua komputer siswa dihubungkan lewat LAN.

Siswa tinggal buka browser dan tanya:

“Apa rumus gaya Newton?”

Chatbot sekolah langsung menjawab, meski internet mati total.

Fun Point untuk Siswa

- *Deployment* = bikin AI siap dipakai bersama.
- **Lokal** = AI tinggal di sekolah, bukan di awan (*cloud*).
- **Ubuntu + Ollama + OpenWebUI** = kombo open source yang keren.
- **LAN sekolah** = jalan tol data, semua bisa akses AI tanpa internet.

Contoh Kasus atau Ilustrasi

Kasus 1: Chatbot Sekolah Tanpa Internet

Situasi:

Sekolah sedang tidak punya koneksi internet. Tapi guru ingin siswa tetap bisa menggunakan chatbot pembelajaran.

Langkah-langkah di OpenWebUI:

Buka browser dan ketik alamat:

`http://192.168.1.10:3000`

1. (ganti IP dengan IP server sekolah)
2. Pilih model **Gemma3:1b** dari daftar model.
3. Ketik pertanyaan di kolom chat:

“Jelaskan apa itu *machine learning* dengan contoh yang sederhana.”

4. Tunggu beberapa detik. AI sekolah langsung menjawab tanpa internet.

Ilustrasi GUI:

Chatbot muncul di layar browser dengan logo sekolah dan nama “AI Sekolah SMK Pintar.”

Jawaban AI ditampilkan di jendela chat secara *real-time*.

Hasil:

“*Machine learning* adalah cara komputer belajar dari data.

Contohnya: komputer belajar mengenali wajah temanmu dari foto-foto sebelumnya.”

Kesimpulan:

AI sekolah tetap bisa membantu siswa belajar walau koneksi internet mati total. Keren, kan?

Kasus 2: Chatbot Membaca Dokumen Pelajaran Sekolah (RAG Lokal)

Situasi:

Guru ingin chatbot bisa menjawab pertanyaan dari buku pelajaran sekolah.

Langkah-langkah di OpenWebUI:

1. Buka tab **Documents** di sisi kiri tampilan.
2. Klik **Upload File**, pilih file **Fisika_Kelas10.pdf**.
3. Setelah upload selesai, klik **Index Document**.

4. Kembali ke tab **Chat**, lalu tulis:

“Dari buku Fisika Kelas 10, jelaskan bunyi Hukum Archimedes.”

5. AI akan membaca isi PDF dan menjawab dengan bahasa yang mudah dimengerti.

Ilustrasi GUI:

Tampil panel *Documents* di kiri dengan daftar file PDF sekolah.
AI menjawab langsung dari isi buku, bukan dari internet.

Hasil:

“Hukum Archimedes menyatakan bahwa benda yang dicelupkan ke dalam cairan akan mendapat gaya ke atas sebesar berat cairan yang dipindahkan.”

Kesimpulan:

Chatbot bisa membaca buku pelajaran sekolah tanpa koneksi online.
Inilah kekuatan *RAG (Retrieval-Augmented Generation)* lokal.

Kasus 3: Chatbot Bimbingan Konseling (BK) Sekolah

Situasi:

Guru BK ingin chatbot membantu siswa memahami perencanaan karier dan jurusan.

Langkah-langkah di OpenWebUI:

1. Buka OpenWebUI di browser.
2. Pilih model **Gemma3:1b** (bisa pakai *offline model* seperti Mistral juga).
3. Ketik pertanyaan:

“Saya suka komputer dan menggambar, jurusan kuliah apa yang cocok?”

4. Lihat jawaban chatbot yang memberi saran karier berdasarkan minat siswa.

Ilustrasi GUI:

Chat AI tampil dengan gaya ramah dan formal.
Di sisi kanan ada nama profil “Konselor Virtual Sekolah.”

Hasil:

“Kalau kamu suka komputer dan menggambar, coba pertimbangkan jurusan Desain Komunikasi Visual atau Multimedia.”

Kesimpulan:

Chatbot lokal bisa digunakan untuk membantu siswa tanpa harus membuka website luar.

Kasus 4: Chatbot Soal Ulangan Otomatis

Situasi:

Guru Informatika ingin membuat latihan soal otomatis untuk siswa, tapi internet mati.

Langkah-langkah di OpenWebUI:

1. Buka halaman chat.
2. Ketik prompt:

“Buatkan 5 soal pilihan ganda tentang *neural network* untuk siswa SMK lengkap dengan jawabannya.”

3. Tunggu beberapa detik.
4. Chatbot menampilkan daftar soal dan kunci jawabannya.

Ilustrasi GUI:

Tampilan *OpenWebUI* berisi daftar soal dan opsi A–D yang diketik otomatis oleh AI.

Hasil:

- ① Apa fungsi *neuron* dalam jaringan saraf?
- A. Penyimpanan data
 - B. Pengolah informasi
 - C. Media komunikasi
 - D. Antarmuka pengguna

Jawaban: B

Kesimpulan:

Chatbot lokal bisa membantu guru membuat soal otomatis, tanpa Word, tanpa internet!

Kasus 5: Chatbot Asisten Lab Sekolah

Situasi:

Di lab SMK, siswa sering lupa cara menjalankan server lokal atau Docker. Chatbot bisa jadi asisten lab digital.

Langkah-langkah di OpenWebUI:

1. Buka *OpenWebUI* di komputer siswa.
2. Ketik prompt:

“Tolong jelaskan langkah-langkah menjalankan Docker dan *OpenWebUI* di Ubuntu.”

3. Tunggu respons AI.
4. Chatbot menampilkan perintah *step-by-step* untuk setup sistem.

Ilustrasi GUI:

Tampilan chat memperlihatkan langkah-langkah praktis dalam bentuk poin. Semua siswa bisa membaca panduannya dari browser.

Hasil:

“Langkah-langkah:

1. Jalankan perintah `sudo systemctl start docker`.
2. Masuk ke folder `open-webui`.
3. Jalankan `docker-compose up -d`.
4. Akses `http://IP-server:3000` di browser.”

Kesimpulan:

Chatbot lokal jadi asisten teknis yang siap membantu siswa belajar sistem server dan *deployment*.

Rangkuman Lima Kasus

No	Nama Kasus	Fungsi Utama	Teknologi Dipakai
1	Chatbot Sekolah Tanpa Internet	Chat lokal untuk belajar	OpenWebUI + Ollama
2	Chatbot Baca Buku Sekolah	AI membaca dokumen pelajaran	OpenWebUI (RAG)
3	Chatbot BK Sekolah	Saran karier siswa	OpenWebUI
4	Chatbot Soal Otomatis	Membuat soal pelajaran	OpenWebUI
5	Chatbot Asisten Lab	Panduan server & Docker	OpenWebUI + Ollama

Catatan

Dari kelima contoh di atas, kamu bisa lihat bahwa:

- Semua bisa dijalankan **tanpa internet**.
- Semua memakai **teknologi open source** (Ubuntu, Ollama, OpenWebUI).
- Semua siswa bisa **berinteraksi dengan AI lewat browser**.

Jadi, walau sinyal hilang, **AI sekolah tetap jalan!**

Dengan sistem *deployment lokal*, kamu bisa belajar, membuat soal, bahkan berkonsultasi — semuanya offline dan mandiri.

Contoh Soal dan Pembahasan

Soal 1: Apa itu Deployment Lokal?

Pertanyaan:

Dalam sistem AI sekolah, apa arti dari *deployment lokal*?

Jawaban:

Menjalankan sistem AI di server sekolah agar bisa diakses lewat jaringan lokal (LAN) tanpa internet.

Pembahasan:

Biasanya AI berjalan di *cloud* (seperti OpenAI atau Google Cloud). Tapi di sekolah, kita menggunakan **Ollama** di Ubuntu Server dan **OpenWebUI** sebagai tampilan web. Jadi, *deployment lokal* artinya AI “tinggal” di sekolah dan bisa diakses oleh semua komputer siswa lewat browser.

Langkah di OpenWebUI:

Buka browser di komputer siswa.

Ketik alamat:

`http://192.168.1.10:3000`

AI siap digunakan walau internet mati!

Soal 2: Apa Bedanya Cloud dan Lokal?

Pertanyaan:

Apa perbedaan utama antara *AI di cloud* dan *AI lokal di sekolah*?

Jawaban:

- *Cloud AI* butuh internet, datanya tersimpan di luar sekolah.
- *AI lokal* bisa berjalan tanpa internet, datanya disimpan di server sekolah.

Pembahasan:

Ketika internet mati, *cloud AI* tidak bisa diakses. Tapi *AI lokal* tetap hidup di lab karena semua file dan model AI disimpan di dalam server Ubuntu sekolah. Itu sebabnya kita pakai **Ollama** dan **OpenWebUI**, dua tool *open source* yang tidak butuh koneksi online.

Ilustrasi di OpenWebUI:

- Tampilan OpenWebUI tetap terbuka di browser.
- Kamu bisa tanya apa saja ke AI, misalnya:

“Apa itu *machine learning*?”

dan AI tetap menjawab karena modelnya berjalan lokal.

Soal 3: Apa Fungsi Ollama dan OpenWebUI?

Pertanyaan:

Sebutkan fungsi dari **Ollama** dan **OpenWebUI** dalam sistem AI sekolah!

Jawaban:

- **Ollama:** Menjalankan model AI lokal seperti *Gemma 2B* atau *Mistral* di server.
- **OpenWebUI:** Menyediakan tampilan *Graphical User Interface (GUI)* agar siswa bisa mengakses AI lewat browser.

Pembahasan:

Keduanya saling terhubung. Ollama bekerja sebagai “otak”, sementara OpenWebUI adalah “wajah”-nya. Semua siswa di lab cukup buka browser dan langsung chat dengan AI tanpa mengetik perintah di terminal.

Langkah Pengujian di OpenWebUI:

1. Pilih model **gemma :2b**.
2. Ketik pertanyaan:

“Jelaskan cara kerja AI sekolah tanpa internet.”
3. AI menjawab langsung dari server lokal.

Soal 4: Bagaimana Cara AI Membaca Dokumen Sekolah (RAG Lokal)?

Pertanyaan:

Bagaimana caranya agar AI sekolah bisa membaca dokumen pelajaran tanpa koneksi internet?

Jawaban:

Dengan fitur **RAG (Retrieval-Augmented Generation)** di OpenWebUI, AI bisa membaca dokumen lokal.

Pembahasan:

Langkahnya mudah banget:

1. Buka tab **Documents** di OpenWebUI.
2. Klik **Upload File**, lalu unggah file **Pelajaran_Fisika.pdf**.
3. Setelah selesai, klik **Index Document** agar AI bisa membaca isinya.
4. Kembali ke chat dan ketik:

“Jelaskan Hukum Archimedes berdasarkan dokumen Fisika.”
5. AI akan menjawab sesuai isi file lokal itu.

Kesimpulan:

AI membaca buku sekolah langsung dari server, bukan dari internet.
Inilah kehebatan *AI RAG lokal* — cepat, aman, dan 100% *open source*.

Soal 5: Bagaimana Siswa di Lab Bisa Akses Bersama AI Sekolah?**Pertanyaan:**

Bagaimana semua komputer di lab bisa mengakses AI yang sama tanpa internet?

Jawaban:

Semua komputer terhubung ke *server AI lokal* lewat jaringan **LAN (Local Area Network)**.

Pembahasan:

Server utama menjalankan **Ollama + OpenWebUI** di Ubuntu.
Komputer siswa hanya perlu membuka browser dan mengakses:

`http://192.168.1.10:3000`

Semua siswa bisa bertanya ke AI yang sama.

Ilustrasi di Lab:

- Server sekolah = otak utama AI
- Komputer siswa = tangan dan mata yang mengaksesnya
- LAN = kabel penghubung otak dengan semua siswa

Kesimpulan:

Seluruh lab bisa belajar AI bersama-sama tanpa koneksi internet.
AI sekolah benar-benar *mandiri* dan siap jadi asisten belajar kapan saja!

Rangkuman Soal

No	Topik Soal	Inti Jawaban	Teknologi yang Dipakai
1	Pengertian Deployment Lokal	Menjalankan AI di server sekolah	Ollama + OpenWebUI
2	Cloud vs Lokal	Cloud butuh internet, lokal tidak	OpenWebUI
3	Fungsi Komponen	Ollama = otak, OpenWebUI = tampilan	Ubuntu, LAN
4	RAG Lokal	AI baca dokumen sekolah	OpenWebUI (RAG)
5	Akses Bersama	Semua komputer akses lewat LAN	OpenWebUI GUI

Catatan

Dari lima contoh soal ini, kamu belajar bahwa:

- AI sekolah bisa berjalan **offline** tanpa internet.
- Teknologinya 100% **open source** (bebas digunakan).
- Semua siswa bisa mengakses AI lewat **OpenWebUI di browser**.
- Sistem ini cocok untuk **lab SMK**, bahkan di daerah yang sinyalnya lemah.

Jadi, kalau internet mati... jangan panik!

AI sekolahmu tetap hidup dan siap membantu belajar.

Fakta Menarik (Fun Facts)

1. AI tetap bisa *hidup* walau tanpa internet, asal dijalankan di *server lokal*.
2. *Ollama* bisa menjalankan model AI seperti *Gemma* langsung dari komputer sekolahmu.
3. *Deployment lokal* lebih aman, karena semua data siswa tetap tersimpan di sekolah.
4. *Ubuntu 24.04 Server* adalah sistem operasi gratis yang dipakai di banyak universitas dan perusahaan besar di dunia.
5. Banyak *data scientist* profesional memakai *Python + Pandas* yang 100% *open source*!
6. *OpenWebUI* bisa diakses lewat browser tanpa instal aplikasi tambahan — cukup ketik alamat IP server.
7. *LAN sekolah* bisa jadi “jalan tol data” untuk AI, tanpa butuh koneksi luar.
8. *Machine Learning* di server lokal bekerja mirip otak manusia: belajar dari pola data.
9. *Python* punya ribuan pustaka gratis, seperti *Keras* dan *TensorFlow*, untuk bikin AI-mu makin pintar.
10. Banyak sekolah di dunia sedang beralih ke sistem *AI lokal* supaya bisa belajar tanpa gangguan sinyal!

Tips Belajar / Tips Cepat Menghafal

1. **Gunakan analogi!**
Bayangkan *server lokal* seperti otak, dan komputer siswa seperti tangan dan mata. Semua bekerja bareng lewat jaringan *LAN*.
2. **Belajar sambil praktek.**
Jalankan satu per satu perintah di Ubuntu, jangan hanya dibaca.
3. **Diskusikan dengan teman.**
Jelaskan konsep *deployment* seolah kamu yang mengajarkan — otakmu akan mengingat lebih kuat!

4. **Coba eksperimen kecil dengan Python.**

Misal, tampilkan data CSV sederhana pakai *Pandas*.

```
import pandas as pd

data = pd.read_csv("nilai.csv")
print(data.head())
```

5. **Catat perintah penting di sticky note!**

Misalnya `docker-compose up -d` atau `ollama pull gemma:2b`.

Tempel di monitor, biar cepat hafal.

6. **Ulangi 3 kali, lalu ajarkan 1 kali.**

Pola 3+1 ini terbukti bikin ingatanmu tahan lama!

7. **Latihan dalam kondisi offline.**

Biar terbiasa pakai AI sekolah tanpa internet.

8. **Pahami konsep, bukan hafalan.**

Misal: “Kenapa harus *server lokal*?”

Karena *AI-nya butuh tempat tetap untuk berpikir!*

9. **Gunakan logika sederhana.**

Jika `internet=False`, maka `AI_local=True`.

Mudah kan?

```
if internet == "mati":

    print("AI lokal tetap jalan 🚀")
```

10. **Tantang diri sendiri!**

Coba deploy ulang *OpenWebUI* dari nol di lab sekolahmu tanpa lihat catatan.

Kalau berhasil, kamu sudah jadi “AI Engineer SMK”!

Aktivitas Siswa

Aktivitas 1: Jalankan Ollama di Server Sekolah

Tujuan:

Menjalankan mesin otak AI (*model LLM*) secara lokal di server sekolah.

Langkah-langkah:

1. Buka terminal di server Ubuntu 24.04.
Jalankan perintah instalasi Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

2. Cek apakah Ollama berhasil terpasang:

```
ollama --version
```

3. Download (pull) model AI open source, misalnya Gemma 2B:

```
ollama pull gemma:2b
```

4. Jalankan modelnya:

```
ollama run gemma
```

Hasil yang Diharapkan:

AI lokal berhasil berjalan. Server siap menjawab pertanyaan tanpa internet!

Challenge Tambahan:

Coba ubah modelnya ke **mistral** atau **llama2** dan bandingkan hasil jawabannya.

Aktivitas 2: Jalankan OpenWebUI di Server Sekolah

Tujuan:

Membuat tampilan web agar AI bisa diakses lewat browser oleh semua komputer di lab.

Langkah-langkah:

- Buka terminal di server.
- Jalankan perintah berikut:

```
git clone https://github.com/open-webui/open-webui.git
cd open-webui
docker-compose up -d
```

- Tunggu hingga sistem selesai berjalan.
- Cek di browser (server atau komputer siswa) dengan mengetik:

<http://192.168.1.10:3000>

(Ganti 192.168.1.10 dengan IP server sekolah.)

Hasil yang Diharapkan:

Tampilan *OpenWebUI* muncul di browser, siap digunakan untuk chat dengan AI.

Challenge Tambahan:

Coba ganti nama server di pengaturan menjadi “AI Sekolah SMK”.

Aktivitas 3: Membuat Chatbot Sekolah di OpenWebUI

Tujuan:

Melatih siswa membuat chatbot pintar yang bisa menjawab pertanyaan pelajaran sekolah.

Langkah-langkah:

1. Buka halaman *OpenWebUI* di browser.
2. Pilih model **gemma3:1b** (atau model lain yang sudah di-*pull*).
3. Ketik prompt seperti ini di kolom chat:
Kamu adalah chatbot sekolah. Jelaskan rumus gaya Newton dengan cara yang mudah.
4. Tekan Enter dan tunggu jawabannya.
5. Diskusikan hasilnya bersama teman di sebelahmu.
Apakah AI-nya menjawab dengan benar dan mudah dimengerti?

Hasil yang Diharapkan:

AI menjawab dengan penjelasan sederhana, misalnya:

“Gaya Newton: $F = m \times a$, artinya gaya sama dengan massa dikali percepatan.”

Challenge Tambahan:

Coba minta chatbot membuat soal latihan tentang rumus gaya Newton!

Aktivitas 4: Menambahkan Dokumen Sekolah ke AI (RAG Lokal)

Tujuan:

Membuat chatbot bisa membaca dokumen sekolah (misalnya buku PDF atau panduan pelajaran).

Langkah-langkah:

1. Siapkan file **Fisika_Kelas10.pdf** atau dokumen pelajaran lain.
2. Buka tab **Documents** di OpenWebUI.
3. Klik **Upload File**, lalu pilih dokumen tersebut.
4. Setelah diunggah, klik **Index Document** agar AI bisa memproses isinya.

Kembali ke halaman chat dan ketik:

Berdasarkan dokumen Fisika Kelas 10, apa itu hukum Archimedes?

Hasil yang Diharapkan:

AI menjawab langsung berdasarkan isi buku yang diunggah.

Tidak perlu internet — semua data dibaca dari server sekolah!

Challenge Tambahan:

Coba unggah dokumen *Sejarah Indonesia* dan tanyakan tentang peristiwa Proklamasi.

Aktivitas 5: Akses Bersama AI Sekolah Lewat LAN

Tujuan:

Menguji apakah semua komputer di lab bisa mengakses AI sekolah bersama-sama.

Langkah-langkah:

1. Pastikan semua komputer dihubungkan ke jaringan *LAN sekolah*. Setiap siswa buka browser dan akses alamat (misalnya):
`http://192.168.1.10:3000`
2. Masing-masing siswa ajukan pertanyaan berbeda, misalnya:
 - “Apa fungsi *machine learning*?”
 - “Apa bedanya AI dan manusia?”
 - “Apa itu *neural network*?”
3. Amati apakah semua komputer tetap bisa terhubung dan AI merespons dengan cepat.

Hasil yang Diharapkan:

Semua komputer di lab bisa mengakses chatbot sekolah secara bersamaan tanpa error.

AI tetap responsif dan cepat walau internet mati.

Challenge Tambahan:

Coba ubah tema tampilan *OpenWebUI* (gelap atau terang), lalu bandingkan tampilannya dengan temanmu!

Catatan

Dari lima aktivitas ini, kamu telah:

- Menjalankan **Ollama** sebagai otak AI sekolah.
- Mengaktifkan **OpenWebUI** untuk tampilan chatbot berbasis web.
- Membuat chatbot sekolah yang bisa menjawab pertanyaan pelajaran.
- Mengunggah dokumen lokal agar AI bisa membaca buku sekolah.
- Menguji akses bersama lewat *LAN sekolah* tanpa internet.

Rangkuman

Bab ini mengajarkan cara membuat **AI sekolah berjalan tanpa internet**. Kamu sudah belajar bahwa *deployment lokal* artinya **menjalankan sistem AI di server sekolah sendiri**. Server itu ibarat **otak pusat**, dan komputer siswa di lab menjadi **akses pintu** untuk bertanya ke AI. Dengan cara ini, semua siswa tetap bisa belajar, walau sinyal internet mati total. ⚡

Teknologi yang digunakan semuanya **open source** dan gratis. Kita memakai **Ubuntu 24.04 Server** sebagai sistem utama, **Ollama** untuk menjalankan model AI seperti *Gemma* atau *Mistral*, dan **OpenWebUI** untuk tampilannya di browser. Ketiganya bekerja sama membentuk sistem yang kuat, aman, dan tidak tergantung pada *cloud*. Jadi, sekolah bisa punya **AI sendiri** tanpa harus bayar layanan online apa pun.

Dengan sistem ini, siswa dapat:

- Mengakses *chatbot sekolah* lewat LAN lokal.
- Menjalankan *RAG lokal* agar AI bisa membaca dokumen pelajaran sekolah.
- Belajar tentang konsep *machine learning* dan *neural network* secara nyata.

Kesimpulannya:

Deployment lokal membuat AI bisa **hidup di dalam sekolah** tanpa internet.

Dengan bantuan *open source tools* seperti **Ollama** dan **OpenWebUI**, siswa bisa belajar dan bereksperimen langsung dengan AI, sambil memahami cara kerja dunia nyata teknologi modern. Inilah langkah awal menjadi **engineer AI mandiri** di masa depan.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Deployment lokal artinya menjalankan sistem AI di server internet.
Jawaban: Salah
2. Ubuntu 24.04 Server adalah sistem operasi *open source* yang gratis.
Jawaban: Benar
3. Dengan *deployment lokal*, AI tetap bisa digunakan walau tidak ada koneksi internet.
Jawaban: Benar
4. *Ollama* digunakan untuk membuat tampilan web antarmuka AI.
Jawaban: Salah
5. LAN di lab sekolah bisa digunakan untuk menghubungkan semua komputer ke server AI lokal.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Apa tujuan utama *deployment lokal*?
 - a. Menjalankan AI di cloud
 - b. Menjalankan AI di server sekolah
 - c. Menyimpan data di internet
 - d. Menghapus model AI**Jawaban:** b
2. Teknologi apa yang berfungsi sebagai tampilan berbasis web untuk AI?
 - a. Python
 - b. OpenWebUI
 - c. GitHub
 - d. Docker Compose**Jawaban:** b
3. Perintah untuk memperbarui paket di Ubuntu adalah:
 - a. `sudo update package`
 - b. `apt upgrade`

- c. `sudo apt update && sudo apt upgrade -y`
- d. `systemctl update`

Jawaban: c

4. Fungsi *Docker* dalam sistem AI lokal adalah:

- a. Menghapus file sementara
- b. Menjalankan aplikasi dalam *container*
- c. Membuat jaringan LAN
- d. Menyimpan model AI di cloud

Jawaban: b

5. *Ollama* digunakan untuk:

- a. Menjalankan model AI lokal seperti *Gemma*
- b. Mengedit tampilan web
- c. Membuat jaringan komputer
- d. Menyimpan data dokumen

Jawaban: a

6. File *OpenWebUI* biasanya dijalankan menggunakan perintah:

- a. `python app.py`
- b. `docker-compose up -d`
- c. `openwebui start`
- d. `sudo run ai`

Jawaban: b

7. Apa alamat standar untuk mengakses *OpenWebUI* di browser?

- a. `http://localhost:8000`
- b. `http://IP-Server:3000`
- c. `http://127.0.0.1:8080`
- d. `http://school-ai.local`

Jawaban: b

8. Mengapa *deployment lokal* penting bagi sekolah di daerah tanpa internet stabil?

- a. Karena AI bisa jalan offline
- b. Karena gratis tanpa lisensi
- c. Karena lebih cepat
- d. Semua jawaban benar

Jawaban: d

9. Model AI seperti *Gemma* dan *Mistral* termasuk dalam kategori:

- a. Sistem operasi
- b. *Large Language Model*
- c. Database lokal
- d. Aplikasi pengolah gambar

Jawaban: b

10. Perintah untuk memeriksa versi *Ollama* adalah:

- a. `ollama check`
- b. `ollama --version`
- c. `sudo ollama version`
- d. `docker info ollama`

Jawaban: b

C. Isian Singkat (5 Soal)

1. Perintah untuk menginstal *Docker* di Ubuntu adalah _____.

Jawaban: `sudo apt install docker.io docker-compose -y`

2. *Deployment lokal* memungkinkan AI dijalankan di _____.

Jawaban: server sekolah atau lab lokal

3. Sistem operasi yang digunakan untuk server AI sekolah adalah _____.

Jawaban: Ubuntu 24.04 Server

4. Antarmuka AI yang bisa dibuka lewat browser disebut _____.

Jawaban: OpenWebUI

5. Perintah untuk mengambil model AI *Gemma 2B* di *Ollama* adalah _____.

Jawaban: `ollama pull gemma:2b`

D. Soal Explorasi (3 Soal)

Soal 1:

Proyek Eksperimen “Chatbot Sekolah Cerdas”

Bayangkan kamu ingin membuat chatbot lokal di lab sekolah yang bisa menjawab pertanyaan seputar pelajaran dan kegiatan sekolah tanpa internet.

Tuliskan langkah-langkah kamu mulai dari membuka OpenWebUI sampai chatbot siap digunakan oleh teman-teman di LAN sekolah.

Contoh Jawaban Siswa:

- 1. Buka browser dan ketik alamat `http://192.168.1.10:3000`.
- 2. Pilih model *Gemma:2b* dari menu *Models*.
- 3. Ketik prompt: “Kamu adalah chatbot sekolah yang menjawab pertanyaan pelajaran dan jadwal.”
- 4. Simpan prompt dengan nama “AI Sekolah”.
- 5. Tes dengan pertanyaan: “Kapan upacara bendera dilaksanakan?”

Soal 2:

Simulasi “AI Membaca Buku Fisika”

Sekolah ingin membuat AI yang bisa menjawab pertanyaan dari buku *Fisika Kelas 10* secara offline. Tulis cara kamu mengunggah dokumen pelajaran ke *OpenWebUI* agar AI bisa membaca dan menjawab pertanyaan dari isi buku.

Contoh Jawaban Siswa:

1. Masuk ke tab **Documents** di *OpenWebUI*.
2. Klik **Upload File** dan pilih file *Fisika_Kelas10.pdf*.
3. Klik **Index Document** agar AI bisa membaca isi buku.
4. Kembali ke tab **Chat**, lalu ketik pertanyaan:
“Jelaskan hukum Newton 2 berdasarkan buku Fisika.”

Soal 3:

Eksperimen “AI Lab Lokal”

Bayangkan sekolahmu tidak punya internet selama seminggu, tapi guru tetap ingin semua siswa bisa belajar AI. Bagaimana cara kamu memastikan sistem AI tetap bisa digunakan oleh seluruh siswa di lab sekolah?

Contoh Jawaban Siswa:

1. Pastikan server Ubuntu 24.04 tetap menyala dan terhubung ke LAN.
2. Jalankan perintah *docker-compose up -d* untuk memastikan *OpenWebUI* aktif.
3. Pastikan semua komputer siswa terkoneksi ke jaringan lokal.
4. Setiap siswa membuka browser dan mengakses *http://IP-Server:3000*.
5. Semua siswa bisa bertanya ke AI sekolah secara bersamaan tanpa internet.

Catatan

Melalui soal-soal ini, siswa diharapkan mampu:

- Memahami konsep *deployment lokal* di lab sekolah.
- Mengetahui fungsi *Ollama*, *OpenWebUI*, dan *Docker* secara praktis.
- Dapat melakukan eksperimen sederhana untuk menjalankan AI lokal tanpa internet.

“Kalau internet mati, AI sekolahmu tetap hidup. Karena yang penting bukan sinyal, tapi semangat belajarmu!”

Tugas Proyek

Proyek 1 – Chatbot Perpustakaan Sekolah

Tantangan:

Buat chatbot di *OpenWebUI* yang bisa membantu siswa mencari buku berdasarkan nama penulis atau kategori.

Langkah-langkah:

1. Buka **OpenWebUI** di browser:
<http://192.168.1.10:3000>
2. Masuk ke menu **Settings** → **Prompts** → **Create New Prompt**.
3. Buat prompt seperti ini:

“Kamu adalah asisten perpustakaan. Tugasmu membantu siswa menemukan buku berdasarkan judul atau penulis.”

4. Simpan prompt dengan nama **Chatbot Perpustakaan**.
5. Uji dengan pertanyaan seperti:

“Tolong carikan buku karya Andrea Hirata.”

Hasil yang diharapkan:

Chatbot akan menjawab daftar buku dan ringkasan singkatnya.

Contoh Jawaban Siswa:

Chatbot menjawab:

“Buku karya Andrea Hirata: *Laskar Pelangi*, *Sang Pemimpi*, dan *Edensor*. Ketiganya bercerita tentang semangat pendidikan.”

Proyek 2 – AI Asisten Tata Usaha Sekolah

Tantangan:

Buat AI yang bisa membantu membuat surat izin sekolah sederhana.

Langkah-langkah:

1. Di *OpenWebUI*, buat prompt baru dengan nama **Asisten Tata Usaha**.
2. Isi prompt-nya:
“Kamu adalah asisten tata usaha. Buatlah surat izin sakit, surat izin kegiatan, atau surat pengumuman sesuai permintaan pengguna.”
3. Simpan prompt.
4. Uji dengan pertanyaan:
“Buatlah surat izin tidak masuk karena sakit selama 2 hari.”

Hasil yang diharapkan:

AI membuat surat resmi dalam format sederhana.

Contoh Jawaban Siswa:

“Kepada Yth. Bapak/Ibu Guru Wali Kelas XI TKJ,
Dengan hormat, saya yang bertanda tangan di bawah ini... (dst)”

Tujuan Proyek:

Siswa belajar bagaimana AI bisa membantu pekerjaan administrasi sekolah secara offline.

Proyek 3 – AI Pembuat Ringkasan Pelajaran

Tantangan:

Gunakan *OpenWebUI* untuk merangkum isi dokumen pelajaran sekolah tanpa internet.

Langkah-langkah:

1. Siapkan file **Pelajaran_TIK.pdf** atau **Fisika_Kelas10.pdf**.
2. Masuk ke menu **Documents** di OpenWebUI.
3. Klik **Upload File** dan pilih dokumen tadi.
4. Setelah selesai, klik **Index Document** agar AI bisa membacanya.
5. Di kolom chat, ketik:
“Tolong ringkas dokumen ini jadi 5 poin penting.”

Hasil yang diharapkan:

AI menampilkan ringkasan poin-poin penting dari isi dokumen.

Contoh Jawaban Siswa:

“Ringkasan Fisika Kelas 10:
① Gaya adalah tarikan atau dorongan.
② Hukum Newton menjelaskan hubungan gaya dan gerak.
③ Energi tidak dapat diciptakan atau dimusnahkan...”

Tujuan Proyek:

Belajar membuat *RAG lokal* agar AI bisa memahami dokumen pelajaran sekolah.

Proyek 4 – Chatbot Informasi Sekolah

Tantangan:

Buat chatbot yang bisa menjawab pertanyaan umum tentang sekolah (jam masuk, jurusan, kegiatan, dsb).

Langkah-langkah:

1. Di *OpenWebUI*, buat prompt baru dengan nama **Info Sekolah SMK**.
2. Isi prompt-nya seperti ini:
“Kamu adalah chatbot sekolah yang menjawab pertanyaan tentang jadwal, jurusan, dan kegiatan sekolah.”
3. Simpan prompt.
4. Uji dengan pertanyaan:
“Kapan jam masuk dan pulang sekolah di SMK Techno Jaya?”

Hasil yang diharapkan:

Chatbot menjawab seperti petugas informasi sekolah.

Contoh Jawaban Siswa:

“Jam masuk pukul 07.00 WIB dan jam pulang pukul 15.30 WIB.
SMK Techno Jaya memiliki jurusan TKJ, RPL, dan DKV.”

Tujuan Proyek:

Melatih siswa memahami konsep *prompt engineering* di sistem AI lokal.

Proyek 5 – AI Pemandu Wisuda Sekolah

Tantangan:

Buat AI yang bisa menjadi *MC virtual* atau pembawa acara wisuda sekolah.

Langkah-langkah:

1. Buka *OpenWebUI*.
2. Klik **Prompts** → **Create New Prompt**.
3. Isi prompt:
“Kamu adalah pembawa acara wisuda sekolah. Buat naskah MC mulai dari pembukaan, sambutan, hingga penutupan.”
4. Simpan prompt sebagai **MC Wisuda Virtual**.
5. Ketik:
“Tolong buat naskah teks pembawa acara wisuda SMK Informatika Tahun 2025.”

Hasil yang diharapkan:

AI menulis naskah lengkap seperti MC profesional.

Contoh Jawaban Siswa:

“Selamat pagi hadirin sekalian,
Puji syukur kita panjatkan ke hadirat Tuhan Yang Maha Esa,
karena hari ini kita berkumpul dalam acara Wisuda SMK Informatika
2025...”

Tujuan Proyek:

Menunjukkan bahwa *AI lokal* bisa digunakan untuk kegiatan sekolah, bahkan untuk acara formal.

Catatan

Dari kelima proyek di atas, siswa belajar bahwa:

- AI bisa digunakan di sekolah **tanpa internet**.
- Semua sistem berjalan lewat **OpenWebUI GUI**, yang mudah diakses lewat browser.
- Dengan *prompt yang tepat*, AI bisa jadi: pustakawan, TU, guru, hingga MC sekolah.
- Semua proyek ini memakai teknologi **open source**, seperti *Ubuntu, Ollama, dan OpenWebUI*.

Jadi, mulai sekarang, jangan cuma jadi pengguna AI.

Jadilah **pencipta AI sekolah lokal** yang bisa membantu teman dan guru di lab sendiri! 🚀

BAB 5: Dokumentasi & Evaluasi Sistem AI

Tujuan Pembelajaran

Di akhir bab ini, kalian diharapkan mampu:

- Menulis **dokumentasi teknis** proyek AI kalian dengan rapi dan mudah dibaca.
- Melakukan **evaluasi sistem** untuk mengetahui seberapa baik AI yang kalian buat.
- Menyusun **laporan akhir proyek AI sekolah** yang bisa dibaca oleh guru, teman, atau adik kelas kalian.

Bayangkan kalian membuat chatbot sekolah di Ubuntu dengan Python dan Keras. Nah, setelah chatbot itu bisa menjawab pertanyaan, kalian harus bisa menjelaskan:

1. Bagaimana cara instalasinya?
2. Bagaimana cara mengujinya?
3. Seberapa akurat hasilnya?

Itulah inti dari **dokumentasi** dan **evaluasi**. Dua hal ini akan membuat proyek kalian hidup lebih lama dan bisa dikembangkan oleh orang lain.

Peta Konsep (Sederhana)

Sistem → Dokumentasi → Evaluasi → Perbaikan

Kalian bisa bayangkan seperti *game development*:

buat game → tulis panduan bermain → tes bug → rilis versi 2.0!

Apersepsi

“Bayangin kamu bikin chatbot super keren buat sekolah. Tapi pas temanmu mau pakai, dia bingung: ‘Ini cara nyalainnya gimana?’

Nah! Supaya nggak bingung, kalian harus punya **dokumentasi**.

Dan biar tahu chatbot kalian benar-benar pintar, kalian perlu **evaluasi**.

Kalau dokumentasi itu *buku panduan*, maka evaluasi itu *rapor nilai proyek AI-mu!*

Penjelasan Konsep

Setiap kali kalian bikin proyek, pasti ada orang lain yang bertanya:

“Ini sistem cara pakainya gimana?”

Atau:

“Bagus nggak sih hasil chatbot yang kamu buat?”

Nah, disinilah pentingnya **dokumentasi** dan **evaluasi**.

- **Dokumentasi** adalah catatan lengkap tentang cara kerja sistem. Bisa berupa tulisan, gambar, langkah instalasi, sampai contoh penggunaan.
- **Evaluasi** adalah proses menilai apakah sistem sudah berjalan baik atau masih ada kekurangan.

Kalau sistem AI hanya dibuat tanpa dokumentasi, orang lain akan bingung.

Kalau tanpa evaluasi, kita tidak tahu apakah chatbot pintar atau malah ngawur.

Apa itu Dokumentasi?

Dokumentasi = buku petunjuk proyek kalian.

Isinya biasanya:

- Tujuan sistem (misalnya chatbot kurikulum sekolah).
- Pertoalan yang dipakai (Ubuntu, Python, Pandas, Keras, *neural network*, Ollama, OpenWebUI).
- Cara instalasi step-by-step.
- Cara penggunaan.
- Contoh hasil chatbot.

Dokumentasi membuat proyek kalian bisa digunakan lagi oleh orang lain, bahkan oleh adik kelas tahun depan.

Apa itu Evaluasi?

Evaluasi = menguji apakah sistem benar-benar **berfungsi**.

Misalnya:

- Apakah chatbot bisa menjawab soal Fisika dengan benar?
- Apakah waktu respon cepat?
- Apakah sistem tetap jalan meski internet mati?

Evaluasi biasanya pakai **metrik** sederhana, misalnya:

- Akurasi jawaban.
- Waktu respon.
- Tingkat kepuasan pengguna (siswa/guru).

Dengan evaluasi, kalian bisa tahu apa yang perlu diperbaiki.

Ilustrasi Kasus

Tim SMK membuat chatbot kurikulum.

- **Dokumentasi** → ditulis cara instalasi di Ubuntu, cara akses di LAN, cara ganti dataset.
- **Evaluasi** → dites oleh siswa: chatbot menjawab 80% soal dengan benar, tapi kadang lambat saat dipakai 20 siswa sekaligus.
- **Perbaikan** → tim memutuskan upgrade server RAM dan memperbaiki indexing dokumen.

Kenapa Penting?

- Dokumentasi membuat proyek tidak hilang begitu saja.
- Evaluasi membuat proyek terus berkembang.
- Tanpa dokumentasi, proyek = “rahasia pribadi”.
- Tanpa evaluasi, proyek = “asal jadi”.

Mix Fun untuk Siswa SMK

- Dokumentasi = “buku harian” proyek AI.
- Evaluasi = “nilai rapor” proyek AI.
- Dengan dua hal ini, proyek kalian bisa dipakai, diuji, dan ditingkatkan.
- Open source seperti **Linux**, **Python**, **Pandas**, **Keras** sukses karena dokumentasi mereka **lengkap dan rapi**.

Contoh Kasus atau Ilustrasi

Contoh Kasus 1 — Chatbot Kurikulum Sekolah

Situasi:

Tim SMK membuat chatbot yang bisa menjawab pertanyaan seputar kurikulum sekolah, seperti jadwal pelajaran dan kompetensi dasar.

Proses Dokumentasi:

- Tim menulis langkah instalasi di Ubuntu (dari *install Python* sampai *menjalankan OpenWebUI*).
- Mereka juga menambahkan contoh pertanyaan yang bisa dijawab chatbot, seperti:
“Apa mata pelajaran kelas 11 RPL?”

Evaluasi:

- Chatbot diuji oleh 10 siswa.
- Hasil: chatbot menjawab benar 8 dari 10 pertanyaan.

- Catatan evaluasi: sistem masih lambat kalau dipakai bersamaan.

Ilustrasi Singkat:

Bayangkan chatbot ini seperti *guru digital* yang bisa ditanya kapan pun. Kalau dokumentasi lengkap, adik kelas tahun depan bisa langsung dijalankan tanpa perlu tanya-tanya ke pembuatnya. Kalau evaluasi dilakukan, tim bisa tahu kenapa chatbot kadang lambat dan bisa memperbaikinya.

Contoh Kasus 2 — Sistem Prediksi Penjualan Kantin Sekolah

Situasi:

Siswa jurusan RPL membuat sistem *machine learning* untuk memprediksi menu apa yang akan laku di kantin minggu depan.

Proses Dokumentasi:

- Menulis sumber data penjualan dari minggu sebelumnya.
- Menjelaskan cara membaca data pakai *Pandas* di Python.
- Menyertakan grafik tren penjualan nasi goreng, sosis bakar, dan jus alpukat.

Evaluasi:

- Model diuji dengan data baru.
- Hasil prediksi: akurasi 78%.
- Kesimpulan: model cukup bagus tapi masih bisa ditingkatkan dengan data tambahan.

Ilustrasi Singkat:

Bayangkan sistem ini seperti *asisten kantin digital*. Dengan dokumentasi, siapa pun bisa tahu cara input data baru. Dengan evaluasi, kalian tahu apakah sistem sudah “pintar” menebak menu favorit siswa minggu depan.

Contoh Kasus 3 — AI Deteksi Emosi Komentar TikTok

Situasi:

Siswa kelas 12 membuat proyek *AI sentiment analysis* untuk mengenali emosi (senang, marah, sedih) dari komentar TikTok.

Proses Dokumentasi:

- Menjelaskan struktur dataset: kolom “teks komentar” dan “label emosi”.
- Menulis langkah pelatihan model *neural network* menggunakan *Keras*.
- Menyertakan contoh hasil prediksi:
“Komentar: Wah keren banget!” → Emosi: Senang.

Evaluasi:

- Uji 500 komentar baru.
- Akurasi model: 86%.
- Catatan: model masih sering salah membedakan komentar sarkasme.

Ilustrasi Singkat:

Proyek ini seperti *AI pembaca perasaan*. Dokumentasi membantu orang lain memahami bagaimana data dikumpulkan dan dilatih. Evaluasi menunjukkan bahwa sistem masih perlu belajar “mengenal nada bicara” — sesuatu yang sulit tapi menantang!

Contoh Kasus 4 — Sistem Absensi Wajah Berbasis Open Source

Situasi:

Tim membuat sistem absensi otomatis berbasis pengenalan wajah menggunakan *OpenCV* dan *Python*.

Proses Dokumentasi:

- Menulis cara instalasi *OpenCV* di Ubuntu.
- Menjelaskan bagaimana kamera digunakan untuk mengambil data wajah siswa.
- Menambahkan contoh output: “Nama: Dita — Hadir pukul 07:05.”

Evaluasi:

- Uji coba dengan 50 siswa.
- Akurasi deteksi wajah: 93%.
- Masalah: pencahayaan di ruang kelas membuat kamera kadang tidak mengenali wajah dengan benar.

Ilustrasi Singkat:

Sistem ini seperti “satpam digital” yang rajin mencatat kehadiran. Dengan dokumentasi, siapa pun bisa pasang ulang di ruang lain. Dengan evaluasi, tim tahu bahwa lampu ruangan ternyata penting untuk hasil akurat.

Contoh Kasus 5 — Sistem Rekomendasi Buku Perpustakaan

Situasi:

Proyek AI sederhana dibuat untuk merekomendasikan buku berdasarkan minat siswa.

Model dibuat menggunakan *Python + Pandas* untuk membaca data peminjaman buku.

Proses Dokumentasi:

- Menulis daftar *library open source* yang digunakan: Pandas, Scikit-learn.
- Menjelaskan format data: nama siswa, judul buku, kategori.
- Menambahkan diagram alur: Data → Analisis → Rekomendasi.

Evaluasi:

- Diuji oleh 20 siswa.
- 17 siswa merasa rekomendasi buku sesuai minat mereka.
- Catatan: sistem masih kurang akurat untuk siswa baru (karena belum punya riwayat membaca).

Ilustrasi Singkat:

Bayangkan AI ini seperti *teman pustakawan digital* yang selalu tahu buku yang kamu suka.

Dokumentasi membantu pustakawan lain memasukkan data baru.

Evaluasi membuat kalian tahu bagaimana meningkatkan sistem untuk semua siswa.

Catatan

Dari lima contoh di atas, kalian bisa lihat bahwa:

- **Dokumentasi** adalah cara untuk *mewariskan ilmu proyekmu* agar tidak hilang.
- **Evaluasi** adalah cara untuk *menilai dan memperbaiki sistem AI* agar makin pintar.
- Semua proyek — dari chatbot sampai sistem prediksi — bisa dibuat dengan tool *open source* seperti **Python, Pandas, Keras, dan OpenCV**.

“AI yang hebat bukan hanya karena kodenya pintar, tapi karena dokumentasinya lengkap dan evaluasinya jujur.”

Contoh Soal dan Pembahasan

Contoh Soal 1 — Pentingnya Dokumentasi

Soal:

Tim SMK membuat *chatbot* sekolah dengan *Python* dan *OpenWebUI*, tapi mereka lupa menulis dokumentasi.

Apa akibatnya?

Jawaban:

Orang lain tidak bisa menjalankan atau memperbaiki chatbot tersebut.

Pembahasan:

Tanpa dokumentasi, proyek seperti “kotak hitam” — hanya pembuatnya yang tahu cara kerja sistem. Misalnya, temanmu ingin menjalankan chatbot itu, tapi tidak tahu langkah-langkah instalasinya. Akhirnya proyek berhenti dan tidak bisa dikembangkan lagi. Jadi, dokumentasi penting supaya proyek bisa diteruskan oleh siapa saja, bahkan adik kelasmu!

Contoh Soal 2 — Isi dari Dokumentasi yang Baik

Soal:

Apa saja isi penting dalam dokumentasi proyek AI berbasis *open source*?

Jawaban:

Tujuan sistem, tool yang digunakan, cara instalasi, cara penggunaan, dan hasil pengujian.

Pembahasan:

Dokumentasi harus menjawab tiga hal:

- *Apa yang dibuat?* → Tujuan dan deskripsi sistem.
- *Bagaimana cara pakainya?* → Langkah instalasi dan penggunaan.
- *Seberapa bagus hasilnya?* → Hasil uji dan evaluasi.

Kalau tiga hal itu lengkap, siapapun bisa menjalankan sistem tanpa harus bertanya ke pembuatnya.

Contoh Soal 3 — Evaluasi Sistem AI

Soal:

Tim membuat sistem *machine learning* untuk mengenali emosi dari teks komentar TikTok.

Dari 100 komentar, sistem mengenali 85 dengan benar.

Berapa akurasi sistemnya?

Jawaban:

Akurasi = 85%.

Pembahasan:

Evaluasi adalah cara untuk menilai seberapa baik sistem AI bekerja.

Rumusnya sederhana:

$$\text{Akurasi} = (\text{Jumlah Jawaban Benar} \div \text{Total Data}) \times 100\%$$

Jadi kalau 85 dari 100 benar, hasilnya 85%.

Angka ini bisa ditulis di dokumentasi proyek sebagai bukti bahwa model kalian bekerja dengan baik.

Contoh Soal 4 — Evaluasi Kecepatan Sistem

Soal:

Chatbot kalian menjawab pertanyaan dalam 3 detik di komputer lab, tapi di laptop lain butuh 10 detik. Apa yang harus ditulis dalam laporan evaluasi?

Jawaban:

Tulis bahwa kecepatan sistem berbeda tergantung spesifikasi perangkat.

Pembahasan:

Evaluasi tidak hanya soal *akurasi*, tapi juga *performansi*.

Kalian harus mencatat hal-hal yang mempengaruhi hasil, seperti:

- RAM terlalu kecil.
- Internet lambat.
- Model AI terlalu besar.

Dengan begitu, laporan kalian jadi lebih jujur dan ilmiah.
Siswa lain bisa tahu bagaimana cara memperbaikinya nanti.

Contoh Soal 5 — Menyusun Kesimpulan Evaluasi

Soal:

Setelah pengujian, tim menemukan bahwa chatbot bisa menjawab 90% soal benar, tapi kadang lambat kalau banyak pengguna. Bagaimana seharusnya kesimpulan evaluasi ditulis?

Jawaban:

Sistem berjalan baik dengan akurasi tinggi, tetapi perlu optimasi agar lebih cepat.

Pembahasan:

Kesimpulan adalah bagian akhir evaluasi.

Tulis secara singkat tapi jelas.

Contohnya:

“Chatbot bekerja dengan akurasi 90%, namun waktu respon perlu diperbaiki agar stabil untuk banyak pengguna.”

Kalimat seperti ini menunjukkan bahwa tim kalian **memahami kekuatan dan kelemahan sistem**, bukan hanya bangga karena “sudah jadi.”

Catatan

Dari lima contoh soal di atas, kamu belajar bahwa:

- Dokumentasi bukan formalitas, tapi *kunci keberlanjutan proyek*.
- Evaluasi bukan sekedar angka, tapi *cermin kualitas kerja tim*.
- AI yang baik tidak hanya *cerdas secara algoritma*, tapi juga *terstruktur secara dokumentasi*.

“Proyek AI tanpa dokumentasi ibarat robot pintar yang kehilangan manualnya.”

Fakta Menarik (Fun Facts)

1. Banyak proyek *open source* gagal bukan karena kodenya salah — tapi karena **tidak ada dokumentasi!**
2. Tim *Python* dan *Linux* menulis dokumentasi lebih lama daripada kodingnya — supaya semua orang bisa paham.
3. Dokumentasi yang baik bisa membuat kamu terkenal di komunitas *open source*!
4. *Machine Learning* tanpa evaluasi = seperti ujian tanpa nilai. Kamu nggak tahu kamu pintar atau belum!
5. Satu baris dokumentasi bisa menyelamatkan ratusan jam debugging.

6. *Neural network* kamu bisa keren, tapi kalau nggak dijelaskan, cuma kamu yang ngerti hasilnya.
7. Banyak proyek *AI sekolah* gagal dilanjutkan karena **tidak ada panduan cara menjalankannya lagi**.
8. Dokumentasi itu seperti "jejak digital" ilmiah — bikin karya kamu abadi!
9. Evaluasi sistem AI sering pakai metrik lucu seperti “akurasi”, “presisi”, dan “recall”. Tapi kalau nggak ditulis, siapa yang tahu?
10. Google, Facebook, dan HuggingFace sukses karena dokumentasi mereka **selalu up-to-date**.

Tips Belajar / Tips Cepat Menghafal

1. Bayangkan dokumentasi seperti **cerita perjalanan proyekmu** — tulis seperti kamu bercerita ke teman.
2. Gunakan *diagram sederhana* saat menjelaskan alur sistem, bukan paragraf panjang.
3. Kalau susah menghafal langkah instalasi, tulis langsung di terminal sambil latihan.
4. Ingat rumus cepat evaluasi:
Akurasi = Jawaban Benar ÷ Total Soal × 100%
5. Gunakan bahasa yang santai dan jelas, bukan istilah rumit, agar semua paham.
6. Baca dokumentasi proyek *open source* terkenal (seperti TensorFlow atau Pandas) buat inspirasi gaya penulisan.
7. Tulis dokumentasi **setiap kali selesai membuat fitur baru**, jangan tunggu sampai akhir proyek.
8. Simpan semua contoh *pseudocode* di satu file `.txt`, biar gampang diingat dan di-copy ulang.
9. Kalau bingung mau mulai dokumentasi, tulis dulu 3 hal: “Tujuan”, “Cara Instal”, dan “Cara Tes”.
10. Jadikan evaluasi seperti “tes harian” buat sistem AI kamu — biar cepat tahu kalau ada yang salah.

Aktivitas Siswa

Aktivitas 1 — Menulis Dokumentasi “Buku Harian Proyek AI”

Tujuan:

Melatih siswa menulis dokumentasi proyek mereka seperti *buku harian teknologi*.

Langkah-langkah:

1. Buka buku catatan atau file teks di laptop.
2. Tulis judul proyek, misalnya: *Chatbot Kurikulum Sekolah*.
3. Jelaskan tujuan proyek dalam 2–3 kalimat sederhana.
4. Tulis tool yang digunakan: Ubuntu, Python, Pandas, Keras, OpenWebUI.
5. Catat setiap perubahan atau ide baru setiap kali kalian bekerja.

Tahapan Jawaban yang Diharapkan:

- Siswa bisa membuat catatan harian proyek.
- Isi mencakup tanggal, aktivitas, dan kendala yang ditemui.
- Dokumen ini bisa dijadikan lampiran laporan akhir proyek.

Catatan:

Dokumentasi harian sederhana bisa membantu kalian mengingat semua langkah saat presentasi di depan guru atau juri lomba.

Aktivitas 2 — Evaluasi Chatbot Sekolah Versi 1.0

Tujuan:

Membiasakan siswa menilai sistem AI yang sudah dibuat dan menemukan bagian yang bisa diperbaiki.

Langkah-langkah:

1. Siapkan 10 pertanyaan dari pelajaran sekolah (misalnya Fisika atau Informatika).
2. Jalankan chatbot dan tanyakan satu per satu.
3. Catat hasilnya di tabel:
 - Kolom 1: Pertanyaan
 - Kolom 2: Jawaban chatbot
 - Kolom 3: Benar/Salah
4. Hitung berapa banyak jawaban benar.
5. Diskusikan hasilnya dalam kelompok.

Tahapan Jawaban yang Diharapkan:

- Chatbot menjawab 7–9 pertanyaan dengan benar.
- Siswa menulis analisis singkat, misalnya:

“Chatbot masih sering salah menjawab pertanyaan Fisika yang panjang.”

Catatan:

Inilah *evaluasi akurasi sederhana*—kalian sudah melatih berpikir seperti peneliti AI!

Aktivitas 3 — Menulis Laporan Evaluasi Sistem AI

Tujuan:

Melatih siswa menyusun laporan evaluasi yang ringkas dan terstruktur.

Langkah-langkah:

1. Buka dokumen baru berjudul “Evaluasi Chatbot Sekolah”.
2. Buat bagian-bagian berikut:
 - Deskripsi singkat sistem
 - Hasil pengujian
 - Masalah yang ditemukan

- Saran perbaikan
- 3. Isi setiap bagian dengan kalimat pendek dan jelas.
- 4. Tambahkan hasil diskusi dari Aktivitas 2.
- 5. Simpan file dan beri nama **Laporan_Evaluasi_TimX.docx**.

Tahapan Jawaban yang Diharapkan:

- Siswa dapat menjelaskan hasil tes sistem AI mereka.
- Laporan berisi minimal 1 paragraf hasil evaluasi dan 1 paragraf saran.

Contoh singkat:

“Chatbot menjawab 80% pertanyaan dengan benar. Masih perlu ditingkatkan di topik Matematika. Disarankan menambah dataset latihan baru.”

Aktivitas 4 — Membandingkan Dua Versi Sistem AI

Tujuan:

Mengajarkan siswa cara mengevaluasi dua versi sistem secara objektif.

Langkah-langkah:

1. Pilih dua versi proyek yang berbeda, misalnya:
 - Chatbot versi 1.0
 - Chatbot versi 2.0 (setelah perbaikan)
2. Jalankan kedua versi untuk menjawab 10 pertanyaan yang sama.
3. Catat hasilnya di tabel perbandingan:
 - Versi 1.0: Jawaban benar
 - Versi 2.0: Jawaban benar
4. Hitung peningkatan akurasi atau kecepatan respon.
5. Tuliskan kesimpulan versi mana yang lebih baik.

Tahapan Jawaban yang Diharapkan:

- Versi 2.0 menunjukkan peningkatan (misalnya dari 80% jadi 90%).
- Siswa bisa menjelaskan kenapa versi baru lebih baik, misalnya:

“Model versi 2.0 lebih cepat karena dokumen sudah diindeks ulang.”

Catatan:

Inilah latihan berpikir ilmiah: membandingkan data sebelum dan sesudah *improvement*.

Aktivitas 5 — Presentasi Dokumentasi Proyek

Tujuan:

Melatih siswa menjelaskan dokumentasi dan hasil evaluasi di depan kelas seperti profesional *developer open source*.

Langkah-langkah:

1. Buat slide sederhana (5–7 halaman).
2. Isi dengan:
 - Tujuan proyek
 - Teknologi open source yang digunakan
 - Tantangan saat membuat sistem
 - Hasil evaluasi dan solusi perbaikan
3. Latih presentasi selama 5 menit per kelompok.
4. Sampaikan dengan gaya santai tapi jelas.
5. Guru dan teman memberi umpan balik.

Tahapan Jawaban yang Diharapkan:

- Siswa mampu menjelaskan isi dokumentasi dengan percaya diri.
- Menggunakan istilah seperti *Python*, *Pandas*, *Keras*, dan *neural network* dengan benar.
- Menyampaikan hasil evaluasi dengan jujur dan terbuka.

Catatan:

Kalian tidak sedang dinilai “siapa yang paling hebat”, tapi siapa yang **paling bisa menjelaskan proyeknya dengan jelas dan terbuka** — seperti komunitas *open source* sungguhan!

Catatan

Dengan lima aktivitas di atas, kalian sudah berlatih seperti tim pengembang AI profesional:

- Menulis dokumentasi seperti *developer open source*.
- Mengevaluasi sistem dengan cara ilmiah.
- Berkolaborasi dan berkomunikasi hasil kerja dengan percaya diri.

“Programmer sejati bukan hanya yang bisa coding, tapi juga yang bisa menjelaskan karyanya.”

Rangkuman

Setelah mempelajari bab ini, kalian sudah tahu bahwa **dokumentasi** adalah *jantung dari proyek AI*. Tanpa dokumentasi, proyek akan seperti robot yang kehilangan memori—tidak tahu siapa pembuatnya dan bagaimana cara hidupnya. Dengan dokumentasi yang baik, siapapun bisa memahami sistem kalian, menjalankannya, bahkan mengembangkan versinya sendiri. Semua teknologi *open source* besar seperti **Python**, **Pandas**, dan **Keras** bisa sukses karena dokumentasinya jelas, rapi, dan mudah dipahami.

Selain dokumentasi, kalian juga belajar pentingnya **evaluasi sistem**. Evaluasi bukan sekadar menilai benar atau salah, tapi cara melihat apakah sistem *machine learning* atau *neural network* yang kalian buat benar-benar *cerdas* dan efisien. Dalam evaluasi, kalian bisa mengukur hal-hal seperti **akurasi**, **waktu respon**, dan **tingkat kepuasan pengguna**. Dengan hasil evaluasi, kalian bisa memperbaiki sistem supaya lebih cepat, lebih pintar, dan lebih berguna bagi pengguna.

Terakhir, kalian memahami bahwa dokumentasi dan evaluasi adalah **dua sahabat terbaik proyek AI**. Dokumentasi menyimpan seluruh perjalanan sistem, sedangkan evaluasi memberi nilai dan arah untuk perbaikan berikutnya. Kalau kalian ingin menjadi *developer open source* sejati, biasakan menulis catatan kerja, hasil uji, dan ide perbaikan di setiap proyek. Ingat, **AI yang hebat bukan hanya soal kode**, tapi juga tentang bagaimana kalian menjelaskan dan menilai hasil kerjanya dengan jujur dan profesional. 🚀

Latihan Soal

Benar atau Salah (5 Soal)

Tulislah “Benar” atau “Salah” pada setiap pernyataan berikut!

1. Dokumentasi hanya dibuat untuk proyek besar yang menggunakan *neural network*.
Jawaban: Salah
2. Evaluasi digunakan untuk mengetahui apakah sistem AI sudah bekerja dengan baik.
Jawaban: Benar
3. Dokumentasi membantu orang lain memahami cara kerja sistem tanpa harus bertanya ke pembuatnya.
Jawaban: Benar
4. Sistem AI yang bagus tidak perlu dievaluasi karena sudah pasti akurat.
Jawaban: Salah
5. Tanpa dokumentasi, proyek bisa sulit digunakan kembali oleh orang lain.
Jawaban: Benar

Pilihan Ganda (10 Soal)

Pilih jawaban yang paling tepat!

1. Fungsi utama dokumentasi dalam proyek AI adalah...
 - a. Menambah ukuran file
 - b. Menjelaskan cara kerja sistem
 - c. Membuat sistem lebih cepat
 - d. Menghapus data lama**Jawaban: b**

2. Contoh tool *open source* yang digunakan dalam proyek AI adalah...
 - a. Windows AI Studio
 - b. Python dan Pandas
 - c. Adobe AI Toolkit
 - d. ChatGPT Plus**Jawaban: b**

3. Evaluasi sistem AI biasanya dilakukan untuk...
 - a. Mengukur akurasi dan kinerja sistem
 - b. Membuat tampilan UI baru
 - c. Menambah warna pada output
 - d. Menghapus bug pada server**Jawaban: a**

4. Dalam dokumentasi, bagian “cara instalasi” sebaiknya berisi...
 - a. Daftar pertoolan laboratorium
 - b. Langkah-langkah menjalankan sistem
 - c. Daftar siswa yang membuat proyek
 - d. Pengantar tentang sejarah AI**Jawaban: b**

5. Contoh metrik dalam evaluasi AI adalah...
 - a. Akurasi jawaban dan waktu respon
 - b. Jumlah siswa dalam tim
 - c. Ukuran file proyek
 - d. Warna antarmuka sistem**Jawaban: a**

6. Kalimat berikut yang *benar* tentang dokumentasi adalah...
 - a. Dokumentasi dibuat setelah proyek selesai saja.
 - b. Dokumentasi bisa ditulis sambil proyek berjalan.
 - c. Dokumentasi hanya dibuat oleh guru pembimbing.
 - d. Dokumentasi tidak wajib untuk proyek sekolah.**Jawaban: b**

7. Dalam proyek chatbot sekolah, evaluasi dilakukan untuk...
 - a. Mengetes kemampuan menjawab pertanyaan

- b. Menghapus dataset lama
- c. Mengganti tampilan OpenWebUI
- d. Menambah file gambar ke sistem

Jawaban: a

8. Hasil evaluasi yang baik seharusnya...
- a. Disembunyikan agar tidak dikritik
 - b. Dijadikan bahan perbaikan sistem
 - c. Dihapus setelah proyek selesai
 - d. Tidak ditulis di laporan

Jawaban: b

9. Dokumentasi yang bagus harus...
- a. Panjang dan rumit
 - b. Ringkas tapi jelas
 - c. Hanya berisi teori
 - d. Ditulis tanpa format

Jawaban: b

10. Open source berarti...
- a. Kode sistem bisa diakses dan dikembangkan oleh siapa saja
 - b. Sistem hanya digunakan di sekolah
 - c. Proyek hanya dijalankan dengan izin khusus
 - d. Akses dibatasi oleh pembuat sistem

Jawaban: a

Isian Singkat (5 Soal)

1. Dokumentasi berfungsi sebagai _____ bagi pengguna lain untuk memahami proyek AI.

Jawaban: panduan

2. Evaluasi dilakukan untuk menilai _____ dan kinerja sistem AI.

Jawaban: akurasi

3. Contoh tool *open source* untuk membuat AI adalah _____ dan _____.

Jawaban: Python, Keras

4. Dalam laporan dokumentasi, bagian yang berisi hasil uji sistem disebut bagian _____.

Jawaban: evaluasi

5. Proyek AI tanpa dokumentasi akan menjadi seperti _____ pribadi yang tidak bisa digunakan ulang.

Jawaban: rahasia

Soal Eksplorasi (3 Soal)

Soal 1:

Studi Kasus Dokumentasi Chatbot Sekolah

Bayangkan kamu membuat *chatbot* untuk membantu siswa mencari jadwal pelajaran. Chatbot ini dibuat dengan *Python* dan *OpenWebUI*. Tuliskan hal-hal apa saja yang akan kamu tulis di dokumentasi proyek agar adik kelasmu bisa menjalankan sistem itu tanpa tanya-tanya.

Kunci Jawaban Eksplorasi:

- Tujuan sistem: membantu siswa mencari jadwal.
- Tools yang digunakan: Python, Ollama, OpenWebUI.
- Langkah instalasi: install dependencies, jalankan *server*, buka *localhost* di browser.
- Cara penggunaan: masukkan pertanyaan, chatbot menjawab otomatis.
- Contoh hasil: chatbot menampilkan jadwal hari Senin dan Rabu.

Soal 2:

Analisis Evaluasi Sistem AI

Tim kamu membuat sistem AI untuk mendeteksi kebersihan kelas. Setelah diuji pada 10 gambar, hasilnya: 8 benar, 2 salah. Tuliskan bagaimana kamu akan menulis hasil evaluasinya dalam laporan proyek.

Kunci Jawaban Eksplorasi:

- Akurasi = $(8 \div 10) \times 100\% = 80\%$.
- Sistem cukup baik, tapi masih perlu perbaikan pada pencahayaan gambar.
- Kesimpulan: AI bekerja baik di kondisi terang, tapi kurang akurat di kondisi gelap.

Soal 3:

Perbaikan Sistem Berdasarkan Evaluasi

Setelah mengevaluasi sistem *chatbot pelajaran*, kamu menemukan responnya lambat jika banyak pengguna. Tuliskan langkah-langkah perbaikan dan dokumentasi baru yang harus kamu buat untuk versi berikutnya.

Kunci Jawaban Eksplorasi:

- Langkah perbaikan: upgrade RAM server, optimasi indexing dokumen.
- Tambahkan catatan “Perubahan Sistem” di dokumentasi versi 2.0.
- Lakukan uji coba ulang dan tulis hasil evaluasi baru dengan data performa yang lebih cepat.

Catatan Akhir:

Soal-soal ini membantumu memahami bahwa **menulis dokumentasi dan evaluasi sistem**

AI itu sama pentingnya dengan membuat kodenya.

Kalian bukan hanya *coder*, tapi juga *pencatat sejarah digital* dari proyek open source kalian sendiri!

Tugas Proyek

Tugas Proyek 1 — “AI Pengenal Gaya Belajar Siswa”

Deskripsi:

Buat sistem sederhana yang membantu mengenali gaya belajar siswa (visual, auditori, kinestetik) berdasarkan hasil kuesioner singkat.

Langkah & Tahapan:

1. Buat 10 pertanyaan sederhana tentang cara belajar (misalnya: suka membaca, suka mendengarkan, suka praktek langsung).
2. Kumpulkan hasil dari 10 teman sekelas.
3. Buat tabel hasil dalam format CSV menggunakan *LibreOffice* atau *Google Sheets*.
4. Analisis hasilnya (berapa siswa visual, auditori, kinestetik).
5. Tuliskan dokumentasi:
 - Tujuan sistem
 - Cara pengumpulan data
 - Hasil analisis
 - Kesimpulan

Kunci Jawaban (Contoh Hasil Akhir):

6 siswa visual, 3 auditori, 1 kinestetik.

Sistem berjalan baik, dokumentasi lengkap dengan grafik hasil analisis.

Catatan:

Gunakan Python + Pandas kalau mau menampilkan grafik sederhana, tapi tidak wajib.

Tugas Proyek 2 — “AI Pendeteksi Kebersihan Ruang Kelas”

Deskripsi:

Kalian menjadi “inspektur digital sekolah”!

Buat laporan dokumentasi tentang sistem AI yang bisa mendeteksi kebersihan kelas dari gambar (misalnya *bersih*, *berantakan*).

Langkah & Tahapan:

1. Kumpulkan 10 foto ruang kelas (5 bersih, 5 berantakan).

2. Simulasikan ide sistem AI yang bisa menilai tingkat kebersihan (tidak perlu coding, cukup deskripsikan).
3. Tuliskan dokumentasi:
 - Tujuan sistem
 - tool open source yang bisa dipakai (contoh: *TensorFlow*, *Keras*)
 - Cara kerja singkat (misal: input gambar → analisis → hasil: bersih/berantakan)
 - Hasil evaluasi dari teman sekelas (berapa yang setuju hasilnya akurat).

Kunci Jawaban (Contoh Hasil Akhir):

AI menilai 9 dari 10 foto dengan benar.

Evaluasi: sistem masih salah saat cahaya terlalu gelap.

Dokumentasi lengkap dengan contoh gambar input-output.

Tujuan: Melatih siswa menulis dokumentasi teknis dengan ilustrasi visual.

Tugas Proyek 3 — “AI Rekomendasi Musik untuk Belajar”

Deskripsi:

Kalian membuat sistem rekomendasi musik berdasarkan mood belajar siswa (fokus, santai, semangat).

Langkah & Tahapan:

1. Buat kuesioner kecil: “Kalau belajar, kamu suka musik seperti apa?”
2. Kumpulkan 10 jawaban temanmu.
3. Buat tabel data dan analisis: berapa yang suka musik instrumental, lo-fi, atau pop.
4. Simulasikan bagaimana sistem AI bisa memberi rekomendasi musik sesuai hasil survei.
5. Tulis dokumentasi dengan bagian:
 - Tujuan
 - Dataset (hasil kuesioner)
 - Analisis hasil
 - Evaluasi sistem

Kunci Jawaban (Contoh Hasil Akhir):

5 siswa memilih lo-fi, 3 instrumental, 2 pop.

Evaluasi: mayoritas siswa merasa rekomendasi musik sudah sesuai.

Dokumentasi mencantumkan tabel data dan kesimpulan singkat.

Catatan:

Kalau mau lanjutkan ke versi coding, kalian bisa pakai *Python + Pandas* untuk membaca dan menganalisis CSV.

Tugas Proyek 4 — “AI Prediksi Cuaca Sekolah”

Deskripsi:

Buat dokumentasi proyek mini tentang sistem AI sederhana untuk memprediksi apakah besok hujan atau cerah, berdasarkan data cuaca sebelumnya.

Langkah & Tahapan:

1. Ambil data cuaca 7 hari terakhir dari internet atau buletin lokal.
2. Buat tabel dengan kolom: Hari, Suhu, Kelembaban, Hasil (Hujan/Cerah).
3. Simulasikan ide bagaimana AI membaca data itu untuk memprediksi besok.
4. Buat evaluasi sederhana:
 - Prediksi AI vs hasil nyata.
 - Akurasi hasil (% benar).
5. Tulis dokumentasi yang memuat:
 - Tujuan sistem
 - Sumber data
 - Hasil prediksi
 - Evaluasi

Kunci Jawaban (Contoh Hasil Akhir):

Prediksi AI: 6 dari 7 hari benar (akurasi 85%).

Evaluasi: data kurang stabil karena cuaca berubah cepat.

Dokumentasi rapi dengan tabel hasil pengamatan.

Catatan:

Gunakan *Python* atau *LibreOffice Calc* untuk menghitung akurasi sederhana.

Tugas Proyek 5 — “AI Peningat Jadwal Sekolah”**Deskripsi:**

Buat dokumentasi sistem *open source* sederhana yang membantu siswa mengingat jadwal pelajaran dan tugas.

Langkah & Tahapan:

1. Catat jadwal pelajaran selama seminggu.
2. Tambahkan kolom “Waktu belajar” dan “Prioritas” (misal: penting/santai).
3. Simulasikan sistem AI yang bisa memberi notifikasi (contoh: “Jangan lupa, hari ini ada praktikum AI jam 09.00!”).
4. Buat dokumentasi:
 - Tujuan sistem
 - Tabel data jadwal
 - Alur kerja sistem (input jadwal → pengingat → output pesan)
 - Evaluasi dari pengguna (berapa siswa merasa terbantu).

Kunci Jawaban (Contoh Hasil Akhir):

90% siswa merasa sistem pengingat membantu mengatur waktu belajar.
Evaluasi: fitur pengingat manual masih perlu dikembangkan otomatis.
Dokumentasi dilengkapi alur sistem dan tabel jadwal belajar.

Catatan:

Proyek ini bisa dikembangkan lebih lanjut dengan *Python CLI* sederhana atau *OpenWebUI plugin*.

Catatan

Dari kelima proyek di atas, kamu belajar bahwa dokumentasi bukan sekadar laporan, tetapi **jejak langkah kerja yang bisa diteruskan orang lain**. Evaluasi juga bukan sekadar nilai, tapi **cara memahami kelemahan sistem agar bisa diperbaiki**.

“Programmer hebat tidak hanya menulis kode, tapi juga menulis cerita di balik kode itu.” ✨

BAB 6: Presentasi & Kolaborasi Tim

Tujuan Pembelajaran

Setelah mempelajari bab ini, kamu akan bisa:

- Bekerja sama dalam satu tim untuk menyiapkan *presentasi proyek AI*.
- Menjelaskan alur kerja proyek mulai dari ide, coding, hingga hasil.
- Menunjukkan *demo* AI yang kamu buat, misalnya chatbot sekolah atau sistem prediksi nilai.
- Mendengarkan *feedback* dari teman, guru, atau juri dan memperbaiki hasil kerjamu.

Dengan kata lain, kamu akan jadi **pembuat AI sekaligus komunikator teknologi**. Bukan cuma jago coding, tapi juga bisa ngomong di depan orang dengan percaya diri!

Peta Konsep

Kolaborasi → Presentasi → Feedback → Revisi

Apersepsi (Pemantik Awal)

Bayangkan kamu dan temanmu baru saja menyelesaikan chatbot sekolah berbasis *Machine Learning* di Ubuntu pakai *Python* dan *Keras*.

Hasilnya keren banget — chatbot bisa menjawab pertanyaan tentang pelajaran dan tugas.

Sekarang, bayangkan proyekmu diikuti lomba nasional.

Kalian berdiri di depan juri, layar besar di belakang menampilkan *demo chatbot* buatanmu. Suasana tegang, tapi juga seru!

Kamu siap menunjukkan bahwa teknologi open source bisa melahirkan karya hebat dari anak SMK Indonesia.

Pertanyaannya:

“Kalau proyek AI ini dinilai juri, bagaimana cara presentasinya biar memukau dan mudah dipahami?”

Nah, jawabannya akan kamu temukan di bab ini.

Siapkan semangat, laptop, dan tim terbaikmu!

Penjelasan Konsep

Bayangkan kalian sudah berhasil bikin chatbot sekolah pakai **Python, Pandas, Machine Learning, Keras, dan Neural Network**. Proyek sudah jadi, sudah dites, dan jalan. Tapi... kalau kalian cuma simpan di laptop, siapa yang tahu hasil kerja keras kalian?

Nah, disinilah pentingnya **presentasi** dan **kerja tim**.

- **Kolaborasi** artinya kerja bareng dalam satu kelompok.
- **Presentasi** artinya menyampaikan hasil kerja ke orang lain.
- Kalau digabung, hasil proyek bukan cuma hidup di komputer, tapi juga bisa dilihat, dipahami, dan diapresiasi banyak orang.

Apa itu Kolaborasi?

Kolaborasi = kerja sama tim.

Setiap orang punya peran.

- Ada yang jago coding pakai Python.
- Ada yang jago desain slide presentasi.
- Ada yang jago bicara di depan kelas.

Kalau digabung, tim jadi kuat.

Kerja kelompok juga bikin proyek selesai lebih cepat, karena tugas dibagi rata.

Apa itu Presentasi?

Presentasi = menjelaskan proyek ke orang lain.

Bisa dengan **slide visual**, bisa juga dengan **demo langsung**.

Tujuannya: biar orang lain paham sistem yang kalian buat.

Dalam proyek AI, presentasi biasanya berisi:

- **Tujuan:** chatbot sekolah untuk bantu belajar.
- **Teknologi:** Ubuntu, Ollama, Python, Pandas, Keras, *neural network*.
- **Demo:** chatbot ditanya soal, lalu menjawab.
- **Hasil Evaluasi:** misalnya akurasi jawaban 85%.

Ilustrasi Kasus

Tim SMK bikin chatbot kurikulum.

Mereka presentasi di depan guru/juri:

- Satu siswa jadi pembuka, jelaskan tujuan.
- Satu siswa jelasin teknis, termasuk pakai open source.
- Satu siswa jalankan demo chatbot live.
- Satu siswa tutup presentasi dengan kesimpulan.

Guru langsung paham, juri langsung terkesan.

Kenapa Penting?

- Presentasi bikin proyek dikenal orang lain.
- Kolaborasi bikin kerjaan lebih ringan.
- Feedback dari guru/juri bisa bikin proyek makin bagus.

Mix Fun untuk Siswa SMK

- Presentasi = pameran hasil karya.
- Kolaborasi = gotong royong ala teknologi.
- Demo live = momen paling menegangkan sekaligus keren.
- Feedback = hadiah, karena bikin proyek kalian naik level.

Contoh Kasus dan Ilustrasi

Kasus 1: Chatbot Sekolah Penjawab Soal

Bayangkan kamu dan tim membuat *chatbot sekolah* yang bisa menjawab soal pelajaran.

Satu anggota fokus membuat model *Machine Learning* pakai *Python* dan *Keras*.

Anggota lain menyiapkan *dataset* pertanyaan menggunakan *Pandas*.

Ada juga yang mengatur tampilan chatbot agar mudah digunakan.

Saat hari presentasi tiba:

- Satu siswa membuka dengan cerita, “Kami ingin bantu teman-teman belajar lewat chatbot.”
- Satu siswa menjelaskan cara kerja model.
- Satu lagi melakukan *demo live*.

Chatbot ditanya:

“Apa rumus luas segitiga?”
dan langsung menjawab benar.
Guru dan juri tersenyum kagum!

Pesan dari kasus ini: **kerja tim + komunikasi = presentasi sukses.**

Kasus 2: AI Deteksi Jenis Sampah

Tim berikutnya membuat sistem *AI Vision* untuk mendeteksi sampah organik dan anorganik.

Mereka menggunakan *Python* dan *TensorFlow (Keras)* di Ubuntu Linux — semua open source. Kamera laptop digunakan untuk menguji model secara langsung.

Saat presentasi:

- Tim menunjukkan cara kerja sistem.
- Satu orang menjelaskan bagaimana *neural network* belajar mengenali gambar.
- Anggota lain memegang kamera dan mencoba berbagai jenis sampah.

Ketika bot berkata “Ini sampah organik 🌱”, semua penonton tepuk tangan. Mereka juga menampilkan *slide visual* berisi hasil akurasi model dan proses pelatihannya.

Dari sini, siswa belajar bahwa **presentasi AI bukan hanya bicara — tapi juga menunjukkan bukti nyata.**

Kasus 3: Sistem Prediksi Nilai Siswa

Tim ketiga membuat sistem *machine learning* untuk memprediksi nilai ujian berdasarkan data absen dan tugas. Mereka memproses data di *Pandas* dan melatih model sederhana di *Scikit-Learn*.

Saat tampil di depan kelas:

- Satu orang menjelaskan tujuan proyek.
- Satu orang menunjukkan grafik hubungan antara kehadiran dan nilai.
- Satu orang menjelaskan hasil prediksi model dengan akurasi 88%.

Juri bertanya, “Bagaimana kalau datanya berubah?”

Tim menjawab, “Kami bisa retrain model pakai data baru, karena semua berbasis open source.”

Hebat! Mereka bukan cuma menguasai teori, tapi juga bisa menjawab pertanyaan dengan tenang. Kuncinya: **latihan dan saling dukung antar anggota tim.**

Kasus 4: Analisis Emosi Komentar Siswa

Tim ini tertarik dengan topik *Natural Language Processing (NLP)*.

Mereka membuat sistem *sentiment analysis* untuk membaca emosi dari komentar siswa. Gunanya? Supaya guru tahu suasana hati murid dari platform diskusi sekolah.

Teknologi yang digunakan:

- *Python* dan *Pandas* untuk mengolah teks.
- *Neural Network* untuk mengenali emosi (positif, negatif, netral).

Dalam presentasi:

- Satu orang membaca komentar, “AI itu bikin belajar seru banget!”
- Sistem menampilkan hasil: “Positif 😊”
Semua tertawa kagum.

Mereka menutup dengan kalimat inspiratif:

“AI kami bukan hanya membaca teks, tapi juga memahami perasaan manusia.”

Pesan penting: **AI bisa dibuat bermanfaat untuk lingkungan sekolah.**

Kasus 5: Rekomendasi Buku Perpustakaan Sekolah

Tim terakhir membuat sistem *AI Recommender* sederhana.

Tujuannya: membantu siswa menemukan buku yang sesuai minat mereka.

Teknologi yang digunakan:

- *Python* dan *Pandas* untuk data buku.
- Model sederhana berbasis *content-based filtering*.

Dalam presentasi:

- Satu siswa menjelaskan ide: “Kami ingin perpustakaan jadi lebih pintar.”
- Satu siswa mendemonstrasikan sistem:
Ketika user mengetik “teknologi”, sistem menyarankan “Belajar AI untuk Pemula”.
- Anggota lain menunjukkan bagaimana datanya bisa ditambah dengan mudah.

Guru terkesan karena sistem bisa jalan **tanpa internet**, cukup di *server lokal Ubuntu*.
Proyek ini menunjukkan bahwa **open source tools bisa menghasilkan solusi nyata di sekolah.**

Catatan

Dari lima kasus di atas, kamu bisa belajar bahwa:

- **Kolaborasi** membuat proyek lebih cepat selesai.
- **Presentasi** adalah cara memamerkan hasil kerja kerasmu dengan bangga.
- **Teknologi open source** seperti *Python*, *Pandas*, dan *Keras* bisa digunakan siapa saja tanpa biaya mahal.
- **Feedback** dari guru atau juri itu penting, karena membantu memperbaiki proyek.
- Dan yang paling penting — **percaya diri itu kunci utama saat tampil!**

Contoh Soal dan Pembahasan

Soal 1: Peran dalam Kolaborasi

Sebuah tim SMK membuat *chatbot sekolah* menggunakan *Python* dan *Keras*. Anggota pertama bertugas membuat *dataset*, anggota kedua melatih model, anggota ketiga menyiapkan *slide presentasi*.

Pertanyaan:

Mengapa pembagian peran penting dalam kolaborasi tim AI?

Jawaban:

Karena setiap anggota bisa fokus pada keahlian masing-masing sehingga pekerjaan lebih cepat selesai dan hasilnya lebih baik.

Pembahasan:

Kolaborasi itu seperti membangun jembatan bersama.

Kalau satu orang bekerja sendiri, butuh waktu lama.

Tapi kalau ada yang fokus di *data*, ada yang di *coding*, dan ada yang di *desain presentasi*, maka proyek AI bisa selesai lebih cepat dan rapi.

Selain itu, kerja tim juga melatih komunikasi dan tanggung jawab.

Ingat, proyek AI bukan cuma soal *machine learning*, tapi juga kerja sama antar manusia.

Soal 2: Makna Presentasi Proyek AI

Tim kamu baru saja menyelesaikan sistem *AI Vision* untuk mendeteksi jenis sampah. Sekarang, kalian harus mempresentasikannya di depan guru dan teman-teman.

Pertanyaan:

Apa tujuan utama dari presentasi proyek AI di sekolah?

Jawaban:

Untuk menjelaskan hasil kerja tim agar orang lain paham, menghargai, dan memberi masukan (*feedback*).

Pembahasan:

Presentasi bukan hanya “bicara di depan kelas.”

Tujuan sebenarnya adalah **berbagi hasil kerja dan ide**.

Melalui presentasi, orang lain bisa tahu:

- Teknologi apa yang digunakan (misalnya *Python* dan *TensorFlow*).
- Masalah apa yang diselesaikan.
- Seberapa efektif sistem yang dibuat.

Dengan begitu, kamu bukan hanya “pembuat AI,” tapi juga **pencerita teknologi** yang menginspirasi.

Soal 3: Feedback dan Revisi

Setelah presentasi, guru memberi saran agar sistem chatbot diperbaiki supaya bisa menjawab pertanyaan lebih cepat.

Pertanyaan:

Apa langkah terbaik tim setelah menerima *feedback* dari guru?

Jawaban:

Tim melakukan revisi proyek sesuai masukan agar sistem menjadi lebih baik dan siap digunakan.

Pembahasan:

Dalam dunia *AI dan Machine Learning*, revisi itu hal biasa.

Masukan dari guru atau juri justru membantu tim memperbaiki kelemahan.

Misalnya:

- Mengganti *dataset* agar lebih lengkap.
- Melatih ulang *neural network* supaya hasilnya lebih akurat.
- Menambahkan *fitur baru* agar sistem lebih interaktif.

Intinya: *feedback* bukan kritik negatif, tapi bahan bakar untuk naik level!

Soal 4: Tantangan dalam Kerja Tim

Dalam satu kelompok, ada anggota yang sering terlambat mengerjakan tugas coding Python. Akhirnya proyek sempat tertunda.

Pertanyaan:

Bagaimana cara terbaik tim menyelesaikan masalah seperti ini?

Jawaban:

Dengan komunikasi terbuka dan pembagian tugas ulang supaya semua anggota bisa berkontribusi.

Pembahasan:

Masalah dalam tim itu wajar.

Kuncinya adalah **bicara baik-baik, bukan menyalahkan**.

Tim bisa mengadakan diskusi kecil:

- Apa kendalanya?
- Apakah butuh bantuan belajar *Python* atau *Pandas*?
- Apakah perlu bagi ulang tugas?

Dengan saling memahami, proyek bisa lanjut dengan semangat baru.

Kolaborasi yang baik itu seperti sistem *neural network*: setiap “neuron” saling terhubung dan bekerja bersama.

Soal 5: Kesiapan Sebelum Presentasi

Satu hari sebelum presentasi proyek AI, laptop tim tiba-tiba error saat menjalankan *demo chatbot*.

Pertanyaan:

Apa langkah cerdas yang bisa dilakukan agar presentasi tetap berjalan lancar?

Jawaban:

Menyiapkan cadangan (backup) seperti video demo, file di flashdisk, atau menjalankan chatbot di komputer lain.

Pembahasan:

Persiapan adalah kunci sukses.

Kadang teknologi tidak bisa ditebak — sistem bisa gagal saat hari H.

Maka, tim AI harus berpikir seperti *problem solver*.

- Simpan *backup* proyek di beberapa tempat.
- Siapkan rekaman video *demo* jika sistem tidak bisa dijalankan langsung.
- Latih semua anggota agar bisa menjelaskan proyek tanpa panik.

Dengan begitu, tim tetap terlihat profesional, walau ada kendala teknis.

Smart team always has a plan B!

Catatan

Dari kelima soal di atas, kamu belajar bahwa:

- Kolaborasi bukan sekadar kerja kelompok, tapi **belajar komunikasi dan tanggung jawab**.
- Presentasi bukan cuma bicara, tapi **cara menunjukkan hasil kerja dengan percaya diri**.
- *Feedback* itu bukan kritik, tapi **arah untuk menjadi lebih baik**.
- Masalah dalam tim bisa diselesaikan dengan **komunikasi terbuka**.
- Dan jangan lupa: **selalu siapkan rencana cadangan sebelum tampil!**

Fakta Menarik (Fun Facts)

- *Python* awalnya dibuat hanya untuk hobi, tapi sekarang jadi bahasa utama dalam dunia *AI* dan *Machine Learning*!
- Banyak *startup AI* besar dunia dimulai dari tim kecil mahasiswa yang kerja di garasi atau ruang kelas — seperti kalian di SMK!
- *Neural Network* itu sebenarnya terinspirasi dari cara otak manusia bekerja saat belajar dan mengingat.
- AI seperti *chatbot sekolah* bisa “belajar” menjawab lebih baik hanya dari data percakapan kalian sendiri.
- Proyek *open source* seperti *Keras* dan *Pandas* dibuat oleh komunitas global — artinya, semua orang bisa ikut berkontribusi!

- Banyak lomba AI nasional dan internasional menerima proyek berbasis *open source*, jadi tidak perlu tool mahal untuk bisa juara!
- Presentasi AI yang kuat tidak selalu tergantung grafik canggih, tapi dari cara tim menjelaskan ide dengan jelas dan percaya diri.
- Dalam presentasi, tim AI terbaik bukan yang paling pintar coding, tapi yang paling kompak dan saling percaya.
- AI modern banyak dijalankan di Ubuntu Linux — sistem operasi open source yang gratis dan ringan.
- *Feedback* dari guru atau juri bukan kritik, tapi “bensin” untuk bikin proyek kalian makin cepat dan keren.
- Riset menunjukkan, kerja kelompok bisa meningkatkan kreativitas hingga 50% dibanding kerja sendirian.

Tips Belajar & Tips Cepat Menghafal

- Pecah materi besar jadi potongan kecil — misalnya: “Kolaborasi”, “Presentasi”, “Feedback”, “Revisi”. Hafalkan seperti urutan langkah proyek.
- Gunakan *visualisasi* saat belajar. Buat mind map atau skema AI kalian di kertas atau *whiteboard*.
- Latihan ngomong di depan kaca! Biar nanti saat presentasi, kalian tidak gugup saat menjelaskan *Python* dan *Machine Learning*.
- Coba jelaskan kembali materi ke teman kalian. Kalau bisa jelaskan dengan bahasa sederhana, artinya kalian sudah paham!
- Belajar sebentar tapi sering lebih efektif daripada belajar lama sekali dalam satu hari.
- Gunakan aplikasi catatan open source seperti *Joplin* untuk menyimpan ide proyek AI atau catatan pelajaran kalian.
- Rekam latihan presentasi kalian, tonton ulang, dan catat bagian mana yang bisa diperbaiki.
- Fokus ke poin utama saja saat menjelaskan proyek — jangan semua teori dihafal, cukup pahami alur kerjanya.
- Saat belajar kelompok, bagi tugas hafalan. Satu orang fokus di *AI model*, satu di *dataset*, satu di *demo*. Lalu saling ajar.
- Ingat! Cara terbaik menghafal bukan hanya membaca, tapi **mempraktikkan langsung** lewat proyek kecil berbasis *Python*.

Aktivitas Siswa

Aktivitas 1: Bentuk Tim dan Tentukan Peran

Sebelum mulai presentasi, kalian harus tahu siapa melakukan apa. Setiap anggota punya kekuatan dan peran penting.

Langkah-langkah:

1. Buat kelompok berisi 4–5 orang.
2. Diskusikan keahlian masing-masing teman.
 - Siapa yang jago coding *Python*?
 - Siapa yang bisa mendesain slide?
 - Siapa yang lancar bicara?
3. Tentukan peran:
 - Ketua tim (koordinator proyek)
 - Programmer (fokus di *Machine Learning* dan *Keras*)
 - Data handler (mengatur *dataset* di *Pandas*)
 - Presenter (menjelaskan proyek ke guru/juri)
 - Visual designer (membuat slide dan tampilan demo)
4. Catat pembagian peran di buku catatan kelompok.
5. Saling berjanji untuk menghormati dan mendukung peran masing-masing.

Pesan: Tim yang kuat bukan yang paling pintar, tapi yang paling kompak.

Aktivitas 2: Siapkan Demo Proyek AI

Waktunya menunjukkan karya kalian!

Gunakan *open source tools* agar mudah dan gratis.

Langkah-langkah:

1. Jalankan proyek AI di laptop atau komputer lab.
 - Misalnya chatbot sekolah yang dibuat dengan *Python* + *Keras*.
2. Pastikan model AI berjalan lancar tanpa error.
3. Siapkan 2–3 contoh pertanyaan atau data untuk *demo live*.
4. Latih anggota tim untuk bergantian menjalankan demo (jangan hanya 1 orang).
5. Simulasikan presentasi di depan teman sekelas untuk menguji kesiapan.

Pesan: Demo yang sukses bukan yang sempurna, tapi yang jelas dan mudah dipahami.

Aktivitas 3: Buat Slide Presentasi Open Source

Presentasi yang baik itu **visual dan informatif**, bukan penuh teks.

Kalian bisa pakai *LibreOffice Impress* (gratis dan open source) atau *Canva versi free*.

Langkah-langkah:

1. Buka *LibreOffice Impress*.
2. Buat 5–6 slide dengan struktur:
 - Slide 1: Judul dan nama tim.
 - Slide 2: Tujuan proyek AI.
 - Slide 3: Teknologi yang digunakan (*Python*, *Pandas*, *Keras*).
 - Slide 4: Demo sistem dan hasil.
 - Slide 5: Feedback & rencana revisi.
 - Slide 6: Kesimpulan.

3. Gunakan warna yang kontras dan tulisan yang besar.
4. Tambahkan gambar dari hasil proyek (bisa tangkapan layar atau grafik).
5. Simpan slide dengan format **.odp** agar tetap *open source friendly*.

Pesan: Slide yang bagus itu seperti iklan — singkat, padat, dan menarik!

Aktivitas 4: Latihan Presentasi Kelompok

Sekarang waktunya latihan bicara.

Gunakan gaya *presentasi profesional tapi santai*.

Langkah-langkah:

1. Berdiri di depan kelas sebagai tim.
2. Tentukan urutan berbicara: pembuka → teknis → demo → penutup.
3. Gunakan kalimat sederhana dan hindari membaca teks panjang.
4. Pastikan semua anggota berbicara, meski hanya 1–2 menit.
5. Rekam latihan kalian pakai HP, lalu tonton ulang untuk melihat kekurangan.

Pesan: Presentasi bukan soal menghafal, tapi soal menyampaikan ide dengan percaya diri.

Aktivitas 5: Terima Feedback dan Lakukan Revisi

Setelah latihan atau tampil di depan guru/juri, pasti ada masukan.

Jangan takut! Ini bagian penting dari proses *kolaborasi dan perbaikan*.

Langkah-langkah:

1. Catat semua *feedback* dari guru, teman, atau juri.
2. Diskusikan bersama tim: mana yang bisa langsung diperbaiki?
3. Revisi proyek atau presentasi kalian sesuai saran.
4. Uji kembali sistem AI setelah diperbaiki.
5. Lakukan *rehearsal* terakhir dengan versi revisi.

Pesan: *Feedback* adalah bahan bakar untuk proyek yang lebih baik.

Tim yang mau menerima masukan akan terus berkembang.

Refleksi

Setelah menyelesaikan semua aktivitas:

- Apa tantangan terbesar saat kerja tim?
- Siapa anggota yang paling banyak membantu?
- Apa yang kalian pelajari tentang komunikasi dan tanggung jawab?

Tulis jawaban di buku catatan kelompok kalian.

Guru bisa menggunakannya sebagai bahan penilaian akhir kolaborasi.

Rangkuman

Presentasi dan kolaborasi tim adalah bagian penting dari dunia teknologi dan *Artificial Intelligence (AI)*. Setelah membuat proyek seperti *chatbot sekolah* atau *sistem prediksi nilai*, kalian perlu tahu cara **menjelaskan hasil kerja dengan jelas dan menarik**. Di sinilah kemampuan presentasi dibutuhkan. Dengan bantuan tool *open source* seperti *Python*, *Pandas*, *Keras*, dan *Neural Network*, kalian bisa menunjukkan cara kerja proyek secara nyata dan mudah dipahami. **Tujuan utama presentasi adalah berbagi ide dan solusi**, bukan hanya pamer hasil.

Selain itu, *kolaborasi* menjadi kunci sukses dalam setiap proyek AI. Setiap anggota tim punya peran: ada yang jago *coding*, ada yang ahli desain slide, dan ada yang percaya diri berbicara di depan orang banyak. Kalau semua bekerja sama dengan baik, proyek bisa selesai lebih cepat dan hasilnya lebih keren. **Kerja tim berarti berbagi tanggung jawab dan saling bantu**, bukan saling menunggu. Inilah yang membedakan tim profesional dari kelompok yang hanya kumpul nama.

Terakhir, jangan lupa bahwa *feedback* atau masukan dari guru, juri, maupun teman adalah hal berharga. Dengan *feedback*, tim kalian bisa memperbaiki sistem, menambah fitur, atau meningkatkan akurasi model AI. **Revisi bukan tanda gagal, tapi bukti berkembang**. Dalam dunia teknologi, belajar dari kesalahan adalah langkah menuju kesuksesan. Jadi, teruslah berlatih, berkolaborasi, dan tampil percaya diri saat mempresentasikan hasil karya kalian — karena di masa depan, mungkin proyek sederhana di kelas ini akan jadi cikal bakal *startup AI* besar buatan anak SMK Indonesia!

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Kolaborasi dalam proyek AI berarti bekerja sendiri agar lebih cepat.
Jawaban: Salah
2. Presentasi proyek AI bisa dilakukan dengan bantuan *slide visual* dan *demo langsung*.
Jawaban: Benar
3. *Feedback* dari guru atau juri tidak perlu diperhatikan karena proyek sudah selesai.
Jawaban: Salah
4. Dalam tim AI, pembagian tugas penting agar setiap anggota fokus pada keahliannya.
Jawaban: Benar
5. *Machine Learning* dan *Keras* adalah teknologi open source yang bisa digunakan di proyek sekolah.
Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Tujuan utama dari presentasi proyek AI adalah...
 - a. Mempamerkan laptop baru
 - b. Menjelaskan hasil kerja tim
 - c. Menghindari pertanyaan juri
 - d. Menyimpan proyek di komputer

Jawaban: b

2. Yang termasuk teknologi *open source* untuk AI adalah...
 - a. Photoshop
 - b. MS Excel
 - c. *Python* dan *Pandas*
 - d. CorelDraw

Jawaban: c

3. Dalam tim proyek AI, siapa yang bertugas menjelaskan alur program di depan kelas?
 - a. Programmer
 - b. Presenter
 - c. Data handler
 - d. Desainer

Jawaban: b

4. Salah satu manfaat kerja tim dalam proyek AI adalah...
 - a. Pekerjaan jadi lebih lambat
 - b. Semua tugas dikerjakan sendiri
 - c. Proyek selesai lebih cepat dan efisien
 - d. Tidak perlu komunikasi

Jawaban: c

5. Saat presentasi, *demo live* biasanya dilakukan untuk...
 - a. Menampilkan hasil kerja secara nyata
 - b. Membaca teori panjang
 - c. Menutup presentasi
 - d. Menghapus program lama

Jawaban: a

6. *Feedback* yang baik akan membantu tim untuk...
 - a. Menyerah lebih cepat
 - b. Membuat revisi dan perbaikan sistem
 - c. Mengulang proyek dari nol
 - d. Menghapus semua data

Jawaban: b

7. Dalam *neural network*, komputer belajar dari...
 - a. Data dan pola
 - b. Warna dan bentuk
 - c. Musik dan video

d. Ukuran file

Jawaban: a

8. Kolaborasi yang baik berarti...
- a. Setiap anggota bekerja tanpa komunikasi
 - b. Semua bekerja dengan saling mendukung
 - c. Hanya satu orang yang aktif
 - d. Tidak ada pembagian tugas

Jawaban: b

9. Berikut yang **bukan** bagian dari presentasi proyek AI adalah...
- a. Tujuan
 - b. Teknologi
 - c. Hasil evaluasi
 - d. Nilai raport

Jawaban: d

10. *Pandas* dalam *Python* digunakan untuk...
- a. Mengedit gambar
 - b. Mengelola dan menganalisis data
 - c. Mendesain tampilan web
 - d. Membuat animasi

Jawaban: b

C. Isian Singkat (5 Soal)

1. Kolaborasi berarti bekerja secara _____ dalam satu tim.

Jawaban: bersama

2. Presentasi bertujuan agar orang lain _____ hasil kerja tim.

Jawaban: memahami

3. Dalam proyek AI, teknologi *open source* seperti _____ membantu membuat model dengan mudah.

Jawaban: Python

4. Setelah presentasi, tim harus melakukan _____ jika mendapat masukan dari guru.

Jawaban: revisi

5. Salah satu contoh proyek AI di sekolah adalah membuat _____ untuk menjawab pertanyaan pelajaran.

Jawaban: chatbot

D. Soal Eksplorasi (3 Soal)

Soal 1:

Kasus: Presentasi Tim Chatbot Sekolah

Kamu dan tim berhasil membuat chatbot sekolah menggunakan *Python* dan *Keras*. Namun, saat *demo live* di depan juri, sistem sempat error dan tidak menjawab pertanyaan.

Tugas: Jelaskan langkah-langkah yang akan kamu dan tim lakukan untuk mengatasi situasi tersebut agar presentasi tetap berjalan lancar.

Contoh Jawaban Eksplorasi:

- Tetap tenang dan tidak panik di depan juri.
- Segera gunakan video *backup demo* yang sudah disiapkan sebelumnya.
- Salah satu anggota menjelaskan logika kerja chatbot sambil menampilkan kode atau diagram.
- Setelah presentasi, tim memperbaiki error dan melaporkan hasil revisi ke guru/juri.

Soal 2:

Kasus: Peran dalam Kolaborasi Tim AI

Dalam kelompok proyek AI, terdapat 4 anggota: A (Programmer), B (Desainer Slide), C (Data Handler), dan D (Presenter).

Tugas: Jelaskan bagaimana cara keempat anggota ini bekerja sama agar proyek AI berjalan efektif dari awal hingga presentasi akhir.

Contoh Jawaban Eksplorasi:

- A membuat logika program menggunakan *Python* dan *Machine Learning*.
- B mendesain *slide visual* untuk mendukung presentasi.
- C mengelola *dataset* dengan *Pandas*.
- D melatih cara berbicara agar bisa menjelaskan sistem ke juri dengan jelas.
- Semua anggota saling memeriksa pekerjaan sebelum tampil di depan kelas.

Soal 3:

Kasus: Menilai Feedback dari Guru

Setelah presentasi, guru memberikan *feedback*: “Tampilan AI bagus, tapi jawabannya masih sering salah.”

Tugas: Jelaskan langkah-langkah timmu untuk memperbaiki proyek agar hasilnya lebih akurat di versi berikutnya.

Contoh Jawaban Eksplorasi:

- Evaluasi dataset: tambahkan lebih banyak contoh pertanyaan dan jawaban.
- Lakukan *retraining* model di *Keras* dengan data yang lebih bersih.
- Uji hasilnya kembali dan bandingkan akurasi.

- Siapkan *slide update* untuk menjelaskan perubahan versi sistem di presentasi berikutnya.

Catatan

Latihan ini membantu siswa SMK memahami bahwa proyek AI bukan hanya soal *coding*, tapi juga **kerja tim, komunikasi, dan kemampuan presentasi**. Dengan semangat *open source*, setiap ide bisa dikembangkan menjadi solusi nyata di sekolah — asal kalian berani mencoba dan berkolaborasi!

Tugas Proyek

Proyek 1 – AI Pengingat Jadwal Belajar Sekolah

Buat sistem sederhana yang bisa mengingatkan jadwal pelajaran atau waktu belajar siswa secara otomatis. Gunakan *Python* dan *Pandas* untuk menyimpan jadwal dan mengatur waktu pengingat.

Langkah-langkah:

1. Kumpulkan data jadwal pelajaran (hari, jam, mata pelajaran, guru).
2. Simpan data tersebut dalam *file* CSV dan baca dengan *Pandas*.
3. Buat fungsi sederhana di *Python* yang menampilkan notifikasi atau pesan pengingat.
4. Presentasikan hasilnya dalam bentuk demo: sistem akan menampilkan “Waktunya belajar AI, jam 10:00!”
5. Satu orang menjelaskan logika program, satu orang menampilkan hasil *demo live*, dan satu orang membuat slide presentasi.

Tujuan: Melatih kerja tim dan kemampuan presentasi proyek *open source* kecil yang bermanfaat di sekolah.

Proyek 2 – Analisis Komentar Positif dan Negatif di Forum Kelas

Kalian membuat *mini AI* untuk membaca emosi dari komentar siswa di forum kelas online. Gunakan *Python* dan *Machine Learning* sederhana untuk memisahkan komentar positif dan negatif.

Langkah-langkah:

1. Kumpulkan 20–30 komentar dari forum diskusi sekolah.
2. Tandai secara manual komentar positif dan negatif.
3. Gunakan *Pandas* untuk membaca dan mengolah data.
4. Buat model sederhana dengan *Keras* atau *Scikit-learn* untuk klasifikasi dua label: positif dan negatif.
5. Tampilkan hasil dalam *slide presentasi* — tunjukkan grafik jumlah komentar positif vs negatif.

Tujuan: Melatih pemahaman *machine learning* sederhana serta cara menyampaikan hasil analisis data secara visual.

Proyek 3 – Prediksi Nilai Siswa dengan Data Kehadiran

Tim kalian membuat sistem *AI Prediction* untuk memperkirakan nilai akhir berdasarkan kehadiran dan tugas. Gunakan *Python* dan *Pandas* untuk analisis data.

Langkah-langkah:

1. Buat dataset sederhana berisi: nama siswa, jumlah hadir, nilai tugas, dan nilai akhir.
2. Latih model sederhana untuk memprediksi nilai akhir dari dua fitur: kehadiran dan tugas.
3. Gunakan *matplotlib* untuk menampilkan grafik hasil prediksi.
4. Buat *slide presentasi* dengan bagian: Tujuan – Dataset – Model – Grafik Hasil – Kesimpulan.
5. Satu siswa menjadi presenter utama, satu siswa menjelaskan cara kerja model, dan lainnya bertugas menyiapkan visualisasi.

Tujuan: Mengajarkan *kolaborasi data analysis* dan kemampuan menyampaikan hasil dengan percaya diri.

Proyek 4 – Chatbot Motivasi Belajar untuk Siswa SMK

Tim kalian membuat *chatbot sederhana* yang memberikan motivasi belajar kepada siswa.

Gunakan *Python* dan pustaka *NLTK* (open source) untuk memahami kata kunci seperti “malas,” “bosan,” atau “semangat.”

Langkah-langkah:

1. Buat daftar 20 pertanyaan dan respon chatbot.
2. Gunakan *Python dictionary* atau *JSON* untuk menyimpan percakapan.
3. Buat tampilan chatbot berbasis teks sederhana di terminal atau web.
4. Presentasikan chatbot secara langsung (tanya: “Saya malas belajar hari ini”, lalu bot menjawab “Ayo semangat! Belajar sebentar aja dulu!”).
5. Setelah presentasi, tim berdiskusi tentang *feedback* dari guru untuk perbaikan chatbot.

Tujuan: Melatih komunikasi, pembagian peran, dan pemanfaatan *AI Natural Language Processing* sederhana.

Proyek 5 – Sistem Rekomendasi Kegiatan Sekolah

Kalian membuat sistem rekomendasi kegiatan berdasarkan minat siswa, misalnya olahraga, seni, atau teknologi. Gunakan *Python* dan *Pandas* untuk memproses data.

Langkah-langkah:

1. Kumpulkan data minat dari 20 siswa (misalnya: musik, coding, olahraga, desain).
2. Simpan data dalam file CSV lalu baca dengan *Pandas*.
3. Buat sistem rekomendasi sederhana berbasis *similarity* (contoh: kalau suka coding → direkomendasikan “klub robotik”).
4. Tampilkan hasil rekomendasi dalam format tabel atau tampilan teks.
5. Presentasikan hasil akhir di depan kelas dengan membagi peran (data, coding, desain, presentasi, evaluasi).

Tujuan: Menumbuhkan kreativitas dalam membangun solusi sekolah menggunakan data dan logika *machine learning* sederhana.

Catatan

- Setiap proyek di atas bisa dikerjakan berkelompok 3–5 orang.
- Gunakan *open source tools* seperti *Python*, *Pandas*, *Keras*, *Matplotlib*, atau *Scikit-learn*.
- Buat *presentasi singkat* (5–7 slide) berisi **tujuan, teknologi, hasil, dan refleksi kerja tim**.
- Ingat! Nilai bukan hanya dari hasil proyek, tapi juga **cara kalian berkolaborasi dan mempresentasikannya**.

BAB 7: Refleksi Etika & Dampak Sosial AI

Tujuan Pembelajaran

Di akhir bab ini, kalian diharapkan bisa:

- Menyadari bahwa *Artificial Intelligence (AI)* bukan hanya soal teknologi, tapi juga soal **etika** dan **dampak sosial**.
- Memahami bagaimana AI bisa mempengaruhi kehidupan di sekolah, rumah, dan masyarakat.
- Belajar berpikir kritis sebelum menggunakan AI — apakah penggunaannya **baik**, **adil**, dan **bermanfaat** bagi orang lain.

Dengan kata lain, kalian belajar bukan hanya *cara membuat AI*, tapi juga *cara menjadi manusia yang bijak* dalam memakai AI.

Peta Konsep Sederhana

AI → Dampak → Etika → Refleksi

Contohnya:

Chatbot sekolah berbasis *Python* dan *neural network* bisa membantu menjawab tugas. Tapi kalau dipakai untuk mencontek, itu sudah keluar dari etika.

Apersepsi (Pemantik Pikiran)

Bayangkan suatu hari, di kelasmu tidak ada guru. Yang mengajar adalah AI yang bisa menjelaskan semua pelajaran, memberi nilai, bahkan menegur kamu kalau tidak belajar.

Lalu kamu bertanya:

“Kalau AI bisa ganti guru, itu baik atau buruk?”

Pertanyaan ini kelihatannya sederhana, tapi jawabannya dalam banget.

Mungkin AI bisa **membantu guru**, tapi **tidak bisa menggantikan hati manusia**.

Di sinilah pentingnya refleksi — supaya kita tidak hanya “cerdas digital”, tapi juga **cerdas moral**.

Penjelasan Konsep

Kalian pasti sudah sering dengar kata **AI (Artificial Intelligence)**.

AI bisa membantu menjawab soal, membuat ringkasan, bahkan membuat gambar.

Tapi pernahkah kalian berpikir:

“Kalau AI bisa ganti guru, itu baik atau buruk?”

Inilah yang disebut **refleksi etika dan dampak sosial**.

Artinya kita harus memikirkan:

- Bagaimana AI dipakai.
- Apa dampaknya buat sekolah, teman, guru, dan masyarakat.
- Apakah pemakaian AI itu adil, jujur, dan bermanfaat?

Apa itu Etika AI?

Etika = aturan moral, tentang benar atau salah.

Kalau dihubungkan dengan AI:

- AI harus digunakan dengan cara yang **jujur**.
- AI tidak boleh dipakai untuk **merugikan orang lain**.
- AI sebaiknya membantu manusia, bukan menggantikan sepenuhnya.

Dampak Sosial AI

AI punya **dampak positif** dan **dampak negatif**.

Dampak positif:

- Membantu siswa belajar lebih cepat.
- Memberi penjelasan yang mudah dipahami.
- Bisa jadi teman diskusi 24 jam.

Dampak negatif:

- Bisa dipakai untuk mencontek.
- Membuat siswa malas belajar mandiri.
- Data pribadi bisa bocor kalau tidak hati-hati.

Ilustrasi Kasus

Misalnya di SMK ada chatbot sekolah berbasis *open source* (Python, Pandas, Keras, *neural network*).

- Kalau dipakai untuk bertanya pelajaran, bagus sekali.
- Tapi kalau dipakai untuk mencontek soal ujian, itu jadi masalah.

Jadi, yang penting bukan hanya teknologinya, tapi juga **cara kita menggunakannya**.

Kenapa Harus Refleksi?

- Supaya kita sadar, teknologi bukan hanya tool, tapi juga punya konsekuensi.
- AI bisa jadi **teman belajar**, tapi jangan sampai jadi **jalan pintas curang**.
- Kita harus selalu berpikir jangka panjang: apakah AI ini membuat kita lebih pintar, atau malah lebih malas?

Mix Fun untuk Siswa SMK

- Etika = “aturan main” supaya AI tidak disalahgunakan.
- AI boleh membantu, tapi jangan sampai “otak sendiri” berhenti bekerja.
- Dampak sosial = apa yang dirasakan orang lain dari penggunaan AI.
- AI keren kalau dipakai untuk belajar, tapi berbahaya kalau dipakai untuk malas.

Contoh Kasus atau Ilustrasi

Kasus 1 — Chatbot Sekolah yang Pintar tapi Disalahgunakan

Di SMK “Techno Cerdas”, siswa membuat chatbot berbasis *Python* dan *neural network* untuk membantu teman-teman belajar. Chatbot ini bisa menjawab soal, menjelaskan konsep, bahkan membantu membuat ringkasan pelajaran. Awalnya semua berjalan baik.

Namun, beberapa siswa mulai memakai chatbot untuk mencontek saat ujian online. AI yang seharusnya membantu belajar malah jadi tool curang.

Refleksi:

- Apakah salah teknologinya? Tidak.
- Yang salah adalah **cara memakainya**.
Etika penting agar AI digunakan dengan jujur dan bertanggung jawab.

Kasus 2 — Guru Membuat Sistem Penilaian Otomatis

Seorang guru di SMK mencoba membuat sistem penilaian otomatis dengan *Pandas* dan *machine learning*. Sistem itu bisa membaca nilai siswa dari file Excel, lalu menilai performa mereka secara cepat. Namun, ada masalah: algoritma yang digunakan kurang akurat. Beberapa siswa yang sebenarnya rajin justru dinilai “rendah”.

Refleksi:

- AI bisa mempercepat pekerjaan guru, tapi tidak boleh menggantikan penilaian manusia sepenuhnya.
- Guru tetap perlu memeriksa hasil AI agar adil dan tidak merugikan siswa.

Kasus 3 — AI Pengenal Wajah di Sekolah

Sekolah mencoba memakai sistem AI pengenal wajah (*facial recognition*) untuk absensi. Programnya dibuat dengan *open source library* seperti *OpenCV* dan *Keras*. Cepat dan keren! Tapi ada masalah kecil: Siswa yang memakai masker atau jilbab kadang tidak dikenali, dan dianggap “tidak hadir”.

Refleksi:

- AI bisa efisien, tapi belum sempurna.
- Penting untuk selalu mengevaluasi agar tidak ada siswa yang dirugikan.
- Etika dalam AI berarti memastikan semua orang diperlakukan adil.

Kasus 4 — AI untuk Memprediksi Nilai Ujian

Tim *data science* siswa SMK membuat model *machine learning* untuk memprediksi nilai ujian dari kebiasaan belajar. Modelnya dibuat dengan *Python* dan *scikit-learn*. Tapi ternyata, data yang dipakai tidak lengkap — hanya siswa kelas unggulan yang dianalisis. Akibatnya, hasil prediksi bias dan tidak mewakili semua siswa.

Refleksi:

- Data yang tidak lengkap bisa membuat AI “tidak adil”.
- Dalam etika AI, *keadilan* dan *transparansi data* adalah hal penting.

Kasus 5 — AI di Dunia Kerja: Robot Menggantikan Pekerja

Salah satu alumni SMK bekerja di pabrik yang mulai memakai AI untuk otomatisasi mesin. Produksi jadi lebih cepat, tapi sebagian pekerja manusia kehilangan pekerjaan.

Refleksi:

- AI membawa efisiensi, tapi juga tantangan sosial.
- Kita perlu berpikir bagaimana AI bisa membantu manusia, bukan menggantikannya.
- Solusinya? Belajar keterampilan baru agar bisa bekerja *bersama* AI, bukan kalah oleh AI.

Catatan

Jadi, teman-teman SMK, teknologi *open source* seperti *Python*, *Pandas*, dan *Keras* memang hebat. Tapi sehebat apa pun AI, manusia tetap harus jadi pengendali utamanya. Ingat — *AI tanpa etika ibarat mesin tanpa rem*.

Contoh Soal dan Pembahasan

Soal 1 — Chatbot untuk Belajar vs Chatbot untuk Mencontek

Soal:

Rian membuat chatbot sekolah berbasis *Python* dan *Keras*. Chatbot itu bisa menjawab soal ujian dengan cepat. Temannya menggunakan chatbot itu saat ujian tanpa belajar. Apakah tindakan itu etis? Jelaskan alasannya.

Jawaban:

Tidak etis.

Karena chatbot dibuat untuk membantu belajar, bukan untuk mencontek.

AI seharusnya mendukung proses belajar, bukan menggantikan usaha manusia.

Pembahasan:

Rian menggunakan *open source* AI untuk kebaikan, tapi temannya menyalahgunakan. Etika AI mengajarkan kita untuk bertanggung jawab atas cara kita memakai teknologi. AI yang jujur membuat kita tumbuh, AI yang curang bikin kita tergantung.

Soal 2 — AI Pengenal Wajah Tidak Adil

Soal:

Sekolah memakai sistem absensi wajah berbasis *OpenCV* dan *Machine Learning*. Namun, beberapa siswa tidak terdeteksi karena memakai masker atau kacamata. Apa masalah etika dalam kasus ini?

Jawaban:

Masalahnya adalah ketidakadilan dalam sistem AI.

AI tidak boleh bias atau merugikan sebagian orang.

Pembahasan:

AI seharusnya memudahkan semua siswa, bukan hanya sebagian. Pengembang harus memperbaiki model datanya agar mengenali berbagai kondisi wajah. Dalam etika AI, *keadilan* dan *inklusi* sangat penting.

Soal 3 — AI Penilai Tugas Otomatis

Soal:

Guru membuat sistem penilai otomatis menggunakan *Pandas* untuk membaca nilai dari file Excel. Tapi hasilnya salah karena data tidak lengkap. Apakah AI bisa dipercaya sepenuhnya?

Jawaban:

Tidak bisa sepenuhnya.

AI hanyalah tool bantu, manusia tetap harus memeriksa hasilnya.

Pembahasan:

AI bisa membantu pekerjaan guru, tapi keputusan akhir tetap ada di tangan manusia.

Etika AI menekankan bahwa tanggung jawab tidak bisa diserahkan penuh ke mesin. Kita perlu memastikan hasil AI tetap adil dan akurat.

Soal 4 — AI di Dunia Kerja

Soal:

Sebuah pabrik memakai sistem *Machine Learning* untuk mengontrol mesin otomatis. Akibatnya, banyak pekerja manusia kehilangan pekerjaan. Apakah penggunaan AI ini etis?

Jawaban:

Etis jika disertai pelatihan ulang bagi pekerja.
Tidak etis jika langsung menggantikan tanpa solusi.

Pembahasan:

AI bisa meningkatkan efisiensi, tapi juga berdampak sosial. Etika AI menuntut adanya keseimbangan antara kemajuan teknologi dan kesejahteraan manusia. Manusia tetap harus jadi pusat keputusan, bukan korban perubahan.

Soal 5 — Privasi Data Siswa

Soal:

Aplikasi pembelajaran online berbasis AI menyimpan data pribadi siswa tanpa izin. Apakah ini melanggar etika AI?

Jawaban:

Ya, melanggar etika.
Data pribadi tidak boleh digunakan tanpa izin pemiliknya.

Pembahasan:

Etika AI juga mencakup *privasi* dan *keamanan data*. Semua sistem AI, termasuk yang dibuat dengan *Python* atau *Neural Network*, harus menjaga informasi pengguna. AI yang aman membuat orang percaya dan merasa dilindungi.

Catatan

Dari semua soal di atas, ada satu pesan penting:

- AI bukan cuma tentang kecerdasan mesin, tapi juga **kecerdasan hati manusia**.
- Kamu boleh jago *coding*, *machine learning*, atau *neural network*, tapi kamu juga harus bijak dalam memutuskan bagaimana teknologi itu dipakai.
- AI akan selalu netral — manusialah yang menentukan arah baik atau buruknya.

Fakta Menarik / Fun Facts

- AI bisa menulis puisi, menggambar, bahkan bikin lagu — tapi belum bisa punya **rasa bersalah** seperti manusia.
- Lebih dari 60% sistem AI di dunia dibuat menggunakan *open source tools* seperti *Python* dan *TensorFlow*.
- *Machine learning* belajar dari data, tapi kalau datanya salah, hasilnya juga bisa salah — ini disebut **bias AI**.
- UNESCO sudah punya panduan global supaya AI tidak merusak manusia, tapi membantu manusia tumbuh.
- Di beberapa negara, siswa sudah belajar etika AI sejak sekolah menengah — bukan cuma cara *coding*-nya, tapi juga cara berpikirnya.
- AI bisa bantu petani, dokter, dan guru — tapi tetap butuh manusia untuk memastikan hasilnya benar dan adil.
- *Neural network* terinspirasi dari otak manusia, tapi tidak bisa meniru **empati** dan **moral** manusia.
- Semakin banyak perusahaan membuka *dataset open source* agar AI bisa dikembangkan secara transparan dan adil.
- Chatbot pintar di sekolah bisa menjawab tugas, tapi **tidak bisa menggantikan diskusi seru di kelas dengan guru dan teman**.
- AI tidak punya niat jahat — tapi pengguna tanpa etika bisa membuat AI jadi berbahaya.

Tips Belajar & Tips Cepat Menghafal

- **Gunakan analogi.** Bayangkan AI seperti motor: cepat, tapi tetap butuh rem (etika).
- **Belajar sambil praktik.** Buat chatbot sederhana dengan *Python*, lalu diskusikan dampaknya kalau disalahgunakan.
- **Diskusi bareng teman.** Etika lebih mudah dipahami lewat cerita dan debat kecil, bukan hafalan.
- **Pakai teknik “AI Reflect”.** Setelah belajar, tanya diri sendiri: “*Kalau saya jadi AI, saya akan berbuat apa?*”
- **Tulis kesimpulan pribadi.** Setiap selesai bab, tulis satu kalimat refleksi seperti “AI harus membantu, bukan menggantikan manusia.”
- **Kaitkan dengan dunia nyata.** Coba cari berita tentang AI, lalu pikirkan: etis atau tidak?
- **Gunakan media visual.** Buat diagram atau meme lucu tentang “AI baik vs AI nakal” biar gampang diingat.
- **Belajar santai tapi fokus.** Otak lebih mudah menyerap konsep etika kalau kamu tenang, tidak terburu-buru.
- **Ulangi dalam konteks berbeda.** Misalnya, etika AI di sekolah, di media sosial, dan di tempat kerja — biar pemahamanmu luas.
- **Belajar pakai AI itu sendiri.** Coba tanya chatbot seperti OpenWebUI atau Ollama: “Bagaimana etika AI yang baik?” — lalu analisis jawabannya.

Aktivitas Siswa

Aktivitas 1: Diskusi Kelas — “Apakah AI Bisa Mengganti Guru?”

Tujuan: Melatih kemampuan berpikir kritis dan etika dalam penggunaan AI.

Langkah-langkah:

1. Bentuk kelompok kecil (4–5 siswa).
2. Bagi dua peran:
 - Kelompok A = Mendukung AI bisa menggantikan guru.
 - Kelompok B = Menolak AI menggantikan guru.
3. Siapkan argumen 3–4 poin. Gunakan contoh seperti *chatbot sekolah berbasis Python atau neural network*.
4. Diskusikan di depan kelas.
5. Guru bertindak sebagai moderator dan memberi kesimpulan.

Hasil Akhir:

Siswa memahami bahwa AI adalah tool bantu, bukan pengganti manusia. Nilai utamanya bukan di teknologinya, tapi pada **etika dan tanggung jawab** dalam penggunaannya.

Aktivitas 2: Simulasi Chatbot Sekolah Beretika

Tujuan: Memahami bagaimana etika diterapkan dalam pembuatan sistem AI.

Langkah-langkah:

1. Buka laptop atau komputer lab dengan OS *Linux* atau *Ubuntu*.
2. Gunakan *Open Source WebUI* atau *Ollama* untuk menjalankan chatbot lokal.
3. Buat percakapan dengan chatbot, misalnya:
“Bagaimana cara belajar jujur menggunakan AI?”
4. Catat jawaban chatbot.
5. Diskusikan: apakah chatbot sudah memberikan jawaban yang **etis dan bertanggung jawab**?

Hasil Akhir:

Siswa dapat menilai dan memahami bahwa AI juga harus mengikuti prinsip moral manusia.

Aktivitas 3: Analisis Kasus Nyata — “AI yang Salah Prediksi”

Tujuan: Menemukan dampak sosial jika AI tidak diuji dengan benar.

Langkah-langkah:

1. Guru menampilkan contoh sederhana model *machine learning* yang salah memprediksi data (misalnya data nilai siswa).
2. Siswa menganalisis kenapa hasilnya bisa salah.
3. Gunakan logika: apakah datanya tidak lengkap? apakah modelnya tidak adil?
4. Diskusikan solusi untuk memperbaikinya.

Hasil Akhir:

Siswa memahami pentingnya *dataset yang bersih* dan *etika dalam pengujian AI*.

Kesimpulan: AI bukan selalu benar, manusia tetap pengawas utamanya.

Aktivitas 4: Poster Digital “Etika AI Versi Sekolahku”

Tujuan: Mengajak siswa berkreasi dan berpikir reflektif tentang dampak sosial AI di sekolah.

Langkah-langkah:

1. Buat kelompok berisi 3–4 orang.
2. Pilih tema seperti “AI untuk Kebaikan”, “AI Bukan Pengganti Manusia”, atau “Etika dalam Dunia Digital”.
3. Gunakan *open source tools* seperti GIMP, Inkscape, atau Canva versi gratis.
4. Buat poster dengan teks singkat dan ilustrasi lucu.
5. Tampilkan di kelas atau papan sekolah.

Hasil Akhir:

Siswa dapat menyampaikan pesan moral tentang etika AI dengan cara kreatif dan visual.

Aktivitas 5: Refleksi Pribadi — “AI dalam Hidupku”

Tujuan: Mengajak siswa menulis dan merenung tentang penggunaan AI dalam kehidupan sehari-hari.

Langkah-langkah:

1. Minta setiap siswa menulis 1 halaman refleksi pribadi dengan judul “AI dalam Hidupku”.
2. Isi tulisan bisa berupa pengalaman pribadi, pendapat, atau cita-cita menggunakan AI di masa depan.
3. Gunakan contoh aplikasi *open source* seperti *Python AI assistant*, *OpenWebUI*, atau *Keras project*.
4. Tuliskan satu kalimat penutup reflektif, misalnya:
“AI akan bermanfaat jika aku menggunakannya dengan hati yang jujur.”

Hasil Akhir:

Siswa dapat menilai diri sendiri, memahami etika digital, dan punya pandangan matang tentang masa depan AI.

Catatan

AI bukan hanya tentang *coding* dan *neural network*, tapi juga tentang **tanggung jawab moral manusia**. Melalui lima aktivitas di atas, kalian belajar bahwa menjadi ahli AI berarti juga menjadi manusia yang bijak, jujur, dan berpikir panjang. Teknologi open source seperti *Python*, *Pandas*, dan *Keras* bisa kalian kuasai kapan pun — tapi etika dan tanggung jawab, itulah yang membuat kalian **berbeda dari mesin**.

Rangkuman

Artificial Intelligence (AI) adalah teknologi yang bisa berpikir dan belajar seperti manusia. AI membantu kita dalam banyak hal: dari menjawab soal, membuat ringkasan, hingga menganalisis data lewat *Python* dan *Pandas*. Namun, **AI tidak punya hati dan moral**, karena itu manusialah yang harus mengatur etika penggunaannya. AI akan menjadi bermanfaat jika digunakan dengan **jujur, adil, dan bertanggung jawab**, tapi bisa berbahaya kalau dipakai untuk hal yang salah seperti mencontek atau menyebar data pribadi.

Dalam dunia pendidikan, AI punya **dampak sosial besar**. Di satu sisi, AI bisa membuat belajar jadi lebih cepat dan seru—misalnya lewat chatbot sekolah berbasis *Keras* atau *Neural Network*. Namun di sisi lain, AI juga bisa membuat siswa **terlalu bergantung pada mesin** dan kehilangan semangat belajar mandiri. Itulah kenapa kita perlu *refleksi etika* — berpikir sebelum bertindak, supaya teknologi yang kita pakai tetap membawa manfaat, bukan masalah.

Sebagai siswa SMK yang akan menjadi generasi digital, kamu perlu paham bahwa **AI hanyalah tool, bukan pengganti manusia**. Kita yang menentukan arah penggunaannya: mau dipakai untuk kebaikan atau disalahgunakan. Gunakan teknologi *open source* seperti *Python*, *Machine Learning*, dan *Keras* bukan hanya untuk membangun sistem pintar, tapi juga untuk membangun **diri yang beretika dan bijak digital**. Ingat, masa depan AI ada di tangan kalian — bukan di server, tapi di **cara berpikir dan bertanggung jawab sebagai manusia**.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. Etika AI berarti aturan moral dalam menggunakan kecerdasan buatan.

Jawaban: Benar

2. Semua sistem AI selalu memberikan hasil yang adil dan tidak pernah salah.

Jawaban: Salah

3. AI bisa digunakan untuk membantu siswa memahami pelajaran dengan lebih cepat.

Jawaban: Benar

4. Chatbot yang digunakan untuk mencontek adalah contoh penggunaan AI yang etis.

Jawaban: Salah

5. AI yang baik adalah AI yang membantu manusia tanpa menggantikan peran manusia.

Jawaban: Benar

B. Pilihan Ganda (10 Soal)

1. Etika AI membahas tentang...

- a. Cara membuat robot
- b. Aturan benar dan salah dalam penggunaan AI
- c. Cara mempercepat komputer
- d. Sistem operasi open source

Jawaban: b

2. AI memiliki dampak sosial karena...

- a. Bisa hidup seperti manusia
- b. Memengaruhi cara orang belajar, bekerja, dan berinteraksi
- c. Tidak bisa digunakan oleh semua orang
- d. Hanya digunakan di luar negeri

Jawaban: b

3. Salah satu contoh penggunaan AI yang **tidak etis** adalah...

- a. Membuat chatbot belajar sekolah
- b. Membantu menjawab soal tanpa belajar
- c. Menyimpan data siswa dengan aman
- d. Membuat sistem absensi otomatis

Jawaban: b

4. Jika data pribadi siswa bocor karena AI, itu termasuk masalah...

- a. Keamanan data
- b. Koneksi internet
- c. Jaringan sekolah
- d. Algoritma visual

Jawaban: a

5. AI dapat membantu guru dalam...

- a. Menggantikan semua pekerjaan guru
- b. Membantu menganalisis nilai siswa dengan cepat
- c. Menghapus data tugas
- d. Menonaktifkan komputer

Jawaban: b

6. Teknologi *open source* yang biasa digunakan dalam pengembangan AI adalah...
- Photoshop
 - Python
 - Excel
 - PowerPoint

Jawaban: b

7. Jika AI digunakan tanpa memperhatikan etika, maka...
- AI akan lebih cerdas
 - AI bisa menimbulkan dampak negatif bagi manusia
 - AI akan berhenti bekerja
 - AI bisa menggantikan semua profesi

Jawaban: b

8. Etika dalam AI perlu diterapkan supaya...
- Manusia bisa bersaing dengan mesin
 - AI digunakan dengan aman dan bertanggung jawab
 - Semua orang bisa membuat AI sendiri
 - AI bisa diinstal di semua komputer

Jawaban: b

9. Berikut ini adalah contoh dampak positif AI di sekolah...
- Membuat siswa malas belajar
 - Membantu siswa belajar mandiri dengan cepat
 - Menyebarkan data pribadi ke internet
 - Menghapus pekerjaan guru

Jawaban: b

10. Organisasi dunia yang membuat panduan etika AI adalah...
- WHO
 - NASA
 - UNESCO
 - FAO

Jawaban: c

C. Isian Singkat (5 Soal)

1. Etika AI berarti aturan tentang _____ dan _____ dalam penggunaan kecerdasan buatan.

Jawaban: benar, salah

2. Dampak negatif AI di sekolah salah satunya adalah siswa menjadi _____ belajar mandiri.

Jawaban: malas

3. AI seharusnya digunakan untuk _____ manusia, bukan menggantikannya.
Jawaban: membantu
4. Teknologi *open source* seperti _____ dan _____ sering digunakan dalam pengembangan AI.
Jawaban: Python, Keras
5. Salah satu cara menjaga etika AI adalah memastikan data pribadi pengguna tetap _____.
Jawaban: aman

D. Soal Eksplorasi (3 Soal)

Soal 1: Kasus Chatbot di Sekolah

Di SMK kamu, ada chatbot berbasis Python dan Neural Network yang bisa menjawab soal pelajaran. Beberapa siswa menggunakannya untuk belajar, tapi ada juga yang memakainya saat ujian. Menurutmu, bagaimana cara agar chatbot ini tetap digunakan secara etis di sekolah?

Contoh Jawaban:

Chatbot bisa dibatasi penggunaannya hanya untuk jam belajar, bukan saat ujian. Sekolah bisa membuat aturan dan mengingatkan siswa tentang tanggung jawab dalam memakai AI. Etika AI harus dipelajari agar siswa tahu perbedaan antara bantuan belajar dan kecurangan.

Soal 2: Dampak Sosial AI di Dunia Kerja

Bayangkan kamu bekerja di pabrik yang mulai menggunakan sistem AI untuk mengontrol mesin otomatis. Sebagian temanmu kehilangan pekerjaan. Apa solusi etis yang bisa dilakukan perusahaan?

Contoh Jawaban:

Perusahaan sebaiknya memberikan pelatihan baru bagi pekerja agar bisa menggunakan atau memperbaiki sistem AI. Dengan begitu, AI membantu meningkatkan kualitas kerja manusia, bukan menggantikannya.

Soal 3: AI dan Masa Depan Pendidikan

AI kini digunakan untuk membuat materi belajar otomatis dan menilai tugas. Bagaimana menurutmu AI bisa membantu guru tanpa membuat siswa kehilangan semangat belajar?

Contoh Jawaban:

AI sebaiknya digunakan untuk membantu guru menjelaskan materi dengan visual menarik, bukan menggantikan interaksi di kelas. Siswa tetap perlu berpikir kritis dan berdiskusi, sementara AI menjadi tool bantu belajar yang seru dan efisien.

Catatan

Soal-soal ini bukan sekadar tentang teknologi, tapi tentang **bagaimana menjadi manusia bijak di era AI**. Gunakan AI seperti *Python* dan *Keras* untuk belajar, bukan untuk menghindari dari belajar. Karena teknologi boleh canggih, tapi **etika tetap nomor satu**.

Tugas Proyek

Proyek 1 — “AI dan Privasi Data Sekolah”

Tantangan:

Telusuri bagaimana data siswa (nama, nilai, kehadiran) disimpan dan dilindungi di sistem digital sekolah. Apakah sudah aman dari kebocoran?

Langkah-langkah:

1. Mewawancarai petugas TU atau guru IT tentang sistem penyimpanan data.
2. Catat bagaimana data diakses dan siapa yang bisa melihatnya.
3. Buat bagan sederhana tentang alur data sekolah.
4. Refleksikan: apa yang bisa diperbaiki agar lebih aman dan etis?

Contoh Jawaban Siswa:

“Data siswa disimpan di Google Sheets, tapi belum ada *password protection*.

Kami menyarankan pakai sistem lokal berbasis *Python Flask* agar lebih aman.

Ini contoh penerapan *etika digital* di dunia nyata.”

Proyek 2 — “AI di Media Sosial: Teman atau Musuh?”

Tantangan:

Cari tahu bagaimana algoritma *AI* di media sosial mempengaruhi cara berpikir dan bersikap remaja.

Langkah-langkah:

1. Pilih satu platform (TikTok, YouTube, Instagram).
2. Amati 10 rekomendasi konten yang muncul di *feed* kamu.
3. Catat: apakah konten itu mendidik, netral, atau malah bikin kecanduan?
4. Buat laporan reflektif 1 halaman dengan saran bagaimana seharusnya *AI rekomendasi* itu bekerja lebih etis.

Contoh Jawaban Siswa:

“Dari 10 video TikTok, 7 tidak berhubungan dengan pelajaran. *AI* seharusnya bisa membantu siswa mencari hal bermanfaat, bukan sekadar hiburan. Saya belajar bahwa *AI recommendation* harus dilatih dengan nilai edukatif.”

Proyek 3 — “AI di Dunia Kerja”

Tantangan:

Riset pekerjaan yang mulai digantikan atau dibantu oleh *AI* dan *machine learning*.

Langkah-langkah:

1. Pilih satu bidang kerja (contoh: desain, otomotif, administrasi, pertanian).
2. Gunakan internet atau sumber sekolah untuk mencari contoh penggunaan AI di bidang itu.
3. Tuliskan 3 dampak positif dan 3 dampak negatifnya.
4. Buat kesimpulan: bagaimana agar AI bisa digunakan secara *etis* di bidang tersebut.

Contoh Jawaban Siswa:

“Di industri otomotif, AI membantu perakitan mobil otomatis. Positifnya: efisiensi tinggi. Negatifnya: beberapa pekerja digantikan robot. Etisnya, perusahaan harus memberi pelatihan baru bagi pekerja agar tetap relevan.”

Proyek 4 — “Desain Poster Kampanye Etika AI”

Tantangan:

Buat poster digital yang mengajak siswa lain untuk menggunakan AI secara etis dan bertanggung jawab.

Langkah-langkah:

1. Pilih pesan utama seperti “AI Membantu, Bukan Menggantikan” atau “Gunakan AI untuk Belajar, Bukan Mencontek.”
2. Gunakan *open source tool* seperti **GIMP**, **Inkscape**, atau **Canva free**.
3. Tambahkan ikon teknologi *Python*, *Machine Learning*, atau *Neural Network*.
4. Sertakan 1 kutipan reflektif (misal: “AI butuh hati manusia untuk tetap adil”).

Contoh Jawaban Siswa:

Poster berjudul “AI dengan Etika” menampilkan robot membantu guru mengajar. Tulisan besar: “*AI hebat, tapi tanpa etika jadi berbahaya.*”
Poster dibuat di Inkscape dan dicetak untuk dinding lab komputer.

Proyek 5 — “Jurnal Harian AI”

Tantangan:

Selama 3 hari, catat semua interaksi kamu dengan AI.

Contohnya: chatbot, rekomendasi YouTube, aplikasi edit foto otomatis, atau *AI grammar checker*.

Langkah-langkah:

1. Buat tabel berisi: tanggal, nama aplikasi, manfaat, dan risiko etika yang ditemukan.
2. Setelah 3 hari, tulis refleksi 1 paragraf tentang bagaimana AI mempengaruhi rutinitasmu.
3. Bandingkan dengan teman sekelas. Apakah kamu sadar betapa sering AI hadir di hidupmu?

Contoh Jawaban Siswa:

“Saya pakai AI di YouTube, Google Lens, dan ChatGPT lokal di OpenWebUI. Kadang membantu, tapi juga bikin saya lupa waktu. Saya belajar bahwa AI bisa produktif kalau dipakai dengan kontrol diri.”

Catatan

Lewat 5 proyek ini, kamu belajar bahwa AI bukan hanya tentang *neural network* dan *machine learning*, tapi juga tentang **sikap, etika, dan tanggung jawab sosial**. Semua tool open source seperti *Python*, *Pandas*, dan *Keras* hanya sekuat pengguna di baliknya. Jadilah siswa SMK yang bukan cuma paham teknologi, tapi juga **bijak digital dan beretika AI**.

BAB 8: Proyek Akhir – AI Sekolah Cerdas

Tujuan Pembelajaran

Di bab ini, kamu akan menyatukan semua yang sudah kamu pelajari dari bab-bab sebelumnya. Kamu akan membuat **sistem AI sekolah cerdas**, yaitu gabungan antara chatbot, pustakawan digital, dan asisten belajar berbasis *open source*.

Tujuannya sederhana tapi keren banget:

- **Menggabungkan semua komponen AI:** dari *input data* sampai *evaluasi sistem*.
- **Melatih kerja tim** seperti profesional IT — ada yang pegang data, ada yang pegang model, ada yang urus server.
- **Membangun AI yang bermanfaat nyata**, bukan hanya teori.
- **Belajar tanggung jawab etis**, supaya AI digunakan untuk kebaikan dan pembelajaran.

Setelah selesai, kamu akan punya pengalaman *end-to-end project* — dari *Python coding* sampai *deployment lokal di server sekolah*. Inilah langkah pertama kamu menuju dunia *AI engineering* yang sesungguhnya!

Peta Konsep

Input → Model → RAG → Deployment → Evaluasi

Peta konsep ini seperti “peta perjalanan” tim kamu: mulai dari *data mentah* sampai *AI siap pakai di sekolah*. Semuanya berbasis *open source*, jadi kamu bebas memodifikasi dan berinovasi!

Apersepsi

Bayangkan kalau di sekolahmu ada **asisten AI cerdas** yang bisa diajak bicara seperti teman belajar. Kamu bisa tanya:

“Bagaimana cara kerja sel surya?”

dan AI akan menjawab dengan penjelasan singkat, lalu menambahkan:

“Lihat juga di buku Fisika halaman 77 untuk eksperimen tambahan.”

Seru, kan?

AI ini bisa jadi **guru cadangan**, **pustakawan digital**, dan **teman diskusi** dalam satu sistem.

Proyek ini bukan sekadar latihan *coding* — tapi latihan berpikir seperti **arsitek teknologi**. Kamu akan menciptakan sistem yang benar-benar hidup, membantu, dan bisa digunakan banyak orang. Dengan *Python*, *Machine Learning*, dan *Neural Network*, kamu bukan cuma pengguna teknologi — kamu adalah **pembangunnya**.

Penjelasan Konsep

Bayangkan kalau sekolah kalian punya **asisten AI cerdas**. Asisten ini bukan hanya chatbot, tapi juga **pustakawan pintar** dan **guru pendamping**. Dia bisa menjawab pertanyaan dari kurikulum, merekomendasikan buku, bahkan menemani belajar dengan cara yang interaktif.

Proyek akhir ini adalah **tahap puncak** dari semua bab sebelumnya. Kalian akan **menggabungkan semua bagian**: mulai dari input data, model AI, RAG, deployment, sampai evaluasi.

Integrasi Semua Bab

Mari kita lihat bagaimana semuanya menyatu:

1. **Input Data**
 - Masukkan dokumen sekolah: kurikulum, buku pelajaran, catatan guru.
 - Data ini jadi bahan bakar AI.
2. **Model AI**
 - Dibangun dengan *open source* seperti **Python, Pandas, Keras, Neural Network**.
 - Model ini melatih AI agar bisa mengerti pertanyaan siswa.
3. **RAG (Retrieval-Augmented Generation)**
 - Bagian ini jadi “mesin pencari pintar” di dalam AI.
 - Ia mencari jawaban di dokumen, lalu menggabungkannya jadi penjelasan yang mudah dipahami.
4. **Deployment Lokal**
 - AI dijalankan di server sekolah pakai **Ubuntu + Ollama + OpenWebUI**.
 - Semua komputer di lab bisa mengakses lewat LAN.
5. **Evaluasi**
 - Sistem dites: apakah jawabannya benar?
 - Kalau ada kekurangan, tim bisa memperbaiki.

Ilustrasi Kasus

Bayangkan sebuah **chatbot sekolah cerdas**.

- Siswa bertanya: “*Apa rumus energi potensial?*”
- AI mencari di buku Fisika, lalu menjawab: “Energi potensial = $m \times g \times h$.”
- AI juga menambahkan rekomendasi: “*Untuk belajar lebih lanjut, baca Bab 4 di buku Fisika Kelas 10.*”
- Bahkan AI bisa memberi tips belajar: “*Coba latihan soal di halaman 55.*”

Dengan begitu, AI tidak hanya menjawab, tapi juga **membimbing siswa belajar**.

Kenapa Proyek Ini Penting?

- Sekolah bisa punya **pembantu digital** untuk belajar.
- Guru terbantu, siswa terbimbing.
- Teknologi tetap **mandiri** karena semua pakai *open source*.
- Siswa SMK belajar pengalaman nyata membangun sistem AI end-to-end, dari nol sampai siap pakai.

Mix Fun untuk Siswa SMK

- AI Sekolah Cerdas = teman belajar 24 jam.
- Kombinasi: chatbot + pustakawan + tutor digital.
- Tim kalian = arsitek yang menyusun seluruh sistem.
- Rasanya seperti bikin mini Google khusus sekolah sendiri!

Contoh Kasus atau Ilustrasi

Kasus 1 — Chatbot Kurikulum Pintar

Deskripsi:

Sekolah punya banyak file PDF kurikulum, buku pelajaran, dan catatan guru. Kalian membuat chatbot yang bisa menjawab pertanyaan berdasarkan dokumen-dokumen tersebut.

Langkah-langkah:

1. Siapkan dokumen: “Kurikulum Informatika SMK”, “Modul Python Dasar”, dan “Catatan Guru Fisika”.
2. Gunakan *RAG (Retrieval-Augmented Generation)* agar chatbot bisa mencari isi file.
3. Jalankan di server lokal dengan *Ollama + OpenWebUI*.

Pseudocode (simulasi alur):

```
upload("kurikulum_smk.pdf")
ask_ai("Apa tujuan pembelajaran Python di SMK?")
AI → mencari isi dokumen
AI → menjawab dari teks asli + memberikan ringkasan
```

Hasil:

Chatbot menjawab:

“Tujuan pembelajaran Python di SMK adalah memahami logika pemrograman dan kemampuan membuat proyek sederhana.”

AI juga bisa menambahkan saran:

“Coba baca bagian 3.2 tentang *looping* di modul Python Dasar.”

Kasus 2 — Pustakawan Digital Sekolah

Deskripsi:

AI berfungsi seperti *pustakawan pintar* yang bisa memberi rekomendasi buku atau materi sesuai topik belajar.

Langkah-langkah:

1. Kumpulkan data katalog buku sekolah dalam format CSV (*open source friendly*).
2. Buat *Python script* sederhana dengan *Pandas* untuk membaca data.
3. Integrasikan dengan AI agar siswa bisa bertanya, “Buku apa yang cocok untuk belajar algoritma?”

Contoh Output:

AI menjawab:

“Kamu bisa membaca buku ‘*Dasar Algoritma dan Struktur Data*’ oleh Bambang.”

“Untuk latihan, buka bab 5 tentang *sorting algorithm*.”

Manfaat:

Siswa tidak perlu mencari manual — AI membantu menemukan sumber belajar tercepat dan paling relevan.

Kasus 3 — Asisten Evaluasi Belajar

Deskripsi:

AI membantu guru mengevaluasi hasil belajar dengan cepat.

Guru bisa mengunggah hasil ujian atau catatan nilai siswa, lalu AI menganalisis datanya.

Langkah-langkah:

1. Kumpulkan file Excel nilai siswa.
2. Gunakan *Pandas* untuk membaca dan menghitung rata-rata.
3. Jalankan hasil analisis lewat chatbot *OpenWebUI*.

Contoh Kode Singkat (Python):

```
import pandas as pd
data = pd.read_excel("nilai_siswa.xlsx")
print(data.mean())
```

Hasil:

AI memberikan laporan ringkas:

“Nilai rata-rata kelas XI TKJ adalah 82. Nilai tertinggi di bidang Python, terendah di Jaringan.”

Refleksi Etis:

AI membantu guru, tapi keputusan akhir tetap di tangan manusia.
Jangan biarkan AI menilai tanpa pengawasan guru!

Kasus 4 — Asisten Belajar Pribadi (Personal Tutor)

Deskripsi:

Setiap siswa bisa punya asisten belajar digital pribadi yang berjalan di *laptop sekolah* tanpa internet. AI ini membantu memahami pelajaran, membuat latihan soal, dan memberi penjelasan tambahan.

Langkah-langkah:

1. Jalankan *Ollama* model seperti *gemma :2b* di Ubuntu.
2. Buka *OpenWebUI* lokal di browser.
3. Tambahkan folder “Materi Belajar SMK” ke sistem *RAG*.
4. Mulai percakapan:
“Jelaskan perbedaan antara *machine learning* dan *deep learning*.”

AI akan menjawab:

“*Machine learning* belajar dari data menggunakan algoritma, sedangkan *deep learning* menggunakan jaringan saraf tiruan (*neural network*) yang lebih kompleks.”

Manfaat:

Belajar jadi lebih seru karena siswa bisa bertanya kapan saja tanpa takut salah!

Kasus 5 — AI Sekolah Cerdas di Server Lokal (Deployment)

Deskripsi:

Tim SMK membuat sistem AI sekolah yang bisa digunakan oleh semua komputer di lab melalui jaringan lokal (*LAN*).

Langkah-langkah Instalasi Singkat:

1. Instal *Docker* di Ubuntu Server.
2. Jalankan *Ollama* dan *OpenWebUI* dengan *docker-compose.yml*.
3. Tambahkan *volume* untuk menyimpan dokumen sekolah (buku, kurikulum, catatan).
4. Atur port agar semua PC lab bisa mengakses via browser
<http://192.168.0.10:3000>.
5. Tes chatbot dengan pertanyaan sederhana.

Pseudocode Deployment:

```
docker compose up -d
open("http://192.168.0.10:3000")
ask_ai("Tunjukkan jadwal pelajaran kelas 11")
AI mencari file jadwal → menampilkan hasil
```

Hasil:

AI menampilkan jadwal pelajaran lengkap, lalu menambahkan saran belajar:

“Gunakan waktu kosong Rabu sore untuk latihan Python proyek AI.”

Manfaat:

Sekolah jadi punya sistem AI *mandiri*, aman, dan tanpa biaya lisensi karena semuanya *open source*!

Catatan

Dari kelima kasus di atas, siswa belajar membangun **AI Sekolah Cerdas** yang nyata dan bermanfaat. Mulai dari chatbot, pustakawan digital, hingga sistem AI yang bisa dijalankan di server sekolah — semuanya bisa dibuat dengan teknologi *open source*.

Kuncinya:

- Gunakan *Python* dan *Pandas* untuk data.
- Gunakan *Keras* dan *Neural Network* untuk model pembelajaran.
- Gunakan *Ollama* dan *OpenWebUI* untuk sistem AI lokal sekolah.

AI ini bukan hanya proyek teknologi, tapi **simbol kreativitas dan kemandirian siswa SMK di era digital!**

Contoh Soal dan Pembahasan

Soal 1 – Integrasi Komponen AI Sekolah

Soal:

Sebutkan lima komponen utama dalam membangun sistem *AI Sekolah Cerdas* dan jelaskan fungsi singkatnya.

Jawaban:

1. **Input Data** – tempat menyimpan dokumen sekolah seperti buku dan kurikulum.
2. **Model AI** – otak sistem, dibangun dengan *Python*, *Pandas*, dan *Keras*.
3. **RAG (Retrieval-Augmented Generation)** – bagian pencarian pintar yang membaca dokumen dan merangkumnya.
4. **Deployment Lokal** – menjalankan AI di server sekolah menggunakan *Ollama* dan *OpenWebUI*.
5. **Evaluasi** – mengetes apakah jawaban AI sudah akurat dan mudah dipahami.

Pembahasan:

Kelima komponen ini saling terhubung seperti rantai.

Tanpa data, model tidak punya bahan belajar.

Tanpa *deployment*, AI tidak bisa digunakan di lab sekolah.

Dan tanpa evaluasi, kita tidak tahu apakah AI sudah “cerdas” atau belum.

Soal 2 – RAG di Dunia Nyata

Soal:

Jelaskan bagaimana cara kerja *RAG (Retrieval-Augmented Generation)* di dalam sistem chatbot sekolah.

Jawaban:

RAG mencari informasi dari dokumen sekolah (misalnya PDF buku atau catatan guru), lalu menggabungkannya dengan jawaban dari model AI.

Pembahasan:

Bayangkan RAG seperti *pustakawan pintar*.

Ketika siswa bertanya, “Apa rumus kecepatan?”, RAG tidak menebak.

Ia membuka buku Fisika di folder dokumen, menemukan halaman yang tepat, lalu membantu model AI menjawab:

“Kecepatan = jarak ÷ waktu.”

Jadi, hasilnya bukan omongan kosong, tapi berdasarkan data sekolah.

Pseudocode Sederhana:

```
upload("buku_fisika.pdf")
ask_ai("Apa rumus kecepatan?")
RAG → cari di dokumen
Model → buat kalimat penjelasan
```

Soal 3 – Deployment Lokal AI Sekolah

Soal:

Bagaimana cara menjalankan *AI Sekolah Cerdas* di server lokal sekolah agar bisa diakses oleh semua komputer di lab?

Jawaban:

1. Instal Ubuntu 24.04 di server sekolah.
2. Instal *Docker* dan *Ollama*.
3. Jalankan *OpenWebUI* dengan *docker-compose*.
4. Tambahkan folder “dokumen sekolah” sebagai sumber RAG.
5. Akses dari browser lab lewat <http://192.168.0.x:3000>.

Pembahasan:

Dengan *deployment lokal*, sekolah tidak butuh internet untuk menjalankan AI.

Semuanya aman, cepat, dan gratis karena berbasis *open source*. Itulah keunggulan sistem lokal SMK: mandiri dan efisien.

Contoh script singkat (`docker-compose.yml`):

```
services:
  ollama:
    image: ollama/ollama
    ports:
      - "11434:11434"
  openwebui:
    image: openwebui/openwebui
    ports:
      - "3000:8080"
    environment:
      - OLLAMA_API_BASE=http://localhost:11434
```

Soal 4 – Evaluasi Sistem AI Sekolah

Soal:

Setelah AI dijalankan, hasilnya kadang salah atau tidak nyambung. Bagaimana langkah evaluasi yang bisa dilakukan siswa SMK untuk memperbaikinya?

Jawaban:

1. Periksa kembali dataset (dokumen yang diunggah).
2. Cek apakah model AI sudah terlatih dengan baik (*fine-tuned*).
3. Lihat apakah RAG sudah membaca semua file dengan benar.
4. Uji beberapa pertanyaan dan catat kesalahan jawaban.
5. Lakukan *retraining* atau *index update* jika perlu.

Pembahasan:

Evaluasi itu seperti *tes harian untuk AI*.

Sama seperti siswa, AI juga bisa salah jawab.

Bedanya, kita bisa memperbaikinya dengan data tambahan atau penyesuaian logika RAG.

Tujuan akhirnya: AI bukan cuma pintar, tapi **tepat dan relevan**.

Soal 5 – Etika dan Dampak Sosial AI Sekolah

Soal:

Mengapa penting memperhatikan etika dan privasi dalam sistem AI sekolah lokal? Berikan contoh penerapannya.

Jawaban:

Etika penting agar AI tidak melanggar privasi siswa dan guru.

Contohnya, sistem AI tidak boleh menyimpan data nilai pribadi tanpa izin.

Pembahasan:

AI bukan hanya soal *coding*, tapi juga tanggung jawab.

Kalau data sekolah bocor, dampaknya besar.

Oleh karena itu, pastikan setiap sistem AI sekolah:

- Menggunakan server lokal (*on-premise*).
 - Menyimpan data di jaringan tertutup (*LAN*).
 - Tidak mengirim data ke internet.
- Dengan begitu, proyek AI SMK tetap **aman, etis, dan profesional**.

Catatan

Dari kelima contoh soal di atas, kamu belajar bukan hanya “bagaimana membuat AI”, tapi juga **bagaimana berpikir seperti tim pengembang profesional**.

Kamu tahu alur:

- dari *Input Data* → *Model* → *RAG* → *Deployment* → *Evaluasi*,
- sampai bagaimana menjaga *etika dan keamanan data*.

Ingat — semua ini bisa dilakukan dengan *open source tools* seperti *Python*, *Pandas*, *Keras*, *Ollama*, dan *OpenWebUI*.

Dan yang paling penting:

“AI Sekolah Cerdas bukan untuk menggantikan manusia, tapi untuk membantu manusia belajar lebih baik.”

Fakta Menarik / Fun Facts

- AI tidak punya hati, tapi bisa *meniru gaya bicara manusia* lewat *machine learning*.
- Kampus besar seperti MIT dan Stanford pakai sistem *AI asisten kampus* — tapi kamu bisa bikin versi SMK-nya sendiri dengan *Python* dan *OpenWebUI*!
- Model *neural network* terinspirasi dari **otak manusia**, tapi otak manusia tetap jauh lebih hebat.
- *Ollama* + *RAG* + *OpenWebUI* bisa dijalankan **tanpa internet**, cukup di server sekolah — jadi AI tetap jalan walau wifi putus!
- Setiap kali kamu menulis kode, AI-mu sebenarnya sedang “belajar” memahami cara berpikir manusia.
- Sistem *AI Sekolah Cerdas* buatan SMK bisa jadi bahan pameran *capstone project* untuk kuliah atau portofolio kerja.
- *Open source tools* seperti *Python* dan *Keras* membuat kamu bisa belajar teknologi mahal — **tanpa biaya lisensi sama sekali**.
- Satu server Ubuntu bisa menampung chatbot, pustakawan digital, dan asisten belajar dalam satu sistem AI!
- Di masa depan, setiap sekolah akan punya *AI lokal*, dan kamu bisa jadi salah satu penciptanya.

- AI tidak menggantikan guru, tapi **membuat guru punya waktu lebih banyak untuk mengajar hal penting.**

Tips Belajar & Tips Cepat Menghafal

- **Belajar sambil praktek.** Jangan cuma baca, langsung coba di *OpenWebUI* atau *Jupyter Notebook*.
- **Pahami alur besar.** AI = *Input* → *Model* → *RAG* → *Deployment* → *Evaluasi*. Hafalkan urutan ini seperti hafal nama teman sekelas.
- **Gunakan analogi.** Bayangkan AI seperti otak manusia yang sedang dilatih untuk menjawab pertanyaan sekolah.
- **Rapi itu penting.** Simpan semua file dataset dan model AI di folder teratur biar tidak bingung saat *debugging*.
- **Diskusi dengan tim.** Kadang ide terbaik muncul dari obrolan santai di lab komputer.
- **Gunakan AI untuk belajar AI.** Coba tanya ke *OpenWebUI*:
“Jelaskan langkah membuat RAG sederhana di Python.”
- **Jangan panik kalau error.** Setiap error itu tanda kamu sedang belajar sesuatu yang baru.
- **Catat perintah penting.** Simpan perintah instalasi Ollama, OpenWebUI, dan dataset di file *notes.txt*.
- **Uji sistem tiap selesai tahap.** Jangan tunggu sampai akhir baru dites — itu resep stres!
- **Fokus pada etika.** AI sekolah harus jujur, aman, dan tidak mengambil data tanpa izin.

Aktivitas Siswa

Aktivitas 1 — Siapkan Dataset Sekolah

Tujuan:

Mengumpulkan dan menyiapkan dokumen yang akan digunakan oleh *AI Sekolah Cerdas*.

Langkah-langkah:

1. Kumpulkan file: buku pelajaran, RPP, kurikulum, dan catatan guru.
2. Simpan semua di folder */data/kurikulum/*.
3. Pastikan format file adalah *.txt* atau *.pdf*.
4. Jalankan skrip *Python* untuk memastikan file bisa dibaca.

Pseudocode sederhana:

```
import os
files = os.listdir("/data/kurikulum/")
for f in files:
```



```
print("Ditemukan:", f)
```

Hasil:

AI punya bahan belajar berupa dokumen sekolah yang siap diproses oleh RAG.

Catatan:

Semakin lengkap datanya, semakin “cerdas” AI sekolahmu nanti!

Aktivitas 2 — Bangun Model AI Dasar

Tujuan:

Melatih model AI sederhana agar bisa memahami bahasa manusia.

Langkah-langkah:

1. Gunakan *Python + Keras* untuk membuat model sederhana berbasis *neural network*.
2. Gunakan dataset kalimat tanya dan jawaban sederhana dari pelajaran sekolah.
3. Simpan model dengan nama `model_ai_smk.h5`.

Contoh kode singkat:

```
from keras.models import Sequential
from keras.layers import Dense
model = Sequential([
    Dense(8, input_shape=(10,), activation='relu'),
    Dense(1, activation='sigmoid')
])
model.save("model_ai_smk.h5")
```

Hasil:

Kamu berhasil membuat *mini brain* untuk AI Sekolahmu!

Model ini nanti akan digunakan untuk membantu menjawab pertanyaan siswa.

Aktivitas 3 — Hubungkan dengan RAG (Retrieval-Augmented Generation)

Tujuan:

Menghubungkan model AI dengan *RAG system* agar AI bisa membaca dokumen dan menjawab berdasarkan sumber asli.

Langkah-langkah:

1. Pastikan dokumen sudah siap di folder `/data/kurikulum/`.
2. Jalankan *Ollama* di server Ubuntu 24.04.
3. Buka *OpenWebUI* dari browser (<http://localhost:3000>).
4. Tambahkan dokumen ke dalam *RAG database*.

Contoh perintah CLI:

```
ollama pull gemma2
ollama run gemma2
```

Lalu di *OpenWebUI*, pilih menu “Add Documents” dan masukkan folder `/data/kurikulum/`.

Hasil:

Sekarang AI bisa membaca buku sekolah dan menjawab seperti pustakawan digital!

Uji coba:

“Apa tujuan pembelajaran Python di kelas XI SMK?”

AI akan menjawab berdasarkan isi dokumen yang kamu unggah.

Aktivitas 4 — Deploy AI di Server Sekolah

Tujuan:

Menjalankan *AI Sekolah Cerdas* agar bisa digunakan semua siswa di lab lewat jaringan LAN.

Langkah-langkah:

1. Siapkan server Ubuntu 24.04.
2. Instal *Docker* dan *Docker Compose*.
3. Buat file `docker-compose.yml` seperti ini:

```
services:
  ollama:
    image: ollama/ollama
    ports:
      - "11434:11434"
  openwebui:
    image: openwebui/openwebui
    ports:
      - "3000:8080"
    environment:
      - OLLAMA_API_BASE=http://localhost:11434
```

4. Jalankan:

```
docker compose up -d
```

5. Coba akses dari komputer lab dengan alamat:

👉 `http://192.168.0.10:3000`

Hasil:

Sekarang seluruh siswa di lab bisa menggunakan AI sekolah lewat browser lokal! Kalian sudah membuat *mini ChatGPT* versi sekolah sendiri — **tanpa internet!**

Aktivitas 5 — Uji dan Evaluasi Sistem AI

Tujuan:

Menguji kemampuan AI sekolah menjawab pertanyaan dan memberi saran belajar.

Langkah-langkah:

1. Siapkan 10 pertanyaan dari berbagai pelajaran (IPA, PPKN, Matematika, dll).
2. Minta AI menjawab semuanya lewat *OpenWebUI*.
3. Catat hasilnya: mana yang benar, mana yang salah, dan mana yang perlu diperbaiki.
4. Berdiskusi bersama tim tentang penyebab kesalahan AI.
5. Perbarui dokumen atau model jika perlu.

Contoh pertanyaan:

“Apa fungsi mikrokontroler dalam sistem IoT?”

“Jelaskan pengertian pancasila sebagai ideologi negara.”

Hasil:

Kamu bisa menilai seberapa akurat AI sekolahmu bekerja.

Kalau salah, berarti waktunya *update dataset* dan *retrain model*!

Catatan

Lima aktivitas di atas membuat kamu benar-benar jadi **pembangun sistem AI nyata**.

Kamu belajar mengatur data, melatih model, menjalankan sistem RAG, dan men-deploy AI lokal dengan teknologi *open source*. Yang kamu lakukan sama seperti yang dilakukan *AI engineer* profesional di dunia kerja!

Hal Penting

- Gunakan *Python* versi terbaru (3.10+).
- Simpan semua file proyek di folder `ai_sekolah_cerdas/`.
- Pastikan server sekolah punya RAM minimal **8GB** agar *Ollama* bisa berjalan lancar.
- Selalu cek etika dan keamanan data — **jangan masukkan dokumen rahasia sekolah**.

Rangkuman

Bab ini mengajarkan bagaimana **semua bagian AI yang telah dipelajari** disatukan menjadi satu sistem utuh bernama *AI Sekolah Cerdas*. Mulai dari **input data**, **model AI**, **RAG (Retrieval-Augmented Generation)**, **deployment**, hingga **evaluasi**, semuanya bekerja seperti mesin yang saling terhubung. Data dari buku sekolah dan catatan guru menjadi *bahan bakar*, model AI menjadi *otak*, RAG menjadi *mesin pencari pintar*, dan *deployment* membuat sistem bisa digunakan seluruh sekolah. Semua teknologi yang digunakan berbasis

open source, seperti *Python*, *Pandas*, *Keras*, *Neural Network*, *Ollama*, dan *OpenWebUI*, agar sekolah bisa mandiri tanpa tergantung layanan berbayar.

Proyek ini bukan hanya soal *coding*, tapi juga tentang **kerjasama, tanggung jawab, dan kreativitas siswa SMK**. Setiap anggota tim belajar menjalankan peran — ada yang mengatur data, ada yang melatih model, ada yang mengatur server, dan ada yang menguji hasilnya. Dari sini siswa belajar cara berpikir seperti *AI engineer* sungguhan: menganalisis masalah, menulis solusi, lalu mengujinya di sistem nyata. Yang menarik, semua proses dilakukan secara lokal, jadi **AI sekolah tetap aman, cepat, dan bisa berjalan tanpa internet**. Dengan *Ollama* dan *OpenWebUI*, kalian telah membangun mini-lab AI yang bermanfaat untuk seluruh warga sekolah.

Melalui proyek ini, kalian membuktikan bahwa **siswa SMK bisa menciptakan teknologi setara kampus besar dunia**. *AI Sekolah Cerdas* bukan sekadar chatbot, tapi **asisten digital yang membantu guru dan siswa belajar lebih efektif**. Guru terbantu dalam menjelaskan materi, siswa terbantu saat mencari penjelasan tambahan. Yang paling penting, semua dilakukan dengan prinsip **etika, keamanan data, dan kemandirian teknologi**. Dengan semangat *open source* dan kerja tim, kalian telah membuka jalan menuju masa depan pendidikan digital di Indonesia.

Latihan Soal

A. Benar atau Salah (5 Soal)

1. AI Sekolah Cerdas hanya bisa dijalankan jika terkoneksi dengan internet.
Jawaban: Salah
2. *RAG (Retrieval-Augmented Generation)* digunakan agar AI bisa mencari jawaban dari dokumen sekolah.
Jawaban: Benar
3. *Deployment lokal* berarti AI dijalankan di server sekolah, bukan di cloud.
Jawaban: Benar
4. *Keras* adalah library *open source* untuk membuat *neural network* di *Python*.
Jawaban: Benar
5. Model AI bisa belajar tanpa data sama sekali.
Jawaban: Salah

B. Pilihan Ganda (10 Soal)

1. Komponen pertama dalam membangun AI Sekolah Cerdas adalah ...
 - a. Model
 - b. Input Data
 - c. Deployment

d. Evaluasi

Jawaban: b

2. *Neural Network* digunakan untuk ...

- a. Membuat jaringan komputer
- b. Meniru cara berpikir otak manusia
- c. Menyimpan dokumen sekolah
- d. Mengatur jadwal pelajaran

Jawaban: b

3. Tools *open source* untuk menjalankan AI secara lokal adalah ...

- a. Microsoft Azure
- b. Ollama + OpenWebUI
- c. Google Cloud AI
- d. ChatGPT Online

Jawaban: b

4. Fungsi *Pandas* dalam proyek AI adalah ...

- a. Menggambar grafik
- b. Mengelola dan membaca data
- c. Membuat chatbot
- d. Mengatur jaringan

Jawaban: b

5. *RAG system* berfungsi seperti ...

- a. Mesin pencari pintar
- b. Aplikasi game
- c. Editor video
- d. Sistem login sekolah

Jawaban: a

6. Deployment AI di sekolah biasanya dilakukan menggunakan ...

- a. Ubuntu + Docker
- b. Windows Notepad
- c. Excel Sheet
- d. Android Emulator

Jawaban: a

7. Evaluasi sistem AI dilakukan untuk ...

- a. Memperindah tampilan
- b. Menguji ketepatan jawaban
- c. Membuat file baru
- d. Menghapus dokumen lama

Jawaban: b

8. Apa keuntungan utama menggunakan *open source*?

- a. Gratis dan bisa dimodifikasi
- b. Hanya bisa dipakai online

- c. Butuh lisensi mahal
- d. Tidak bisa digunakan di sekolah

Jawaban: a

9. Dalam proyek AI Sekolah Cerdas, data dari buku dan catatan guru disebut ...
- a. Output
 - b. Input Data
 - c. Model
 - d. Deployment

Jawaban: b

10. *Python* dipilih dalam proyek AI karena ...
- a. Sulit dipelajari
 - b. Tidak bisa dipakai di Ubuntu
 - c. Mudah, kuat, dan open source
 - d. Hanya bisa dijalankan online

Jawaban: c

C. Isian Singkat (5 Soal)

1. Model AI yang dibangun dengan *Keras* dan *Neural Network* disebut _____.

Jawaban: Model Pembelajaran Mesin (*Machine Learning Model*)

2. Sistem AI sekolah bisa diakses lewat browser menggunakan alamat _____.

Jawaban: <http://localhost:3000> atau IP server LAN

3. Data yang digunakan AI sebagai sumber jawaban disebut _____.

Jawaban: *Dataset*

4. Library Python yang digunakan untuk mengolah data tabel adalah _____.

Jawaban: *Pandas*

5. Proses mengecek kebenaran hasil AI disebut tahap _____.

Jawaban: Evaluasi

D. Soal Eksplorasi (3 Soal)

Soal 1 – Mini Chatbot Sekolah

Kamu diminta membuat *mini chatbot* untuk menjawab pertanyaan seputar sekolah, seperti “Siapa kepala sekolah?” atau “Apa visi misi SMK kita?”.

Gunakan *OpenWebUI* dan *Ollama* dengan dokumen sekolah (profil, visi-misi, struktur organisasi) sebagai *dataset*.

Langkah:

- 1. Siapkan file teks [profil_sekolah.txt](#).
- 2. Jalankan *Ollama* model [gemma2:2b](#).

3. Tambahkan folder data ke *RAG*.
4. Tes pertanyaan dari temanmu lewat *chat interface*.

Kunci Jawaban (contoh hasil):

“Kepala sekolah adalah Ibu Sri Rahayu, visi sekolah adalah mencetak lulusan unggul di bidang teknologi.”

Soal 2 – Evaluasi Jawaban AI

Kamu menemukan bahwa AI sekolah sering menjawab salah saat ditanya soal Matematika.

Tugasmu adalah **mengevaluasi dan memperbaikinya**.

Langkah:

1. Cek folder *dataset*, pastikan buku Matematika sudah diunggah.
2. Periksa log di *OpenWebUI*: apakah file terindeks dengan benar?
3. Tambahkan file baru *materi_matematika.txt* lalu re-run RAG.
4. Tes ulang pertanyaan seperti “Apa rumus luas segitiga?”.

Kunci Jawaban (contoh hasil):

Setelah update dataset, AI menjawab benar: “Luas segitiga = $\frac{1}{2} \times \text{alas} \times \text{tinggi}$.”

Soal 3 – AI Rekomendasi Buku Pelajaran

Sekolah ingin punya sistem AI yang bisa memberi rekomendasi buku pelajaran untuk tiap jurusan (TKJ, RPL, Akuntansi).

Tugasmu adalah membangun sistem *rekomendasi sederhana* menggunakan *Python + Pandas*.

Langkah:

1. Buat file *buku.csv* berisi: *Judul, Jurusan, Kategori*.
2. Gunakan kode berikut:

```
import pandas as pd
data = pd.read_csv("buku.csv")
jurusan = input("Masukkan jurusan: ")
print(data[data["Jurusan"] == jurusan])
```

3. Jalankan program dan masukkan nama jurusanmu.

Kunci Jawaban (contoh hasil):

Jika jurusan: “RPL”, hasil:
Buku 1: *Pemrograman Python Dasar*
Buku 2: *Algoritma dan Struktur Data*

Catatan

Soal-soal ini melatih kamu memahami **alur kerja sistem AI Sekolah Cerdas** — dari *Input Data*, *Model*, *RAG*, sampai *Deployment dan Evaluasi*. Kamu belajar membangun AI yang **open source, etis, dan berguna untuk sekolah**. Dengan latihan ini, kamu bukan hanya pengguna AI, tapi **pencipta AI lokal yang mandiri**.

Tugas Proyek

Proyek 1 — Chatbot Jadwal Sekolah Otomatis

Deskripsi:

Buat chatbot yang bisa menjawab pertanyaan seputar jadwal pelajaran sekolah, seperti “Hari ini pelajaran apa?” atau “Jam ke-2 kelas TKJ belajar apa?”

Langkah-langkah:

1. Siapkan file `jadwal.csv` berisi kolom: *Hari, Jam, Kelas, Mata Pelajaran, Guru*.
2. Gunakan *Pandas* untuk membaca file tersebut.
3. Jalankan sistem *RAG lokal* dengan *Ollama + OpenWebUI*.
4. Uji chatbot di browser lokal dengan pertanyaan dari siswa.

Contoh pseudocode:

```
import pandas as pd
data = pd.read_csv("jadwal.csv")
hari = "Senin"
print(data[data["Hari"] == hari])
```

Kunci Jawaban (contoh hasil):

Chatbot menjawab:

“Kelas XI TKJ hari Senin belajar Jaringan Komputer di jam ke-2, guru: Pak Budi.”

Catatan:

Kamu bisa menambahkan *RAG plugin* agar AI bisa membaca jadwal langsung dari file.

Proyek 2 — AI Peningat Tugas Siswa

Deskripsi:

Buat sistem AI yang bisa memberi pengingat tugas sekolah berdasarkan tanggal dan mata pelajaran.

Langkah-langkah:

1. Buat file `tugas_siswa.csv` berisi kolom: *Tanggal, Pelajaran, Deskripsi*.
2. Gunakan *Python* untuk membaca file dan menampilkan tugas sesuai tanggal hari ini.
3. Tambahkan fitur *voice reminder* (suara teks) dengan library *pyttsx3*.
4. Integrasikan ke *OpenWebUI* agar bisa diakses lewat browser lokal.

Contoh pseudocode:

```
import pandas as pd
from datetime import date
data = pd.read_csv("tugas_siswa.csv")
hari_ini = date.today().strftime("%d-%m-%Y")
print(data[data["Tanggal"] == hari_ini])
```

Kunci Jawaban:

“Hari ini, kamu punya tugas Matematika: Kerjakan latihan halaman 23.”
AI bisa membaca otomatis data yang ada di file CSV lokal.

Proyek 3 — Pustakawan AI Sekolah

Deskripsi:

AI ini membantu siswa mencari buku berdasarkan judul, kategori, atau penulis.

Langkah-langkah:

1. Buat dataset `buku_sekolah.csv` dengan kolom: *Judul, Penulis, Kategori, Rak*.
2. Jalankan RAG di *Ollama + OpenWebUI*.
3. Tambahkan folder `/data/buku/` sebagai sumber data untuk AI.
4. Tanyakan lewat *chat interface*:
“Tunjukkan buku tentang jaringan komputer.”

Contoh setting OpenWebUI:

- Model: `gemma3:1b`
- Folder sumber RAG: `/data/buku/`

Kunci Jawaban (contoh hasil):

“Buku ‘*Dasar Jaringan Komputer*’ oleh Budi Santosa, tersedia di Rak B, kategori Teknologi.”

Bonus Challenge:

Tambahkan fitur *rekomendasi buku serupa* berdasarkan kategori.

Proyek 4 — AI Evaluator Nilai Siswa

Deskripsi:

AI ini membaca data nilai siswa dan memberi umpan balik otomatis.

Contohnya: siswa bisa bertanya, “Bagaimana nilai rata-rata saya di semester ini?”

Langkah-langkah:

1. Siapkan file `nilai_siswa.xlsx` berisi: *Nama, Mapel, Nilai*.
2. Gunakan *Pandas* untuk menghitung nilai rata-rata tiap siswa.
3. Tambahkan logika sederhana:
 - o Jika nilai > 80 → “Sangat Baik”
 - o Jika 70–79 → “Cukup”
 - o Jika < 70 → “Perlu bimbingan”
4. Tampilkan hasil lewat *OpenWebUI Chatbot*.

Contoh pseudocode:

```
import pandas as pd
data = pd.read_excel("nilai_siswa.xlsx")
rata = data.groupby("Nama")["Nilai"].mean()
print(rata)
```

Kunci Jawaban:

“Nilai rata-rata kamu 82, kategori: Sangat Baik. Pertahankan semangat belajarnya!”

Proyek 5 — Sistem AI Rekomendasi Pelajaran Tambahan

Deskripsi:

Buat sistem AI yang memberi rekomendasi pelajaran tambahan berdasarkan minat dan nilai siswa.

Langkah-langkah:

1. Buat dataset `minat_siswa.csv` berisi kolom: *Nama, Minat, Nilai Rata-rata*.
2. Buat logika sederhana di *Python*:
 - o Jika minat “teknologi” dan nilai > 80 → rekomendasikan *AI & Coding Club*.
 - o Jika minat “bisnis” dan nilai > 75 → rekomendasikan *Kewirausahaan Digital*.
3. Integrasikan dengan *Ollama RAG* agar siswa bisa tanya langsung.

Contoh pseudocode:

```
if minat == "teknologi" and nilai > 80:
    print("Rekomendasi: Gabung kelas AI & Coding Club.")
```

Kunci Jawaban:

“Berdasarkan minatmu di bidang teknologi dan nilai rata-rata 85, kamu direkomendasikan bergabung dengan *AI & Coding Club*.”

Catatan

Kelima proyek ini melatih kamu menjadi **AI Builder sejati di sekolah**.

Semua proyek bisa dijalankan **tanpa internet**, karena memakai teknologi *open source* seperti *Python*, *Pandas*, *Keras*, *Neural Network*, *Ollama*, dan *OpenWebUI*.

Kamu tidak hanya belajar membuat sistem AI, tapi juga belajar:

- **Berpikir kritis,**
- **Berkerja sama dalam tim,**
- **Membangun solusi nyata untuk sekolah.**

“AI bukan hanya soal teknologi, tapi soal bagaimana kamu membuatnya berguna bagi orang lain.”

Institut Teknologi Tangerang Selatan (ITTS) dan Penulis.

Para penulis adalah **dosen aktif di Institut Teknologi Tangerang Selatan (ITTS)**, sebuah **institusi pendidikan tinggi berbasis teknologi** yang berperan aktif dalam pengembangan **solusi digital** dan *kecerdasan buatan (AI)*. Tidak hanya mengajar, mereka juga merupakan **praktisi dan inovator** yang menghasilkan **karya-karya nyata di bidang teknologi mandiri**.

Berlokasi di **Komplek Komersial BSD, Serpong Utara, Tangerang Selatan**, ITTS hadir sebagai **pusat inovasi** yang mengintegrasikan **pembelajaran, penelitian, dan pengembangan teknologi secara menyeluruh**. Fokus utama ITTS mencakup bidang strategis seperti *AI, Machine Learning, Deep Learning*, jaringan, *cloud*, keamanan siber, pemrograman, sistem informasi, dan multimedia.

ITTS menawarkan lingkungan akademik yang dinamis dan kolaboratif. Setiap minggu, diselenggarakan berbagai *workshop, demo teknologi*, seminar, hingga *webinar*—mayoritas **gratis dan tersedia secara terbuka di YouTube**. Ini menciptakan **kultur belajar yang aktif dan relevan dengan perkembangan industri**.

Fasilitas ITTS menunjang kegiatan riset dan inovasi secara optimal. Lab komputer dilengkapi *GPU RTX 4060* untuk eksperimen *AI*, serta **laboratorium teknologi dengan puluhan server, perangkat IoT, sistem komunikasi radio, dan infrastruktur jaringan jarak jauh**. Dari ekosistem ini lahir **produk-produk mandiri** seperti **ChatGPT versi lokal** untuk institusi pendidikan dan **sistem perpustakaan digital yang dapat diakses tanpa koneksi internet**.

Daya saing ITTS juga tercermin dari karakter dosen dan mahasiswa yang tidak hanya akademik, tetapi juga **produktif secara praktis**. Mahasiswa didorong aktif menjadi **narasumber, penulis, dan peneliti** sejak dini. Banyak diantaranya telah menghasilkan **karya buku, artikel ilmiah, serta terlibat langsung dalam proyek-proyek teknologi**.

Kolaborasi strategis dengan berbagai mitra industri—seperti *IDCloudHost, Ubuntu, XecureIT, Dinas KOMINFO Tangerang Selatan*, berbagai *ISP*, dan komunitas IT nasional—menjadi **jembatan nyata antara kampus dan dunia profesional**. Hasilnya, **lebih dari 75% mahasiswa ITTS telah bekerja bahkan sebelum lulus, dan hampir seluruh lulusan langsung terserap industri**.

ITTS bukan hanya tempat belajar, tapi tempat mencipta. Dosen sebagai **penulis dan praktisi**, mahasiswa sebagai **peneliti dan kontributor**—semuanya berperan **membangun ekosistem teknologi yang mandiri dan berdaya saing tinggi**.

Institut Teknologi Tangerang Selatan (ITTS)

Komplek Komersial BSD

Jl. Raya Serpong No. Kav. 9, Lengkong Karya, Serpong Utara

Kota Tangerang Selatan, Banten 15331

☎ (+62) 877-7277-1775

✉ info@itts.ac.id

🌐 www.itts.ac.id